



UNIVERSITÀ DEGLI STUDI DI PADOVA
DIPARTIMENTO DI MATEMATICA “TULLIO LEVI-CIVITA”
MASTER THESIS IN COMPUTATIONAL FINANCE
CLASS LM-16

Statistical Arbitrage: Crude Oil Futures Market Pairs Trading

SUPERVISOR:
Prof. Claudio Fontana
Università degli Studi di Padova

CANDIDATE:
Tommaso Zanatta
ID: 2103411

Academic Year 2024/2025

Acknowledgements

I would like to express my deepest gratitude to Professor Claudio Fontana for his invaluable guidance, support, and the academic rigor he brought throughout the development of this thesis. His insights and lectures are what led me to choose this topic for the thesis, while ultimately being the person that convinced me to enroll in this Master's.

Special thanks go to the Department of Mathematics at the University of Padua for offering an intellectually stimulating and supportive environment, which played a crucial role in my academic development.

This journey would not have been as fulfilling and intellectually rewarding without the incredible people I had the privilege to work alongside.

A heartfelt thank you goes to the teams at IG Solutions and GateDev, whose mentorship, friendship, and technical challenges have played a meaningful role in shaping my growth, both professionally and personally.

Lastly, I want to acknowledge those who, through discussions, feedback, or even simple questions, contributed directly or indirectly to this work.

*To my parents, Barbara and Simone, and to
my brother, Edoardo: your support has been
the driving force behind my studies.
In every setback, you were there to lift me up.
In every success, you were the first to celebrate.
Your faith in me has been my foundation.
From the bottom of my heart: "Grazie".*

*To the entirety of my family.
To my best friends, Andrea and Giulio, for
supporting me and being there every step of
the way, relentlessly driving me towards this
accomplishment even without knowing it.*

*And lastly to me, as a reminder and a promise
for the future.*

Contents

1	Introduction	3
1.1	Introduction	3
1.2	Motivation and Background	4
1.3	Cointegration Intuition	6
1.4	Contributions and Extensions	6
1.5	Thesis Structure	7
2	Background	9
2.1	Futures Contracts	9
2.1.1	Brent Crude Oil Futures	9
2.1.2	West Texas Intermediate (WTI) Crude Oil Futures	10
2.1.3	Dubai Crude Oil Futures	10
2.1.4	Shanghai Crude Oil Futures	11
2.2	Ornstein-Uhlenbeck Hidden Markov Model (OU-HMM)	11
2.2.1	Regime Switching in Cointegration Spread	11
2.2.2	Ornstein-Uhlenbeck Process (OU)	12
2.2.3	Hidden Markov Model (HMM)	14
2.2.4	Hybrid Model (OU-HMM)	18
2.2.5	Model Discretization	20
2.2.6	Theoretical Parameter Estimation	22
2.3	Trading Strategies	28
2.3.1	Strategy: Plain Vanilla	29
2.3.2	Strategy: Probability Interval (ProbI)	30
2.3.3	Strategy: Realized Increment (RI)	32
2.3.4	Strategy: Prediction Interval (PredI)	33
2.3.5	Strategy: Predicted Increment (PI)	34
2.4	Performance Measurement	35
2.4.1	Risk Analysis	38

3	Methodology	41
3.1	Data Collection	41
3.1.1	Time Series and Timeframes	41
3.1.2	Data Preprocessing	42
3.2	Cointegration Testing	43
3.2.1	Unit Root Tests	43
3.2.2	VAR Model Estimation and Lag Selection	44
3.2.3	Johansen Trace Test	46
3.2.4	Cointegration Spread Construction	47
3.3	OU-HMM Implementation	51
3.3.1	Model Initialization and Clustering	51
3.4	Trading Strategy Implementation	60
3.4.1	Benchmark Strategy: Buy and Hold	60
3.4.2	Plain Vanilla Strategy Implementation	60
3.4.3	Probability Interval Strategy Implementation	62
3.4.4	Realized Increment Strategy Implementation	63
3.4.5	Prediction Interval Strategy Implementation	64
3.4.6	Predicted Increment Strategy Implementation	65
4	Results	67
4.1	Trading Strategies Empirical Performance	67
4.2	Performance Measures	68
4.3	Risk Analysis	70
4.4	Results Comparison	72
5	Conclusion	75
A	Code Listings	77
A.1	Data Preprocessing	77
A.2	OUHMM Model	82
A.3	Trading Strategies	97
	Bibliography	111

Chapter 1

Introduction

ABSTRACT. This thesis enhances statistical arbitrage strategies in international crude oil futures markets, extending the analytical framework of Fanelli, Fontana, and Rotondi (2023) by introducing an Ornstein-Uhlenbeck Hidden Markov Model (OU-HMM). This model addresses the limitations of static cointegration by allowing the arbitrage spread's parameters (mean, reversion speed, volatility) to shift across unobserved market regimes. Focusing on Brent, WTI, Dubai, and Shanghai crude oil futures, a key contribution is the extended dataset through March 2025, capturing recent market dynamics. Rigorous testing confirmed three cointegrating relationships. The OU-HMM successfully identified three distinct market regimes. Back-tested from October 2023 to March 2025, a persistent bull market period, the Buy and Hold benchmark yielded the highest return (16.30%). However, the study's market-neutral strategies, notably Realized Increment (RI) and Prediction Interval (PredI), excelled as risk-reduction tools. These strategies demonstrated substantially lower drawdowns, VaR, and CVaR compared to Buy and Hold, highlighting their value in capital preservation and diversification during a bull market.

1.1 Introduction

This thesis investigates the application of statistical arbitrage strategies in the context of international crude oil futures markets, building upon and extending the analytical framework developed by Fanelli, Fontana, and Rotondi (2023). Statistical arbitrage, unlike classical arbitrage which assumes the possibility of obtaining risk-free profit opportunities, relies on exploiting persistent but locally temporary deviations from a theoretical long-term equilibrium among related financial assets. These deviations, often caused by short-term market inefficiencies or noise, are expected

to revert over time, allowing the trader to profit from the convergence back to equilibrium. However, due to the inherent uncertainty regarding the timing and magnitude of this mean reversion, statistical arbitrage strategies are probabilistic in nature and require robust statistical modeling to identify and quantify mispricings.

1.2 Motivation and Background

The underlying principle that enables such strategies is the concept of *cointegration* among asset prices. **Cointegration** is a statistical property of a collection of time series variables which indicates that a linear combination of them has a stable mean and variance over time, even though the individual series may be non-stationary. This implies the existence of a stable long-run equilibrium relationship, despite short-term fluctuations. In the context of futures markets, especially those tied to the same underlying commodity (such as crude oil), the assumption of cointegration is economically justified by the existence of common stochastic trends, arbitrage constraints, and shared macroeconomic drivers. For example, despite being traded on different exchanges and under varying specifications, Brent and WTI crude oil futures are fundamentally linked through physical supply chains, transportation infrastructure, and global demand shocks. (Brandt 2010; Hull 2017)

This work considers four major benchmark crude oil futures: Brent, West Texas Intermediate (WTI), Dubai, and the Shanghai crude oil futures introduced by the Shanghai International Energy Exchange (INE) in March 2018. While the long-term cointegrated relationship among Brent and WTI has been widely studied and empirically validated, more recent studies, such as Fanelli, Fontana, and Rotondi (2023), provide preliminary evidence that Shanghai futures also tend to exhibit cointegration with these benchmarks. This thesis builds upon such findings, evaluating the persistence and robustness of these relationships over a broader and more recent time horizon, extending the dataset through March 2025. This extension allows us to incorporate the impact of significant geopolitical and macroeconomic developments, such as post-COVID economic easing, energy supply chain disruptions, inflationary pressures, and recent geopolitical tensions, all of which could materially influence pricing dynamics and arbitrage conditions.

A key limitation in classical cointegration-based models is the assumption of constant long-run relationships. Namely, they assume a static cointegration vector, which implicitly assumes that the equilibrium relationship among assets remains unchanged through time. However, financial markets are dynamic and exhibit regime-dependent behavior due to changing liquidity, volatility clustering,

macroeconomic shocks, and shifts in investor sentiment. Empirical evidence suggests that cointegration relationships can experience structural breaks, slow drifts, or sudden realignments. In such settings, static models can lead to incorrect inference, suboptimal strategy performance, or even arbitrage losses.

To address this limitation, the methodology adopted in this thesis models the cointegration spread as a stochastic process with regime-switching behavior. A **stochastic process** is a mathematical object defined as a collection of random variables. In this context, it allows for the modeling of the cointegration spread as a dynamic entity that evolves over time, capturing the inherent uncertainty and variability in financial markets. Specifically, a hybrid framework is introduced: an Ornstein-Uhlenbeck (OU) process is driven by an unobserved (hidden) Markov chain, which governs the regime under which the process evolves. An **Ornstein-Uhlenbeck process** is a type of stochastic process that describes the evolution of a variable that tends to revert to its long-term mean over time. It is characterized by its mean-reversion property and is commonly used in financial modeling to capture the dynamics of interest rates, stock prices, and other financial variables. On the other hand, a **hidden Markov chain** is a statistical model that represents systems which are assumed to be a Markov process with unobservable (hidden) states. In this context, it allows for the modeling of different market regimes that influence the behavior of the cointegration spread. Each hidden regime corresponds to a distinct set of OU parameters (mean, reversion speed, and volatility), capturing time-varying dynamics in the spread's behavior. The resulting model, referred to as the OU-HMM (Ornstein-Uhlenbeck Hidden Markov Model), is thus capable of adapting to evolving market conditions while preserving the tractability of a closed-form stochastic process.

From a methodological standpoint, the OU-HMM framework merges continuous-time stochastic modeling with discrete-time state transitions, making it especially suitable for real-world financial applications where asset prices are observed at discrete intervals. Since the regimes themselves are latent and unobservable, a filtering algorithm is required to infer the posterior probabilities of the current regime and estimate the model parameters recursively. For this purpose, a filter-based Expectation-Maximization (EM)¹ algorithm is employed, allowing for efficient parameter estimation and regime identification from historical data. This recursive approach ensures that the model remains adaptive to incoming information, making it well-suited for online updating and potentially real-time trading applications. On a further note, the implementation must discretize the continuous-time model to handle

¹Consistently with the work of Fanelli, Fontana, and Rotondi (2023), the **EM algorithm** is employed to iteratively update the parameter estimates and improve the model fit.

the practical constraints of daily futures price observations. This discretization is done to preserve the regime-switching structure while enabling numerical optimization and likelihood maximization techniques to be applied efficiently.

1.3 Cointegration Intuition

The economic intuition behind cointegration in crude oil futures markets arises from their common exposure to global supply and demand conditions, inventory cycles, and expectations about future spot prices. Despite being traded in different locations under varying delivery and contract conditions, benchmark crude oil futures share the same fundamental underlying: the global price of crude oil. Arbitrage mechanisms, whether through storage, transport, or financial derivatives, serve to synchronize prices over the long run, ensuring that no persistent arbitrage opportunity can survive indefinitely. This is widely recognized as the *law of one price*, foundation on which the concept of Efficient Market Hypothesis (EMH) is built. (Fama 1970)

Statistically, cointegration captures this notion by identifying whether a stationary linear combination exists among non-stationary price series. When such a relationship holds, the spread S_t derived from this combination can be interpreted as a deviation from long-run equilibrium. This spread can then be modeled and traded using mean-reversion-based (Uhlenbeck and Ornstein 1930) strategies. However, if the cointegration vector is unstable over time, or if the spread exhibits structural breaks, traditional methods may fail to detect profitable opportunities or may generate misleading signals (Ang and Timmermann 2012; J. Bulla and I. Bulla 2006). The OU-HMM framework directly addresses this issue by allowing the parameters of the mean-reverting spread process to evolve according to latent regimes, thus accommodating the time-varying nature of economic and statistical relationships among assets. Specifically, we regard the OU process as the key to model local dynamics of the spread, while the HMM captures the higher-level regime-switching behavior that influences these dynamics.

1.4 Contributions and Extensions

This thesis offers several key contributions and extensions. Firstly, a primary contribution is the extension of the dataset. This work incorporates nearly two additional years of observations, extending the sample period from the original study's timeframe of 2018 to mid-2023 up to March 2025. This temporal expansion is crucial as it allows for a more comprehensive assessment of arbitrage relationships

under recent and diverse market dynamics. Specifically, it captures the impacts of geopolitical shocks and post-COVID macroeconomic stabilization, factors that could significantly influence crude oil pricing relationships and the efficacy of statistical arbitrage conditions over time. Secondly, while the original study focused primarily on identifying and validating statistical arbitrage opportunities via an Ornstein-Uhlenbeck Hidden Markov Model (OU-HMM)-modulated spread, this thesis places greater emphasis on the reproducibility and scalability of the strategy under evolving market conditions. This involves a detailed examination of the OU-HMM model's robustness and interpretability, crucial aspects for its practical deployability in a dynamic financial environment potentially laying the groundwork for a live implementation in a real trading system, moving beyond theoretical validation to address the exigencies of continuous market operation. Thirdly, this work meticulously lays out a comprehensive pipeline for the implementation of the OU-HMM-based statistical arbitrage strategy. This pipeline encompasses every critical stage, starting from data preprocessing and rigorous cointegration testing, through dynamic parameter estimation and regime inference facilitated by the OU-HMM, and culminating in thorough backtesting of trading performance assessing the performance of various trading strategies.

1.5 Thesis Structure

The remainder of this thesis is meticulously structured to guide the reader through the process of developing and evaluating critically the hypothesis laid out.

Chapter 2 provides the foundational theoretical context essential for understanding the subsequent analysis. It thoroughly introduces the financial instruments involved, while also establishing the theoretical underpinnings of statistical arbitrage, drawing inspiration from pairs trading strategies that exploit temporary deviations from long-term equilibrium. Detailed descriptions of the various statistical arbitrage strategies implemented in this thesis are provided. Furthermore, it comprehensively covers the performance measurement methodologies, such as Cumulative Return, Maximum Drawdown, Sharpe Ratio, and Calmar Ratio, and the critical risk analysis techniques, specifically Value at Risk (VaR), Conditional Value at Risk (CVaR), and Kernel Density Estimation (KDE) via Monte Carlo Simulation. Chapter 3 elaborates on the rigorous analytical framework employed throughout the thesis. It outlines the comprehensive cointegration testing process, which includes unit root tests (ADF, PP, DF-GLS, KPSS). This is followed by Vector Autoregressive (VAR) model estimation and lag selection using the Schwarz Information Criterion (SIC). Crucially, it details

the application of the Johansen Trace Test to identify cointegrating relationships indicating a robust long-term equilibrium. This foundational analysis leads to the construction of the cointegration spread process (St), which is then modeled using an Ornstein-Uhlenbeck Hidden Markov Model (OU-HMM). The chapter explains how this regime-switching process captures time-varying cointegration regimes and how its parameters are dynamically estimated via a filter-based Expectation Maximization (EM) algorithm, ensuring the model is constantly tuned to market conditions. Chapter 4 details the empirical performance and risk analysis of the trading strategies developed in the study. It first presents performance measures, including cumulative return, maximum drawdown, Sharpe ratio, and Calmar ratio, for all six strategies. Next, a comprehensive risk analysis is presented, utilizing Monte Carlo-based Kernel Density Estimation (KDE), Value at Risk (VaR), and Conditional Value at Risk (CVaR). The chapter then provides a results comparison that contrasts these findings with prior literature, notably Fanelli, Fontana, and Rotondi (2023). This comparison highlights that while the active strategies in this thesis functioned primarily as risk-reduction tools in a bull market, prior studies found them to be alpha-generating endeavors in declining markets, underscoring that the efficacy of statistical arbitrage is deeply correlated with the prevailing market regime. Ultimately, Chapter 5 provides the conclusion of the thesis. It synthesizes the key findings, emphasizing the successful development and implementation of statistical arbitrage strategies in international crude oil futures markets using an Ornstein-Uhlenbeck Hidden Markov Model (OU-HMM). It reiterates the model's ability to adapt to evolving market conditions by allowing spread parameters to shift across unobserved market regimes.

Chapter 2

Background

2.1 Futures Contracts

A futures contract is a standardized agreement to buy or sell an asset at a predetermined price at a specified time in the future. These contracts are traded on organized exchanges and are primarily used for hedging against price volatility or for speculative purposes by financial institutions, corporations, and individual investors. Brandt (2010) provides an in-depth econometric treatment of futures pricing and modeling.

The global crude oil market, due to its strategic importance and price volatility, represents one of the most liquid and actively traded futures markets in the world. Chevallier (2011)

The futures taken into consideration in this thesis are:

- Brent Crude Oil Futures
- West Texas Intermediate (WTI) Crude Oil Futures
- Dubai Crude Oil Futures
- Shanghai Crude Oil Futures

2.1.1 Brent Crude Oil Futures

Introduced in 1988, Brent futures, benchmarked against the ICE Brent Index, are a key trading instrument in the global oil market. Brent accounts for over 60% of the world's traded crude oil volumes, making it the most widely used reference price globally.

Brent contracts are typically cash-settled, but delivery is possible also via the Exchange for Physical (EFP) mechanism. The physical benchmark is based on a set of North Sea crude oils, with the ICE Brent Index representing the assessed spot price.

Linked contracts on platforms such as CME BB/BZ and Dubai EFS establish seamless connectivity between markets. CME BB/BZ refers to standardized crude oil futures traded on the Chicago Mercantile Exchange using the BB and BZ symbols, which are key benchmarks. Dubai EFS (Exchange Futures for Swaps) is a platform designed to facilitate efficient, cash-settled trading of crude oil in the Dubai market.

2.1.2 West Texas Intermediate (WTI) Crude Oil Futures

WTI futures were introduced in 1983 and are traded on the New York Mercantile Exchange (NYMEX), now part of CME Group, and are linked to the Cushing, Oklahoma storage hub for the delivery of physical crude oil.

With daily volumes frequently exceeding one million contracts, WTI futures represent one of the most liquid energy derivatives worldwide. Settlement happens primarily through physical delivery, although cash-settled variants also exist, including those offered by the Dubai Gold and Commodities Exchange (DGCX) under the DWTI ticker.

The contract's strong linkage to U.S. storage and pipeline logistics makes it highly sensitive to domestic inventory levels and infrastructure constraints.

2.1.3 Dubai Crude Oil Futures

Dubai crude, measured by S&P Global Platts since the 1980s, serves as a key benchmark for pricing Middle Eastern exports to Asia. The associated futures contract was launched by the Intercontinental Exchange (ICE) in 2011, while the CME version followed in 2014. Both are cash-settled using monthly average Platts assessments, referred to as MOPJ.

Dubai futures generally exhibit lower liquidity compared to Brent and WTI, but are strategically important due to their use in pricing long-term contracts between Gulf producers and Asian refiners. Typical daily volumes are in the tens of thousands of contracts.

These contracts are cash-settled, which lowers monthly volatility, and they are connected to global benchmarks via mechanisms such as the Dubai EFS and linked contracts on ICE and JPX.

2.1.4 Shanghai Crude Oil Futures

Launched in March 2018 by the Shanghai International Energy Exchange (INE), it was introduced to increase pricing power in Asia and reduce dependence on Brent and WTI benchmarks.

Similarly to the WTI contracts, the Shanghai contract is primarily physically settled, and delivery occurs at designated storage facilities within China. While international participation has grown, the market still remains partially segmented due to capital controls, regulatory restrictions, and currency exposure. These structural differences potentially reduce the strength of cointegration with other international benchmarks, a hypothesis that is empirically tested in the following sections.

2.2 Ornstein-Uhlenbeck Hidden Markov Model (OU-HMM)

In this section, we introduce the Ornstein-Uhlenbeck Hidden Markov Model (OU-HMM), a hybrid mathematical framework designed to model the statistical properties of mean-reverting processes, namely the cointegration spread S_t , a crucial element derived from the underlying asset relationships. This model provides a more flexible and empirically grounded framework for analyzing the behavior of mean-reverting spreads in financial time series, given a context in which underlying relationships among assets are themselves time-varying.

2.2.1 Regime Switching in Cointegration Spread

Once the cointegration relationships among the weekly closing prices of the four crude oil futures contracts considered in this thesis introduced in the previous section are established, we can proceed with the next steps. Through the Johansen Trace Test (Johansen 1991), we identify statistically significant cointegrating relationships that give rise to a spread process S_t defined as a linear combination of the log-prices of the futures contracts, with cointegration vector $\beta \in \mathbb{R}^4$.

Limitation of Theoretical Literature. Existing literature, such as Engle and Granger (1987), assumes that the linear combination characterizing the spread is constant over time, and the spread itself is a mean-reverting process fixed around a long-term equilibrium. This fundamental statistical property allows for the creation of pairs trading strategies, as well as other forms of statistical arbitrage. Although

assuming a constant cointegrating relationship is mathematically convenient and simplifies the modeling process, in practice market dynamics—such as structural breaks or shifts in underlying economic relationships—can yield time-varying cointegration relationships, rendering traditional pairs trading strategies based on fixed cointegration vectors suboptimal.

Empirical Studies. Empirical studies (see, e.g. Gregory and Hansen (1996), Baillie and Bollerslev (1996)) have documented that cointegrating relationships are not constant, rather, the dynamics are often characterized by regime-switching behavior. As previously introduced, factors such as policy changes, geopolitical tensions, market sentiment shifts, and variations in supply-demand dynamics introduce structural breaks in the underlying price series. As a consequence, the cointegration vector β and the long-term mean of the spread S_t may themselves be time-varying, resulting in regime-dependent properties of the spread process. Such limitations in cointegration models motivate the need for a more flexible and adaptive framework, one that allows the parameters governing the spread dynamics to change over time in response to unobservable latent states of the market.

Intuition In order to capture the time-varying, regime dependent dynamics of the cointegration spread, we construct a model that merges two powerful stochastic frameworks: the **Ornstein-Uhlenbeck (OU)** process and the **Hidden Markov Model (HMM)**. Each of these components serves a distinct purpose. The OU process provides a mathematically elegant formulation for modeling continuous and mean-reverting behavior, essential to capture the natural tendency of the cointegration spread S_t to revert toward equilibrium. On the other hand, the HMM introduces a probabilistic layer of regime-switching, allowing parameters of the OU process to shift according to unobserved latent states of the market. The intuition behind combining these two models lies in leveraging the OU process to model local dynamics, while the HMM governs higher-level shifts between qualitatively distinct market regimes. The resulting model is the OU-HMM, a flexible yet interpretable model capable of adapting to evolving structural relationships in financial time series data.

2.2.2 Ornstein-Uhlenbeck Process (OU)

The OU process is a continuous-time stochastic process commonly used to model mean-reverting behavior. It is defined by the following stochastic differential equation (SDE):

$$dS_t = \theta(\mu - S_t)dt + \sigma dW_t \quad (2.1)$$

where:

- S_t is the process at time t , in this case representing the cointegration spread.
- $\theta > 0$ represents the speed of mean reversion, indicating how quickly the process reverts to its long-term mean μ . The larger the θ , the faster the process is expected to return to the long term mean.
- μ is the long term mean of the process.
- $\sigma > 0$ is the volatility parameter, representing the intensity of random fluctuations around the mean.
- W_t is a standard Wiener process (Brownian motion), which introduces randomness into the system.

The term $\theta(\mu - S_t)$ is deterministic, meaning that it drives the process toward the long-term mean μ at a rate proportional to the distance from the mean. This term acts as a restoring force: when $S_t > \mu$ the drift becomes negative, while when $S_t < \mu$ the drift becomes positive, hence pushing the process back toward the mean. (Uhlenbeck and Ornstein 1930)

This process is the exact continuous-time formulation of an **Auto Regressive** model of order 1 (**AR (1)**), which is a discrete-time model that can be expressed as:

$$S_t = \mu + \phi(S_{t-1} - \mu) + \eta\epsilon_t, \quad \epsilon_t \sim \mathcal{N}(0, 1) \quad (2.2)$$

where $\phi = 1 - \theta\Delta t$, and ϵ_t is an i.i.d. standard normal random variable. This AR (1) formulation makes the OU process intuitive. Over small steps Δt , the OU process evolves similarly to this model, but offers significant advantages due to its continuous-time formulation. These include closed-form expressions for the transition density and likelihood functions for parameter estimation, not available in discrete time. Another important property of the OU process, shared with the AR model, is stationarity: assuming the process runs for infinite time, it will have constant mean μ , variance $\frac{\sigma^2}{2\theta}$, and an exponentially decaying autocorrelation function (ACF) given by:

$$\text{ACF}(k) = \left(e^{-\theta k \Delta t}\right) \quad (2.3)$$

This exponential decay is mathematically convenient, as it allows for analytical interpretation and estimation of the model parameters. Financially speaking, this shows that the speed of mean reversion is quantifiable and parameterized, allowing more accurate estimates of price dynamics and volatility.

More generally, the OU process is widely used in finance for modeling interest rates, volatility, and other mean-reverting phenomena.¹

Mean and Variance. The mean (conditional expectation) and variance of S_t given S_0 always under the assumption of stationarity, are given by:

$$\begin{aligned}\lim_{t \rightarrow \infty} \mathbb{E}[S_t] &= \mu \\ \lim_{t \rightarrow \infty} \text{Var}(S_t) &= \frac{\sigma^2}{2\theta}\end{aligned}$$

This clearly shows that, as $t \rightarrow \infty$, the process converges to a normal distribution and is by definition stationary:

$$S_t \sim \mathcal{N}\left(\mu, \frac{\sigma^2}{2\theta}\right) \quad (2.4)$$

Given the stationarity and normal distribution of the OU process, we can infer that the process is **ergodic** (Gardiner 1985; A. R. Barron and T. J. Barron 1994), meaning that time averages converge to ensemble averages. This property is crucial for statistical inference and parameter estimation, as it allows us to use historical data to estimate the parameters θ , μ , and σ in the SDE 2.1.

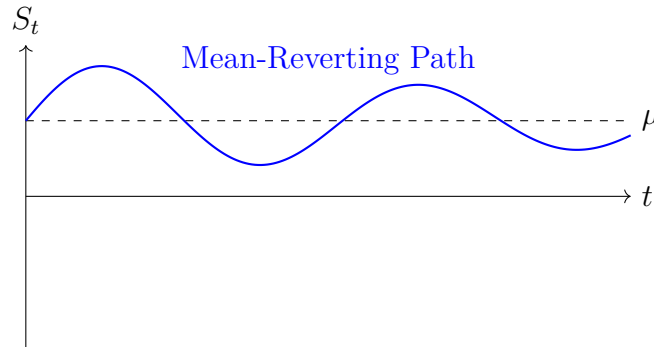


FIGURE 2.1. Illustration of an Ornstein-Uhlenbeck process with mean $\mu = 1$ and a decaying oscillatory path. The process exhibits mean-reverting behavior, oscillating around the long-term mean. The blue curve represents a sample path of the OU process, showing how it fluctuates around the mean over time.

2.2.3 Hidden Markov Model (HMM)

The Hidden Markov Model (HMM) is a statistical model and probabilistic framework widely used to model time series data in which the whole system's

¹For more financial applications such as the Vasicek model and option pricing, see Vasicek (1977), Lewis (2000)

dynamics are governed by unobserved (hidden) states. In contrast to traditional time series models that directly model observable variables, HMMs assume that the observed data are generated by a process whose dynamics are influenced by a latent² discrete-time Markov chain. These hidden states are not directly observable, but they influence the distribution of the observed variables, and the objective is to infer both the latent states and the parameters governing them. Rabiner (1989)

Hidden States. The model consists of a finite set of hidden states $\{Z_1, Z_2, \dots, Z_K\}$, where each state represents a distinct regime or structural condition of the system. These may include economically meaningful regimes such as bull or bear markets, high or low volatility periods, or liquidity conditions. The hidden states are unobserved and must be inferred from the observable data, yet they govern the evolution of the underlying dynamics of the time series.

Transition Probability Matrix. The evolution of the hidden state sequence is governed by a transition probability matrix $\mathbf{P} = [p_{ij}] \in \mathbb{R}^{K \times K}$, where each element $p_{ij} = \mathbb{P}(Z_t = Z_j \mid Z_{t-1} = Z_i)$ gives the probability of moving from state i to state j at time t . Each row of this matrix must sum to one: $\sum_{j=1}^K p_{ij} = 1 \ \forall i$ to ensure valid probabilistic interpretation.

Emission Distributions. For each hidden state, the observed variable Y_t is generated from a probability distribution conditional on the current state. Formally, $Y_t \sim f(y \mid Z_t = Z_i)$, where $f(\cdot)$ is the state-specific emission density, hence:

$$\mathbb{P}(Y_t = y \mid Z_t = Z_i) = f(y \mid Z_i), \quad i = 1, \dots, K \quad (2.5)$$

Initial Distribution. The model is initialized by a probability vector $\pi = (\pi_1, \dots, \pi_K)$, where $\pi_i = \mathbb{P}(Z_0 = Z_i)$ represents the probability that the system starts in state i . This distribution defines the initial conditions of the hidden state sequence and can either be assumed uniform or estimated from the data using maximum likelihood or Bayesian methods.

²The term **latent** refers to variables or states that are not directly observed but are inferred from the observed data. In the context of HMMs, latent states represent underlying conditions or regimes that influence the observable outcomes, but are themselves not directly measurable.

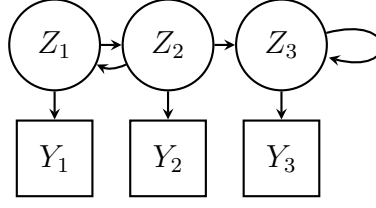


FIGURE 2.2. Graphical representation of a Hidden Markov Model (HMM) with three hidden states Z_1 , Z_2 , and Z_3 , and their corresponding observed variables Y_1 , Y_2 , and Y_3 . The arrows indicate the transition probabilities between hidden states and the emission probabilities from hidden states to observed variables.

Mathematical Justification. As we introduced, the HMM is defined by a set of hidden states, a transition probability matrix, emission distributions, and an initial distribution. Formally and more rigorously, it is better defined as a double stochastic process, where the hidden states follow a Markov chain and the observed variables are generated conditionally on these states. The joint distribution of the observed sequence $Y = (Y_1, Y_2, \dots, Y_T)$ and the hidden state sequence $Z = (Z_1, Z_2, \dots, Z_T)$ can be expressed as:

$$\mathbb{P}(Y, Z) = \mathbb{P}(Z_1) \mathbb{P}(Y_1 | Z_1) \prod_{t=2}^T \mathbb{P}(Z_t | Z_{t-1}) \mathbb{P}(Y_t | Z_t) \quad (2.6)$$

The key assumptions underpinning the HMM are:

$$\mathbb{P}(Z_{t+1} | Z_t, Z_{t-1}, \dots, Z_0) = \mathbb{P}(Z_{t+1} | Z_t) \quad (2.7)$$

$$\mathbb{P}(Y_t | Z_t, Y_{t-1}, \dots, Y_0) = \mathbb{P}(Y_t | Z_t) \quad (2.8)$$

These assumptions, respectively, state that the future state Z_{t+1} depends only on the current state Z_t (the Markov property), and that the current observation Y_t depends only on the current state Z_t (the emission property). Together, they ensure that the full likelihood of a sequence $(Y_1, \dots, Y_T)$ can be computed recursively using dynamic programming, enabling efficient inference algorithms like the **Forward-Backward Algorithm** for computing marginal state probabilities, and the **Viterbi Algorithm** for finding the most likely path of hidden states $(Z_1^*, \dots, Z_T^*)$.

Under mild regularity conditions³, the HMM is a parametrically identifiable model, meaning that the joint distribution of the observed sequence uniquely determines the model parameters up to label switching. Parameter estimation is commonly performed via the **Baum-Welch Algorithm**, a form of Expectation-Maximization (EM), which iteratively maximizes the likelihood of the observed data by inferring

³**Mild regularity** conditions refer to the assumptions that the model parameters are identifiable and that the state space is finite or countably infinite.

posterior state probabilities and updating emission and transition parameters.⁴ When the emission distributions f_i are Gaussian, the joint distribution of observations $(Y_1, \dots, Y_T)$ becomes a **mixture of Gaussians**, meaning that each component of the mixture corresponds to a different hidden state, with state-dependent parameters. This yields flexibility in modeling multimodal and heteroskedastic behavior in (specifically) financial time series, particularly useful for capturing volatility clustering and structural breaks (Hamilton 1989).

⁴For a detailed treatment of the Baum-Welch algorithm, see Baum and Petrie (1966). This algorithm is also used in Fanelli, Fontana, and Rotondi (2023).

2.2.4 Hybrid Model (OU-HMM)

Following the previous sections, we now define the hybrid model. The Ornstein-Uhlenbeck Hidden Markov Model (OU-HMM) is a hybrid stochastic model that integrates the continuous-time and mean-reverting properties of the Ornstein-Uhlenbeck process with the regime-switching framework of the Hidden Markov Model. This way of construction is designed to capture not only the stationary and mean-reverting nature of the cointegration spread S_t , but also to account for structural shifts in market dynamics through latent regimes.

This framework assumes that the parameters governing the OU process are not fixed, but switch between different values depending on the current latent regime $Z_t \in \{1, \dots, N\}$ governed by a Markov chain. Each regime is characterized by its own set of OU parameters $(\theta_i, \mu_i, \sigma_i)$, allowing for flexible modeling of time-varying dynamics.

Model Specification. Let $(S_t)_{t \geq 0}$ be a continuous-time stochastic process satisfying:

$$dS_t = \theta_{Z_t}(\mu_{Z_t} - S_t) dt + \sigma_{Z_t} dW_t \quad (2.9)$$

where:

- $Z_t \in \{1, \dots, K\}$ is a latent (unobserved) discrete-time Markov chain with transition matrix $P = [p_{ij}]$, governing the active regime at time t .
- $(\theta_i, \mu_i, \sigma_i)$ are regime-specific parameters associated with state i .
- W_t is a standard Brownian motion independent of Z_t .

This formulation means that at any given time t , the dynamics of S_t follow an OU process whose parameters are dictated by the current markovian hidden state Z_t . In contrast to the previously explained OU model with fixed coefficients, the OU-HMM allows for dynamic adaptation, meaning that when the regime switches, the entire set of parameters (θ, μ, σ) changes, and consequently, the behavior of the spread process adapts in response.

Interpretation and Intuition. This model assumes that market conditions can shift over time due to various macroeconomic reasons or evolving cointegration relationships among crude oil futures. Each of these conditions is reflected in a different hidden state. For example:

- **Regime 1** may correspond to stable, low-volatility environments where the spread reverts quickly to a tight equilibrium.

- **Regime 2** may represent high-volatility environments where the spread reverts slowly, and noise dominates the signal.
- **Regime 3** may model persistent market dislocations, where μ_t is substantially shifted, indicating a new perceived equilibrium.

By conditioning the OU parameters on the latent state through the HMM, we allow the model to shift between these dynamics without needing to re-estimate the entire model structure at each point in time. In the context of this thesis, this is particularly important, as it enables the model to adapt to changing market conditions in real-time while maintaining the statistical rigor and underlying properties of the process.

Mathematical Properties. Given the regime-switching nature of the model, the process S_t is no longer a simple Gaussian process, but a **regime-dependent Gaussian process**, or equivalently, a **mixture of Gaussian OU processes**, where each component corresponds to a different latent regime. Conditional on the latent regime path $(Z_0, \dots, Z_t)$, the process S_t remains Gaussian and admits the explicit solution:

$$S_t = \mu_{Z_t} + (S_0 - \mu_{Z_t})e^{-\theta_{Z_t}t} + \sigma_{Z_t} \int_0^t e^{-\theta_{Z_t}(t-s)} dW_s \quad (2.10)$$

However, since Z_t is hidden, the marginal distribution of S_t becomes a **mixture distribution** over all possible regime paths. Inference must therefore rely on filtering techniques such as the Forward-Backward algorithm, generalized to continuous-time HMMs, or particle filtering in more complex cases. Parameter estimation can be performed via a Baum-Welch algorithm tailored to switching diffusion processes, as outlined in Rabiner (1989) and consistently applied in Fanelli, Fontana, and Rotondi (2023).

Inference and Strategy. In the context of this thesis and according to the foundational empirical work, the OU-HMM is used to model the cointegration spread between crude oil futures. The regime classification inferred from the model enables the construction of regime-specific trading strategies, including:

- Long/short positioning when the spread reverts within certain thresholds in low-volatility regimes.
- Market-neutral exposure adjustments during volatile or trending regimes.
- Exit signals based on transition probabilities exceeding critical thresholds.

These strategies are adaptive, since the model continuously updates the posterior probabilities of each regime as new data becomes available. The transition matrix P encodes the persistence of each regime, and the stationarity of the Markov chain ensures that asymptotic regime probabilities can be interpreted as unconditional regime frequencies.

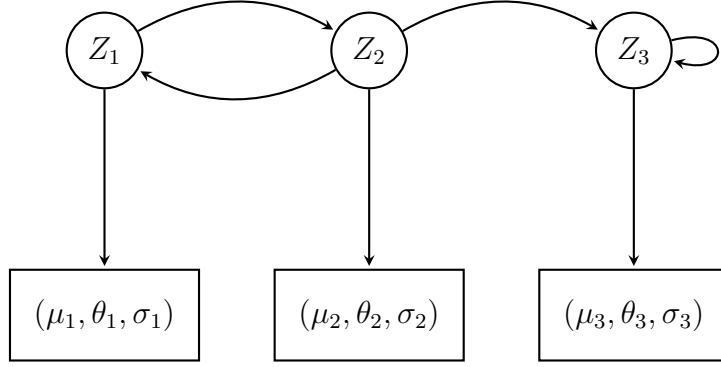


FIGURE 2.3. Graphical depiction of the OU-HMM: each latent regime Z_t governs a specific set of OU parameters $(\mu_i, \theta_i, \sigma_i)$ for the spread process S_t . The transition between the regimes is governed by a Markov chain, allowing the OU parameters to switch dynamically based on the current regime. This structure captures the time-varying nature of the cointegration spread, enabling adaptive modeling of market dynamics.

Related Work and Literature. The OU-HMM framework is conceptually rooted in regime-switching models of financial time series introduced by Hamilton (1989), and further generalized to continuous-time stochastic processes in Robert J Elliott, Aggoun, and Moore (1995).

2.2.5 Model Discretization

While the OU-HMM defined in Equation (2.9) is a continuous-time model, in practical financial applications, we work with data taken at discrete-time intervals. As a result, the continuous-time stochastic differential equation must be discretized to enable both practical implementation and statistical inference using observed data.

Discretization refers to the **approximation** of a continuous-time stochastic process by a discrete-time process that captures its essential dynamics over finite time intervals. This step is essential both for statistical estimation and for strategy implementation, as the continuous trajectory of the spread S_t is not observable in practice, and model inference must hence rely exclusively on a discrete sequence of observations $\{S_t\}_{t=0}^T$, sampled at a fixed time interval, denoted as Δt . Rigorous implementation in this work is done with daily observations, not consistent with the

work performed for the other approaches, due to the need for a larger sample size to accurately estimate the model parameters.

Discrete-Time Approximation Given that S_t evolves according to regime-dependent OU dynamics (as defined in Equation (2.9)), recall:

$$dS_t = \theta_{Z_t}(\mu_{Z_t} - S_t) dt + \sigma_{Z_t} dW_t \quad (2.11)$$

we seek a discretized process $\{S_t\}_{t=0}^T$ that approximates the continuous-time diffusion over time intervals of length Δt . Following Fanelli, Fontana, and Rotondi (2023), and consistently with classical treatments (see Bibby and Sørensen (2004)), we adopt the exact transition density of the OU process to derive the discrete-time evolution of the process (see Equation (2.14)). Conditionally on the current regime $Z_k = i$, the process admits the exact discretization:

$$S_{k+1} = \mu_i + (S_k - \mu_i)e^{-\theta_i \Delta t} + \eta_k, \quad \eta_k \sim \mathcal{N}\left(0, \frac{\sigma_i^2}{2\theta_i} (1 - e^{-2\theta_i \Delta t})\right) \quad (2.12)$$

where:

- $S_k := S_{t_k}$ is the observed value of the spread at time $t_k = k\Delta t$.
- $\mu_i, \theta_i, \sigma_i$ are the parameters of the OU process associated with the active regime $Z_k = i$ at time t_k .
- η_k is a Gaussian innovation with regime-dependent variance reflecting the conditional distribution of S_{k+1} given S_k and the current regime $Z_k = i$. The term **innovation** refers to the new information that affects the process at time k . In this context, it represents the random component of the OU process that introduces uncertainty into the spread dynamics.

This formulation preserves the exact first and second moments of the continuous-time OU process under each regime and thus avoids the bias introduced by Euler-Maruyama approximations when Δt is not infinitesimally small. **Euler-Maruyama** is a numerical method used to approximate solutions to stochastic differential equations, specifically it involves discretizing the time variable and applying a finite difference scheme, which can lead to inaccuracies when the time step is not sufficiently small. Notably, this discretized transition is fully tractable and can be directly embedded into the **Expectation-Maximization (EM)** algorithm for parameter estimation, as shown in Fanelli, Fontana, and Rotondi (2023). **Tractable** refers to the ability to

obtain a solution in a reasonable amount of time and with available computational resources.

Regime-Switching Structure. Recalling from Section 2.2.3 we know that $Z_k \in \{1, \dots, K\}$ denotes the discrete-time Markov chain governing the regime at time k , with transition matrix $P = [p_{ij}]$, where:

$$p_{ij} := \mathbb{P}(Z_{k+1} = j \mid Z_k = i), \quad \sum_{j=1}^K p_{ij} = 1 \quad \forall i \quad (2.13)$$

The full model, consisting of (S_k, Z_k) , hence constitutes a **Hidden Markov Model with regime-dependent Gaussian emissions** (Cicmilović 2025; Suchato et al. 2024; Endres and Stübinger 2018; Behme, Di Tella, and Sideris 2024). The observation equation becomes:

$$S_{k+1} \mid (S_k, Z_k = i) \sim \mathcal{N} \left(\mu_i + (S_k - \mu_i)e^{-\theta_i \Delta t}, \frac{\sigma_i^2}{2\theta_i} (1 - e^{-2\theta_i \Delta t}) \right) \quad (2.14)$$

This discrete-time representation is theoretically consistent with the continuous-time OU-HMM and serves as the foundation for the likelihood-based Expectation-Maximization (EM) estimation procedure, detailed in Section 2.2.6. It also enables the application of HMM filtering techniques to infer the posterior probabilities of the latent regimes based on observed spread trajectories.

2.2.6 Theoretical Parameter Estimation

To estimate the parameters $\{\theta_i, \mu_i, \sigma_i\}_{i=1}^K$ and the Markov chain transition matrix $P = [p_{ij}]$, we implement a likelihood-based Expectation-Maximization (EM) algorithm. This approach is grounded in the framework proposed by Fanelli, Fontana, and Rotondi (2023), which adapts the general HMM estimation methodology from Robert J Elliott, Aggoun, and Moore (1995), and specifically the regime-switching OU calibration introduced in Erlwein and Mamon (2006).

Given a discrete sample $\{S_k\}_{k=0}^T$ of the cointegration spread observed at regular intervals Δt , the process (S_k, Z_k) is modeled as a Hidden Markov Model with Gaussian regime-dependent emissions as described in Section 2.2.5.

Complete Likelihood. Let $\psi = \{(\theta_i, \mu_i, \sigma_i)\}_{i=1}^K \cup \{P, \pi_0\}$ denote the full set of parameters. The complete data likelihood, conditioned on the latent regime path

$(Z_0, \dots, Z_T)$, is given by:

$$\mathcal{L}_{\text{complete}}(\psi) = \mathbb{P}(Z_0) \prod_{k=0}^{T-1} \mathbb{P}(Z_{k+1} \mid Z_k) \prod_{k=0}^{T-1} \mathbb{P}(S_{k+1} \mid S_k, Z_k) \quad (2.15)$$

Data Likelihood in HMMs refers to the probability of the observed data given the model parameters, integrating over all possible hidden state sequences. Simply put, the likelihood of the complete data means that for each possible hidden state sequence, the model assigns a probability based on how likely it is to transition between states (*transition probabilities*) and how likely each state is to produce an observation (*emission probabilities*).

The EM procedure iteratively maximizes the expected log-likelihood, calculated by taking the logarithm of the complete likelihood and then its expectation over the hidden states' posterior distribution. We define the following probabilities:

$$\gamma_k^i = \mathbb{P}(Z_k = i \mid S_0, \dots, S_T) \quad (2.16)$$

$$\xi_k^{ij} = \mathbb{P}(Z_k = i, Z_{k+1} = j \mid S_0, \dots, S_T) \quad (2.17)$$

Here, γ_k^i is the smoothed probability of being in regime i at time k , and ξ_k^{ij} is the joint smoothed probability of transitioning from regime i at time k and regime j at time $k + 1$.

In Fanelli, Fontana, and Rotondi (2023), these smoothed probabilities are implicitly represented through auxiliary sufficient statistics, recursively estimated via filtering. Specifically, the following cumulative quantities are introduced:

- $J_{ij}^t := \sum_{n=1}^t \langle Z_{n-1}, e_i \rangle \langle Z_n, e_j \rangle$, which approximates the expected number of transitions from state i to state j up to time t , where e_i and e_j are the indicator functions for states i and j , respectively;
- $O_i^t := \sum_{n=1}^t \langle Z_n, e_i \rangle$, which captures the total occupation time of regime i ;
- $G_i^t(f) := \sum_{n=1}^t \langle Z_{n-1}, e_i \rangle f_n$, Used to estimate regime-specific moments, where f_n is a generic function of the observable process y , such as $f_n = y_n$, $f_n = y_n^2$, or $f_n = y_n y_{n-1}$.

Auxiliary Sufficient Statistics are values used to summarize the information in the observed data that is relevant for estimating the model parameters. Approaching the problem as such, the model output would be the filtered estimates of the latent states and the sufficient statistics themselves.

Since the regime process X_t is unobserved, these quantities are filtered through the observation filtration $\mathcal{F}_t^y = \sigma(y_0, \dots, y_t)$, representing the available information

set. Denoting by $\hat{X}_t := \mathbb{E}[X_t \mid \mathcal{F}_t^y]$ the filtered estimate of the latent state at time t , the recursive estimates of the sufficient statistics are expressed as:

$$\hat{J}_{ij}^t, \quad \hat{O}_i^t, \quad \hat{G}_i^t(f) \quad (2.18)$$

which are functions of the posterior probabilities $\gamma_t^i = \mathbb{P}(Z_t = i \mid \mathcal{F}_t^y)$ and $\xi_t^{ij} = \mathbb{P}(Z_t = i, Z_{t+1} = j \mid \mathcal{F}_T^y)$, but written in a form that allows for recursive updating without explicitly storing the full posterior path.

These filtered quantities will be used in the subsequent **Expectation** step to compute the expected complete log-likelihood and to perform closed-form maximization of the parameter estimates. Notably, the recursive structure of $\hat{J}_{ij}^t, \hat{O}_i^t, \hat{G}_i^t(f)$ offers an efficient implementation of the EM algorithm in the hidden regime-switching framework, while preserving the sufficient statistics necessary for maximum-likelihood inference.

As established in Fanelli, Fontana, and Rotondi (2023), the model parameters $\alpha(X_t), \gamma(X_t), \eta(X_t)$, which are functions of the underlying OU parameters $\theta_i, \mu_i, \sigma_i$, can be estimated via moment-based updates derived from these quantities. The parameters $\alpha_i, \gamma_i, \eta_i$ estimated from the discretized AR (1) process are not equivalent to the continuous-time Ornstein-Uhlenbeck parameters $\theta_i, \mu_i, \sigma_i$. The correct mapping is nonlinear: $\alpha_i = e^{-\theta_i \Delta t}$, $\gamma_i = \mu_i(1 - \alpha_i)$, and $\eta_i = \sigma_i \sqrt{\frac{1 - \alpha_i^2}{2\theta_i}}$. These expressions must be inverted to retrieve the original continuous-time model parameters. Specifically, at each EM iteration, the updated parameters are given by:

$$\hat{\alpha}_i(t) = \frac{\hat{G}_i^t(y_n y_{n-1})}{\hat{G}_i^t(y_{n-1}^2)}, \quad (2.19)$$

$$\hat{\gamma}_i(t) = \frac{\hat{G}_i^t(y_n) - \hat{\alpha}_i(t) \hat{G}_i^t(y_{n-1})}{\hat{O}_i^t}, \quad (2.20)$$

$$\hat{\eta}_i^2(t) = \frac{1}{\hat{O}_i^t} \sum_{n=1}^t \hat{\gamma}_{n-1}^i \left(y_n - \hat{\gamma}_i(t) - \hat{\alpha}_i(t) y_{n-1} \right)^2 \quad (2.21)$$

where y_n is the observed value of the spread at time n , and $\hat{\gamma}_i(t)$, $\hat{\alpha}_i(t)$, and $\hat{\eta}_i(t)$ are the estimated regime-specific parameters at time t . These estimators serve as empirical analogs to the OU-HMM discretization in Equation (2.14), and can be interpreted as the regime-specific coefficients of the OU process (K. Chen 2019; Ngo and Pham 2014; P. Chen 2023), set in autoregressive form, where the next value of the spread y_{t+1} is expressed as a function of the current value y_t , the regime-dependent

mean $\gamma(X_t)$, and a noise term scaled by the regime-dependent volatility $\eta(X_t)$:

$$y_{t+1} = \gamma(X_t) + \alpha(X_t)y_t + \eta(X_t)z_t, \quad z_t \sim \mathcal{N}(0, 1) \quad (2.22)$$

The next section elaborates on the complete EM algorithm, using these recursive quantities in the E-step and updating the parameters accordingly in the M-step, enabling consistent estimation under partial observability of the latent regime process.

E-Step (Expectation). The goal of the E-step is to compute previously stated smoothed probabilities that will be used to estimate the expected complete log-likelihood. Specifically, we compute the posterior probabilities of the hidden states and their transitions given the observed data:

$$\gamma_k^i = \mathbb{P}(Z_k = i \mid S_0, \dots, S_T) \quad (2.23)$$

$$\xi_k^{ij} = \mathbb{P}(Z_k = i, Z_{k+1} = j \mid S_0, \dots, S_T) \quad (2.24)$$

These smoothed probabilities represent the expected value of the latent state indicators and state transition indicators respectively. The quantity γ_k^i gives us the expected “occupation” of regime i at time k , while ξ_k^{ij} gives us the expected number of transitions from regime i to regime j between times k and $k + 1$. To compute these posterior probabilities efficiently we employ the Forward-Backward algorithm, which uses dynamic programming⁵ to avoid enumerating all possible state sequences. This algorithm computes forward probabilities α_k^i (the probability of observing data up to time k given the current state $Z_k = i$) and backward probabilities β_k^i for each state i at each time step k , which are then used to derive the smoothed probabilities γ_k^i and ξ_k^{ij} . (see Rabiner (1989)).

The smoothed posteriors are then obtained as:

$$\gamma_k^i = \frac{\alpha_k^i \cdot \beta_k^i}{\sum_{j=1}^K \alpha_k^j \cdot \beta_k^j} \quad (2.25)$$

$$\xi_k^{ij} = \frac{\alpha_k^i \cdot p_{ij} \cdot g_j(S_{k+1} \mid S_k) \cdot \beta_{k+1}^j}{\sum_{a=1}^K \sum_{b=1}^K \alpha_k^a \cdot p_{ab} \cdot g_b(S_{k+1} \mid S_k) \cdot \beta_{k+1}^b} \quad (2.26)$$

where p_{ij} represents the transition probability from regime i to regime j , and $g_j(S_{k+1} \mid S_k)$ is the emission probability of the Ornstein-Uhlenbeck process for regime j . These expected sufficient statistics will be used in the M-step to update the model

⁵**Dynamic Programming** is a method for solving complex problems by breaking them down into simpler subproblems, which can be solved recursively. In the context of HMMs, it allows for efficient computation of probabilities by storing intermediate results.

parameters.

M-Step (Maximization). Moving on from the Expectation, the M-step aims to maximize the expected complete log-likelihood computed in the E-step to update the model parameters consistently with the new information. This involves updating the model parameters $\psi = \{(\theta_i, \mu_i, \sigma_i)\}_{i=1}^K \cup \{P, \pi_0\}$ based on the smoothed probabilities and sufficient statistics derived from the observed data.

- **Initial distribution:**

$$\hat{\pi}_0^i = \gamma_0^i \quad (2.27)$$

We start by defining the initial distribution of the hidden states, which, as per the definition given previously, is simply the smoothed probability of being in state i at time $k = 0$.

- **Transition probabilities:**

$$\hat{p}_{ij} = \frac{\sum_{k=0}^{T-1} \xi_k^{ij}}{\sum_{k=0}^{T-1} \gamma_k^i} \quad (2.28)$$

Then, the transition probabilities are updated based on the smoothed joint probabilities of transitioning from state i to state j over the entire sequence of observations. This ensures that the estimated transition matrix reflects the observed dynamics of regime switching.

- **OU parameters.** Define, for each regime i , the regime-weighted observations:

$$\bar{S}_i = \frac{\sum_{k=0}^{T-1} \gamma_k^i S_k}{\sum_{k=0}^{T-1} \gamma_k^i} \quad (2.29)$$

$$\bar{S}_{i+1} = \frac{\sum_{k=0}^{T-1} \gamma_k^i S_{k+1}}{\sum_{k=0}^{T-1} \gamma_k^i} \quad (2.30)$$

Which can be defined as the weighted average of the observed spread values at time k and $k + 1$ for each regime i . These averages are used to compute the regime-specific OU parameters.

Then, the parameter updates are obtained via nonlinear least squares on the following closed-form expressions (derived from the OU discrete transition):

$$\hat{\theta}_i = -\frac{1}{\Delta t} \log \left(\frac{\sum_{k=0}^{T-1} \gamma_k^i (S_{k+1} - \mu_i)}{\sum_{k=0}^{T-1} \gamma_k^i (S_k - \mu_i)} \right) \quad (2.31)$$

The parameter $\hat{\theta}_i$ is estimated as the negative logarithm of the ratio of the

weighted differences between the observed spread at time $k + 1$ and the regime-specific mean μ_i to the weighted differences at time k . This captures the rate of mean reversion in the OU process for each regime.

$$\hat{\mu}_i = \frac{\sum_{k=0}^{T-1} \gamma_k^i (S_{k+1} - e^{-\theta_i \Delta t} S_k)}{\sum_{k=0}^{T-1} \gamma_k^i (1 - e^{-\theta_i \Delta t})} \quad (2.32)$$

The parameter $\hat{\mu}_i$ is estimated as the weighted average of the observed spread at time $k + 1$ minus the exponentially decayed value of the spread at time k , normalized by the sum of the weights. This represents the long-term mean of the OU process for each regime.

Finally, the volatility parameter is estimated as:

$$\hat{\sigma}_i^2 = \frac{2\theta_i}{1 - e^{-2\theta_i \Delta t}} \cdot \frac{1}{\sum_{k=0}^{T-1} \gamma_k^i} \sum_{k=0}^{T-1} \gamma_k^i \left(S_{k+1} - \mu_i - (S_k - \mu_i) e^{-\theta_i \Delta t} \right)^2 \quad (2.33)$$

These steps are iterated until convergence of the log-likelihood:

$$\ell(\psi) = \log \sum_{i=1}^K \alpha_T^i \quad (2.34)$$

where $\ell(\psi)$ is the log-likelihood of the observed data given the model parameters ψ . The EM algorithm converges when the change in the log-likelihood between iterations falls below a predefined threshold, indicating that the parameter estimates have stabilized.

2.3 Trading Strategies

Following the work of Fanelli, Fontana, and Rotondi (2023), this thesis explores statistical arbitrage strategies built upon the cointegration relationships identified among the crude oil futures markets. We identify 3 main strategies that can be implemented based on the cointegration spread S_t constructed in Section 3.2.

- **Plain Vanilla:** A benchmark approach from the pairs trading literature, as seen in Burgess (1999), which enters a position every time the spread deviates from its mean. It is simple but sensitive to transaction costs due to its frequent trading nature.
- **Probability Interval (ProbI):** A statistically-informed strategy that triggers trades only when the spread breaches a predefined confidence band (e.g. Bollinger Bands at 95% confidence), reducing noise and number of trades while also improving robustness.
- **Realized Increment (RI):** Based on the idea of significant spread movements, this strategy opens trades only when the change in S_t exceeds historical quantile thresholds, filtering out minor fluctuations and, similarly to **ProbI**, reduce the number of trades, effectively cutting transaction costs.

Additionally, we identify two additional strategies⁶ that are based on the OU-HMM framework, which is detailed in Section 2.2.4:

- **Prediction Interval (PredI):** A strategy that uses the OU-HMM to predict the future spread and enters trades based on the predicted values, effectively leveraging the regime-switching dynamics of the spread process.
- **Predicted Increment (PI):** A strategy that focuses on the predicted changes in the spread, entering trades when the forecasted increment exceeds a certain threshold.

These strategies are designed to exploit the time-varying nature of the spread dynamics captured by the OU-HMM, allowing for more adaptive trading decisions based on the current regime of the spread process. The following sections will detail the theoretical framework, assumptions, mechanics, and payout structures of each strategy, providing a comprehensive understanding of their implementation and potential profitability in the context of statistical arbitrage.

⁶These strategies were initially implemented in Fanelli, Fontana, and Rotondi (2023), and have been adapted to the four-asset system in this thesis.

2.3.1 Strategy: Plain Vanilla

Theoretical Framework. From Burgess (1999) we understand that the plain vanilla strategy represents the most fundamental and widely studied form of statistical arbitrage in the pairs trading literature. The capitalization of the cointegration relationship is done by exploiting the mean-reverting nature of the spread process S_t by entering positions whenever the spread deviates from its long-run mean, while under the assumption that such deviations are systematically temporary and will revert back to the series' long-term equilibrium.

Assumptions. From a purely theoretical standpoint, the strategy operates under the following assumptions:

- The spread process $S_t = \beta^\top Y_t$ is stationary, and mean reverting.
- Temporary deviations are exploitable through an opposite position, namely by shorting or going long on the spread.⁷
- The Law of One Price (Fama 1970)⁸ holds in the long-run, assuming that the prices of the underlying assets tied by a cointegration relationship will reconverge.
- Market dynamics follow discrete-time observations with continuous pricing, allowing for real-time trading and execution.
- For sake of simplicity, we assume also that prices are frictionless.
- A two-asset system consisting of two cointegrated assets, denoted $Y_t = [y_{1,t}, y_{2,t}]$, with cointegration matrix $\beta = [\beta_1, \beta_2]^\top$, where β_1 and β_2 are the cointegration coefficients for the two assets.

Mechanics and Payout Structure. We define the strategy mechanics and payout structure by following on the chronological steps of the strategy process.

At time $t - 1$, if $S_t > \bar{\mu}$, this implies that the spread is above the long term mean, indicating that the first asset $y_{1,t-1}$ is overpriced relative to the second asset $y_{2,t-1}$. Hence we short the spread as such:

⁷Shorting the spread involves selling the asset that appears overpriced while simultaneously buying the asset that appears underpriced. Conversely, if a trader anticipates that the current mispricing will intensify rather than revert, they can take a long spread position. In this case, the trader buys the asset that is relatively expensive and sells the asset that is relatively cheap, betting that the spread will

⁸The **Law of One Price** states that in efficient markets, identical products should sell for the same price if they share the same features, hence eliminating theoretical arbitrage opportunities.

- A short position is opened on the first asset $y_{1,t-1}$, and a long position is opened on the second asset $y_{2,t-1}$.
- The position is closed at time t when the spread reverts back to its mean, i.e. when $S_t = \bar{\mu}$.

This leads to the theoretical payout at time t being:

$$\Pi = -(y_{1,t} - y_{1,t-1}) + (y_{2,t} - y_{2,t-1}) = y_{2,t} - y_{1,t} + y_{1,t-1} - y_{2,t-1} \quad (2.35)$$

In absence of transaction costs, this strategy would theoretically offer unlimited frequency of trades and profit opportunities.

Extension to a Four-Asset System. While the classical pairs trading strategy operates on two assets, this thesis extends this framework to a four-asset cointegrated system, as illustrated in Section 3.2.3. In line with the Johansen framework, the spread takes the form:

$$S_t = \beta_1 y_{1,t} + \beta_2 y_{2,t} + \beta_3 y_{3,t} + \beta_4 y_{4,t} \quad (2.36)$$

where β is the cointegrating vector estimated from the system. Trading mechanics remain similar, with the key difference being that the spread is now a linear combination of four assets, allowing for more complex interactions and potentially higher profitability. The payout structure is adjusted accordingly to account for the additional asset.

When $S_t > \bar{\mu}$, the strategy shorts the assets with positive coefficients in the cointegration vector β , and goes long on those with a negative coefficient, effectively betting on mean reversion. Ultimately, this configuration allows for a more diversified exposure to the underlying market dynamics, potentially enhancing the robustness of the strategy, and allowing for more complex mispricing exploitation.

2.3.2 Strategy: Probability Interval (ProbI)

Theoretical Framework. The Probability Interval (ProbI) strategy enhances the traditional pairs trading framework by integrating a statistical decision boundary around the mean of the spread process S_t . Rather than executing trades on every deviation, this approach filters for positions only when the spread breaches predefined confidence intervals, such as 95%, 99%, or 97.5% as formalized in Fanelli, Fontana, and Rotondi (2023), and assuming normality of S_t , these intervals correspond to quantiles of the standard normal distribution. These intervals can be constructed

using rolling standard deviations, more commonly known as Bollinger Bands (Leeds 2012), quantile bounds, or empirical density estimates. Targeting only statistically significant deviations, this strategy reduces the risk of overtrading and enhances the robustness of the trading signal.

Assumptions. The ProbI strategy operates under the following assumptions:

- As per the Plain Vanilla strategy, the spread process S_t is stationary and mean-reverting, the Law of One Price holds in the long-run, and we consider a two-asset system with cointegration matrix $\beta = [\beta_1, \beta_2]^\top$.
- Entry and exit triggers are based on statistically significant deviations from $\bar{\mu}$, typically defined by a confidence band of the sort $\bar{\mu} \pm z \cdot \sigma_t$, with z chosen based on the desired confidence level.

Mechanics and Payout Structure. The mechanics of the ProbI strategy are similar to the Plain Vanilla strategy, with the key difference being the entry and exit conditions based on statistical significance rather than absolute deviation from the mean.

Assuming no position is open at time $t - 1$, the strategy operates iff:

$$S_t \notin (\bar{\mu}_{t-n:t-1} \pm q_\alpha \cdot \sigma_{t-n:t-1}) \iff |z_t| > q_\alpha \quad (2.37)$$

where:

$$z_t = \frac{S_t - \bar{\mu}_{t-n:t-1}}{\sigma_{t-n:t-1}} \quad (2.38)$$

with q_α being the quantile corresponding to the desired confidence level, and n being the rolling window size used to compute the mean and standard deviation of the spread. The payout structure remains the same as explained in Equation (2.35) for the Plain Vanilla strategy.

Extension to a Four-Asset System. In line with the generalization of the cointegration framework to more than two assets, the ProbI strategy can be extended to a four-asset system by redefining the spread as in Paragraph 2.3.1, where $\beta \in \mathbb{R}^4$. The confidence interval is then constructed based on the statistical properties of the spread, with rolling estimates of σ_t obtained via a windowed standard deviation of S_t .

2.3.3 Strategy: Realized Increment (RI)

Theoretical Framework. The Realized Increment (RI) strategy, originally introduced in Dunis, Laws, and Evans (2006), offers a statistically grounded method to reduce overtrading. The difference between this strategy and Probi lies in the entry condition, which is based on the distribution of the realized increments of the spread process S_t rather than on its absolute value. Instead of monitoring the spread itself, the RI strategy operates on the *relative increment* $x_t := \frac{S_t}{S_{t-1}} - 1$, capturing the magnitude of recent movements in the cointegration spread. Trading signals are generated only when the spread exhibits an unusually large jump compared to its historical behavior, determined via empirical quantile thresholds, effectively filtering out minor fluctuations and noise.

Assumptions. The RI strategy operates under the following assumptions⁹:

- The time series of realized increments $x_t := \frac{S_t}{S_{t-1}} - 1$ has a stable empirical distribution over a rolling window, from which empirical quantiles can be estimated.
- Entry and exit conditions are determined by thresholding x_t based on two empirical quantiles $q_{\alpha_l, x}$ and $q_{\alpha_u, x}$, which correspond to the lower and upper bounds of the desired confidence interval.

Mechanics and Payout Structure. The mechanics of the RI strategy are similar to the Probi strategy, with the key difference being the entry condition based on the realized increment rather than the absolute value of the spread.

Assuming no position is open at time $t - 1$, the strategy operates iff:

$$x_t \notin [q_{\alpha_l, x}, q_{\alpha_u, x}] \quad (2.39)$$

with $q_{\alpha_l, x}$ and $q_{\alpha_u, x}$ denoting the empirical quantiles of the spread increment x over the rolling window n . Positions are exited when the spread re-enters the interval, i.e. when $x_t \in [q_{\alpha_l, x}, q_{\alpha_u, x}]$. The payout structure is computationally equal to Equation (2.35) for the Plain Vanilla strategy, as the payout is still based on the difference between the prices of the two assets at time t and $t - 1$.

Extension to a Four-Asset System. The RI strategy naturally generalizes to a multi-asset cointegration setting by redefining the spread as in Paragraph 2.3.1,

⁹Assumptions about the market, spread process, Law of One Price, and system are equal to those of the Plain Vanilla strategy and the Probi strategy, hence not repeated here.

where $\beta \in \mathbb{R}^4$. The empirical quantiles $q_{\alpha_l, x}$ and $q_{\alpha_u, x}$ are computed based on the distribution of the realized increments of the spread S_t , allowing for a robust entry and exit mechanism that adapts to the dynamics of the multi-asset system.

2.3.4 Strategy: Prediction Interval (PredI)

Theoretical Framework. The Prediction Interval strategy enhances the probabilistic logic of the ProbI rule by leveraging the forward-looking structure of the stochastic model introduced in Section 2.2.5. Rather than relying on backward-looking rolling estimates of volatility and mean, this strategy adopts a model-driven approach to forecast the next-period spread distribution. Inspired by the methodology in Robert J. Elliott et al. (2005), and as implemented in Fanelli, Fontana, and Rotondi (2023), the decision to open a position is based on whether the next spread value is expected to lie outside a model-implied prediction interval, constructed from one-step-ahead conditional moments.

Assumptions. This strategy operates under the following assumptions:

- The spread process S_t is generated by a state-dependent AR (1) model modulated by a latent Markov chain, as introduced in Section 2.2.5.
- The hidden Markov state Z_t is unobservable and must be estimated using recursive filtering techniques.
- One-step-ahead conditional expectations and variances are computed based on the filtered regime at time $t - 1$.
- The filtered Markov state $\hat{Z}_{t-1}$ provides sufficient information to infer model parameters $\hat{\alpha}_{t-1}, \hat{\gamma}_{t-1}, \hat{\eta}_{t-1}$.

Mechanics and Payout Structure. Assuming no open position at time $t - 1$, the decision to open a trade at time t is made if the observed spread S_t lies outside the prediction interval:

$$S_t \notin \left(\mathbb{E}[S_t \mid \mathcal{F}_{t-1}^y] \pm q_\alpha \cdot \sqrt{\text{Var}[S_t \mid \mathcal{F}_{t-1}^y]} \right) \quad (2.40)$$

where:

$$\mathbb{E}[S_t \mid \mathcal{F}_{t-1}^y] = \hat{\gamma}_{t-1}(\hat{Z}_{t-1}) + \hat{\alpha}_{t-1}(\hat{Z}_{t-1})S_{t-1} \quad (2.41)$$

$$\text{Var}[S_t \mid \mathcal{F}_{t-1}^y] = \hat{\eta}_{t-1}^2(\hat{Z}_{t-1}) \quad (2.42)$$

The quantile threshold q_α is chosen based on the desired significance level. The filtered regime $\hat{Z}_{t-1}$ determines which set of parameters is used in the forecast, giving rise to regime-sensitive prediction bounds.

If the condition holds, the position is opened in the same directional manner as the Plain Vanilla and Probi strategies: going short on positive deviations and long on negative ones. The payout function follows Equation (2.35).

Extension to a Four-Asset System. The model-driven mechanics of the PredI strategy can naturally extend to a four-asset cointegration structure. Once the spread $S_t = \beta^\top Y_t$ is computed from the multivariate series $Y_t \in \mathbb{R}^4$, the prediction logic remains identical, as long as the latent regimes and parameter estimates are updated with respect to the same spread dynamics.

2.3.5 Strategy: Predicted Increment (PI)

Theoretical Framework. The Predicted Increment strategy takes the forward-looking perspective of the PredI strategy one step further by evaluating not the predicted spread itself, but the *predicted return* of the spread. This transforms the strategy into a model-based directional momentum detector: positions are opened only when the expected next-period relative change in spread is significantly large in either direction. The rule was proposed in Fanelli, Fontana, and Rotondi (2023) as a model-aware alternative to strategies based on lagged momentum or residuals.

Assumptions. The PI strategy operates under the following set of assumptions:

- The stochastic model defined in Section 2.2.5 accurately captures the spread dynamics.
- At time t , we compute the conditional expectation of the next-period spread, S_{t+1} , given the filtration $\mathcal{F}_t^y$.
- The expected relative increment of the spread is given by:

$$\hat{x}_t := \frac{\mathbb{E}[S_{t+1} \mid \mathcal{F}_t^y]}{S_t} - 1 \quad (2.43)$$

- The strategy assumes no open position at time $t - 1$, and reacts only when $\hat{x}_t$ is statistically significant.

Mechanics and Payout Structure. At each time step t , the investor computes the predicted increment $\hat{x}_t$ and enters a position only if:

$$\hat{x}_t \notin (q_\alpha, q_{1-\alpha}) \quad (2.44)$$

where $(q_\alpha, q_{1-\alpha})$ are the model-calibrated quantiles of the historical distribution of $\hat{x}_t$, chosen to filter for statistically significant forecasts. As in the other strategies, a long position is taken when $\hat{x}_t$ is sufficiently positive (above $q_{1-\alpha}$) and a short one when it is sufficiently negative (below q_α).

Once the position is opened, it is closed based on mean reversion, a fixed holding period, or a reversal in signal, depending on the strategy's design. The payout function again mirrors Equation (2.35), adjusted for trade direction and timing.

Extension to a Four-Asset System. As with all strategies considered, the PI strategy is compatible with a multi-asset framework. The key requirement is the computation of the spread $S_t = \beta^\top Y_t$, which then feeds into the prediction step. The model-based forecasts and increment logic apply identically, albeit potentially with higher predictive richness given the dimensionality of the spread construction.

2.4 Performance Measurement

The measurement of the performance of such trading strategies is crucial to assess their effectiveness and profitability, although computationally complex due to market dynamics, transaction costs, and miscellaneous factors such as slippage, liquidity, fees. For each strategy, then, we will focus on the following performance metrics:

- **Cumulative Return**
- **Maximum Drawdown**
- **Sharpe Ratio**
- **Calmar Ratio**

Portfolio Value. All the strategies are evaluated based on the **portfolio's equity value**, denoted as V . We begin with an initial equity of value $V_0 = 10,000$ USD. The portfolio value is updated sequentially at each time step k .

The portfolio value at time k , denoted V_k , is defined as the sum of the portfolio value at the previous time step, V_{k-1} , and the net profit or loss generated from all trades executed and closed at time k . This includes profits or losses from positions entered and closed on the same day k , as well as profits or losses from positions opened prior to k but closed at k .

$$V_k = V_{k-1} + \Pi_k \quad (2.45)$$

where Π_k represents the realized net profit or loss at time k . This term explicitly accounts for:

- **Gains/Losses from closed positions:** The difference between the closing price and opening price (adjusted for direction, i.e. long or short) for all trades that are fully closed at time k .
- **Transaction Costs:** We assume a frictionless and zero-cost environment for simplicity, but in practice, transaction costs would be subtracted from the portfolio value.
- **No Mark-to-Market:** Importantly, V_k reflects only *realized* profits and losses. It does not include unrealized gains or losses from open positions that are still active at time k .

The final portfolio value at time T , denoted V_T , is the culmination of this sequential process, representing the total equity after all trades within the backtesting period are concluded and accounted for. It is calculated as the initial equity plus the cumulative sum of all realized net profits and losses throughout the backtesting period:

$$V_T = V_0 + \sum_{k=1}^T \Pi_k \quad (2.46)$$

This formulation provides a clear and auditable trail of the strategy profitability based solely on realized gains and losses, offering a direct measure of capital growth. While this approach aligns with the practical realities of measuring performance in live trading (where only closed positions contribute to accessible equity), it is important to acknowledge that calculating V_T for statistical arbitrage strategies is not straightforward, as highlighted by Gatev, Goetzmann, and Rouwenhorst (2006). This difficulty primarily stems from the continuous, high-frequency nature of such strategies, often involving numerous small, offsetting trades, complex spread definitions, and the intricate accounting of varying transaction costs and financing expenses that can significantly erode profits if not meticulously tracked.

Cumulative Return. Cumulative Return quantifies the total profitability of the strategy over the entire backtesting period. Formally, it is defined as:

$$CR = \frac{V_T}{V_0} - 1 \quad (2.47)$$

where V_T is the final portfolio value at time T and V_0 is the initial portfolio value. In the context of statistical arbitrage, this allows to assess whether the strategy is able to generate positive returns over time. (Gatev, Goetzmann, and Rouwenhorst 2006)

Maximum Drawdown. Maximum Drawdown (MDD) measures the largest observed loss from a peak to a subsequent depth in the equity curve. It is defined as:

$$MDD = \max_{t \in [0, T]} \left(\frac{\max_{\tau \in [0, t]} V_\tau - V_t}{\max_{\tau \in [0, t]} V_\tau} \right) \quad (2.48)$$

capturing the downside risk in a non-parametric manner. In pairs trading, large drawdowns could indicate structural breaks or shifts in the cointegration relationships as explained in Vidyamurthy (2004).

Sharpe Ratio. The Sharpe Ratio evaluates the risk-adjusted return of a strategy.

$$\text{Sharpe Ratio} = \frac{\mathbb{E}[r_t]}{\sigma_r} \sqrt{N} \quad (2.49)$$

where r_t denotes the strategy's return at time t , $\mathbb{E}[r_t]$ is the mean return, σ_r is the standard deviation of returns, and N is annualization factor (in this case, $N = 52$ for weekly returns). This metric is particularly useful in cointegration-based strategies, as returns are typically small but frequent, emphasizing the importance of a favorable return-to-volatility trade-off, Lo (2002). A Sharpe Ratio greater than 1 is generally considered acceptable, while a ratio above 2 is considered excellent.

Calmar Ratio. The Calmar Ratio complements the Sharpe Ratio by evaluating the return relative to the corresponding unit of drawdown risk:

$$\text{Calmar Ratio} = \frac{\text{Annualized Return}}{MDD} \quad (2.50)$$

This ratio is particularly relevant for statistical arbitrage strategies, as it provides a measure of how well the strategy compensates for the risk of significant drawdowns. A higher Calmar Ratio indicates a more favorable risk-return profile, and is often used in conjunction with the Sharpe Ratio to provide a more comprehensive view of performance as more explicitly explained in Young (1991).

2.4.1 Risk Analysis

Beyond performance evaluation, a thorough risk analysis is essential to validate the robustness and reliability of the trading strategies under adverse conditions. In this work, the following methods are employed:

- **Value at Risk (VaR):** A probabilistic measure that quantifies the potential maximum loss of a portfolio over a specified time horizon, at given confidence levels.
- **Kernel Density Estimation (KDE) via Monte Carlo Simulation:** A non-parametric approach for estimating the probability density function (PDF) of returns, based on Monte Carlo resampling of the empirical distribution.
- **Conditional Value at Risk (CVaR):** An extension of VaR that measures the expected loss in the “worst-case” scenarios, providing a more complete and accurate picture of tail risk.

Value at Risk (VaR). Value at Risk (VaR) is employed to quantify the potential downside risk of the strategy. It represents the threshold loss level such that the probability of incurring a loss greater than this threshold is no more than α . Formally, for a return distribution $F(r)$, the VaR at confidence level $1 - \alpha$ is defined as:

$$\text{VaR}_{1-\alpha} = \inf \{x \in \mathbb{R} : F(x) \geq \alpha\} \quad (2.51)$$

In this thesis, we adopt the Monte Carlo simulation approach to estimate VaR. This involves simulating a large number of synthetic return paths from the empirical distribution, capturing nonlinearities and tail risks more effectively than parametric approaches, consistent with the methodology outlined by Fanelli, Fontana, and Rotondi (2023). This involves simulating a large number of synthetic return paths from the empirical distribution, capturing nonlinearities and tail risks more effectively than parametric approaches. VaR is computed at the 99%, 95%, and 90% confidence levels, to provide a granular view of the strategy’s tail risk exposure.

Monte Carlo-based Kernel Density Estimation (KDE). To further characterize the distributional properties of strategy returns, we employ Kernel Density Estimation (KDE) on the Monte Carlo-generated return samples. This non-parametric technique produces a smoothed approximation of the return distribution’s probability density function (PDF), allowing us to visualize deviations from normality such as skewness and kurtosis. KDE helps identify the presence of fat tails or multimodality

in the empirical returns, which are critical considerations in the context of model risk and tail event sensitivity. Robert and Casella (2004)

Conditional Value at Risk (CVaR). While VaR identifies the threshold of a potential loss, it provides no information about the magnitude of losses that might occur beyond this point. To address this limitation, we also compute the Conditional Value at Risk (CVaR), also known as Expected Shortfall (ES). CVaR measures the expected loss given that the loss has exceeded the VaR threshold. It's a more comprehensive risk metric that better captures the severity of tail events. Mathematically, the CVaR at a confidence level of $1 - \alpha$ is the conditional expectation of the loss, given the loss is greater than or equal to the VaR:

$$\text{CVaR}_{1-\alpha} = \mathbb{E}[-R \mid -R \geq \text{VaR}_{1-\alpha}] \quad (2.52)$$

where R is the portfolio return. Unlike VaR, CVaR is a coherent risk measure, as defined by Artzner et al. (1999), satisfying properties such as subadditivity, which ensures that the risk of a merged portfolio is not greater than the sum of its parts. CVaR is estimated from the Monte Carlo samples by averaging all simulated losses that fall within the worst $\alpha\%$ of outcomes.

Chapter 3

Methodology

3.1 Data Collection

3.1.1 Time Series and Timeframes

The data used in this thesis consists of daily settlement prices for the crude oil futures contracts introduced in Section 2.1. The sample period spans from March 26, 2018 when it was first introduced, Reuters (2018), to March 26, 2025, at a daily frequency. The series were aligned across markets by accounting for exchange-specific holidays and differences in trading calendars, ensuring synchronized observations across all instruments.

Shanghai Crude Oil Futures are denominated in CNY, and in order to normalize all series to a common currency, daily CNY/USD exchange rates were sourced and applied to convert the Shanghai futures into USD. This allows for consistent cross-market comparisons and cointegration testing on a unified scale.



FIGURE 3.1. Daily crude oil futures prices for Brent, WTI, Dubai, and Shanghai contracts from March 2018 to March 2025.

3.1.2 Data Preprocessing

First, date standardization was applied across all dataframes to ensure consistent formatting and temporal alignment. This step was necessary due to heterogeneous formatting in source files, which included both ISO and locale-specific formats (e.g., YYYY-MM-DD, MM/DD/YYYY). Subsequently, returns were computed as discrete daily log returns on the closing prices to account for nonstationarities and ensure robustness of data.

Frequency Aggregation. Given the goal of modeling medium-term price relationships, all daily time series were aggregated to a weekly frequency. This aggregation was done using dynamic windowing techniques that preserve OHLC structure, cumulative returns, and log-price trajectories. Specifically, the aggregation preserved:

- The first opening price of the week
- The maximum weekly high and minimum weekly low
- The last available closing price
- The sum of daily returns (as weekly return)
- The last log-transformed price

This means that even with the aggregation, no data was synthetically produced, and no change in magnitude was recorded effectively creating a weekly series. The aggregation doesn't show any empirical difference in testing, and was done to reduce computational load and to keep the foundation of the work as similar as possible to Fanelli, Fontana, and Rotondi (2023).

Calendar Synchronization. A reference calendar was derived from the Shanghai Futures Exchange given the fact that it was the shortest series available, reducing the risk of forward filling dates with inconsistent data. All other series were then aligned on this calendar, ensuring consistency in time steps while preserving nulls for non-trading days.

Train/Test Split. Each weekly series was split into training and testing datasets using a 70/30 ratio, without shuffling, to preserve time-order integrity.

Missing Data. Null counts were printed and visually inspected across all assets. No significant gaps were found in any series, and all nulls are due to non-trading days.

The code used for data preprocessing is available in Appendix A.1, resulting in the following table:

Dataset	Length	Null Count	Start Date	End Date
WTI	358	0	2018-03-26	2025-03-24
Shanghai	358	0	2018-03-26	2025-03-24
Dubai	358	0	2018-03-26	2025-03-24
Brent	358	0	2018-03-26	2025-03-24

TABLE 3.1. The shape check for the test sets of crude oil futures data after calendar synchronization

3.2 Cointegration Testing

In this section, a rigorous analysis is performed to test whether the selected crude oil futures (Brent, Shanghai, Dubai, and WTI) exhibit a long-term equilibrium relationship. The methodology includes: (i) stationarity testing of individual log-price series, (ii) estimation of an optimal VAR (p) model, (iii) implementation of the Johansen Cointegration Trace Test, and (iv) construction of the cointegration spread process S_t .

3.2.1 Unit Root Tests

To ensure the validity of cointegration testing, we begin by testing for the presence of unit roots in each log-price series. A robust assessment of stationarity is conducted using multiple tests: the Augmented Dickey-Fuller (ADF) test (Dickey and Fuller 1979), the Phillips-Perron (PP) test (Phillips and Perron 1988), the Dickey-Fuller Generalized Least Squares (DF-GLS) test (G. Elliott, Rothenberg, and Stock 1996), and the Kwiatkowski-Phillips-Schmidt-Shin (KPSS) test (Kwiatkowski et al. 1992). The ADF, PP, and DF-GLS tests share the null hypothesis of a **unit root**, i.e. that the series is non-stationary. In contrast, the KPSS test adopts the null hypothesis of stationarity. Employing both types of tests provides a cross-validation framework to mitigate test-specific limitations, effectively enhancing inference reliability. The results are reported in Table 3.2.

Series	ADF S.	ADF P _v .	PP S.	PP P _v .	DF-GLS S.	DF-GLS P _v .	KPSS S.	KPSS P _v .
Brent	-1.9610	0.3039	-2.0490	0.2655	-1.9604	0.0496	1.8958	0.0001
WTI	-2.2621	0.1845	-2.5705	0.0992	-2.2628	0.0237	2.1310	0.0001
Shanghai	-1.9826	0.2942	-1.8262	0.3675	-1.9765	0.0478	2.0604	0.0001
Dubai	-2.5471	0.1044	-2.0675	0.2578	-2.4073	0.0162	2.0092	0.0001

TABLE 3.2. Unit root test statistics for crude oil log-price series. The columns represent respectively the test statistic and the p-value for each test performed.

The results of the unit root tests consistently suggest that the log-price series of all four crude oil benchmarks are non-stationary in levels. Specifically, the Augmented Dickey-Fuller (ADF), Phillips-Perron (PP), and DF-GLS tests fail to reject the null hypothesis of a unit root at conventional significance levels (1%, 5%, 10%) for all series, with p-values well above the rejection threshold in most cases. Although the DF-GLS test for Dubai and WTI exhibits slightly lower p-values (0.0097 and 0.0191, respectively), they are insufficient to establish stationarity across the full set of tests. The KPSS test, with the null hypothesis being stationarity, strongly rejects the null for all series, with test statistics far exceeding critical values and p-values effectively at 0.0001.

Taken together, these results provide robust evidence about the existence of a cointegration relationship, as cointegration requires that individual series be integrated of order one, $I(1)$, but that a linear combination may be stationary. Thus, the statistical properties of the data are consistent with the assumptions underlying cointegration-based approaches.

3.2.2 VAR Model Estimation and Lag Selection

Johansen Cointegration Test. The Johansen cointegration test (Johansen 1991) is a multivariate statistical method that determines the number of cointegration relationships among a set of non-stationary time series. Unlike the Engle-Granger pairwise approach, the Johansen test accounts for multiple cointegration vectors and allows for a full-system estimation under a Vector Error Correction Model (VECM) framework. By applying the test to the vector Y_t of log-prices, we can assess the existence and dimensionality of the long-term equilibrium relationships among the four series.

Let $Y_t = [\log P_t^{\text{Brent}}, \log P_t^{\text{Shanghai}}, \log P_t^{\text{Dubai}}, \log P_t^{\text{WTI}}]^\top$ denote the vector of log-prices. To proceed with the Johansen test, we first estimate a VAR(p) model for Y_t , where the lag order p is selected via the minimization of the Schwarz Information Criterion (SIC) (Schwarz 1978).

$$SIC(p) = T \cdot \log |\hat{\Sigma}_u| + (Np^2 + N) \cdot \log T \quad (3.1)$$

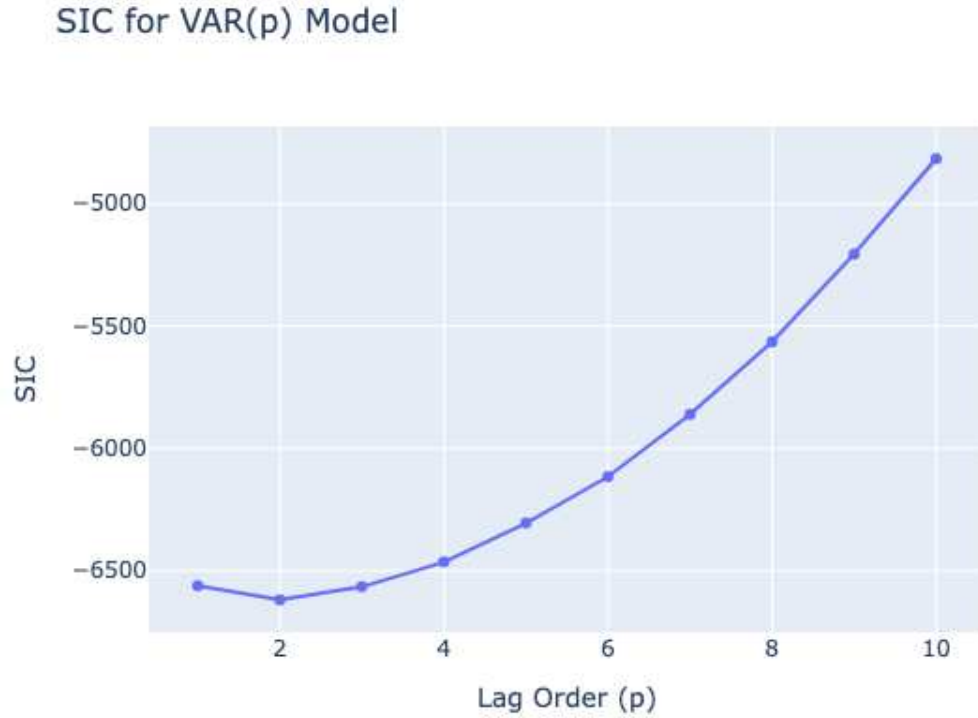


FIGURE 3.2. SIC values for VAR (p) model lag selection.

Figure 3.2 displays the SIC values for $p = 1$ to $p = 10$. The minimum SIC suggests the optimal lag order $\hat{p}$, which is $\hat{p} = 2$, balancing model complexity and goodness of fit. The code used for VAR computation and SIC minimization is provided in Appendix A.2, leading to the results summarized in Table A.3.

The regression output confirms the relevance of several lagged terms across all equations, with many coefficients statistically significant at conventional levels. For instance, in the first equation (referred to the first asset, Brent) y_1 , both $L1.y1$ and $L2.y2$ are highly significant, indicating strong autoregressive behavior. Cross lag effects such as $L1.y3$ and $L2.y3$ highlight dynamic interaction across assets, suggesting that spread dynamics are influenced by more than just own-history. The overall model fit is robust as shown by the high log-likelihood 2208.02 and very low **Final Prediction Error** ($FPE = 2.32 \cdot 10^{-12}$), while residual correlation matrix reveals substantial co-movement among the series, with especially strong correlation between y_1 and y_4 with a $\rho = 0.93$. These findings justify and demonstrate the correct identification of a VAR (2) system for modeling this joint log-price evolution,

supporting furthermore the assumption of interdependent asset behavior in the system.

3.2.3 Johansen Trace Test

Using the optimal lag order from the VAR estimation, we perform the Johansen trace test for cointegration¹. The null hypothesis H_0 : ‘there exist less than or equal to r cointegrating relationships’ is tested sequentially against the alternative H_1 : ‘ $r + 1$ or more’. The test statistic is compared against critical values at the 10%, 5%, and 1% levels.

r	Reject H_0	Trace Stat.	Critical Val.	p-Value
0	1	157.792	47.854	<0.05
1	1	48.622	29.796	<0.05
2	1	20.225	15.494	<0.05
3	0	3.225	3.842	>0.05

TABLE 3.3. Johansen trace test results for cointegration rank. Reject H_0 means presence of at least r cointegrating relations.

The Johansen trace test results in Table 3.3 (associated with the code in Appendix A.4) strongly rejects the null hypothesis of at most r cointegrating vectors for $r = 0, 1$, and 2 , with test statistics far exceeding the corresponding 5% critical values. However, the null is not rejected at $r = 3$, implying that the system admits exactly $r = 3$ cointegrating relationships. This suggests a high degree of long-run equilibrium interaction among the four crude oil futures series, consistent with full-rank cointegration in a system of four endogenous variables. The test specification includes an intercept in the cointegrating relations, allowing for non-zero means in the spread dynamics.

The significance of the results justifies the use of cointegration-based modeling approaches, such as the Vector Error Correction Model (VECM), to capture both short-term dynamics and long-term equilibrium relationships among the series. The next step is to construct the cointegration spread process S_t based on the estimated cointegration vectors.

¹The **Johansen Test for Cointegration** refers to a framework for testing cointegration as specified in Section 3.2.2, whereas the mentioned **Johansen Trace Test** provides one specific test statistic within that framework, namely used to determine the number of cointegrating relationships. Johansen (1988)

3.2.4 Cointegration Spread Construction

The Johansen procedure in Section 3.2.3 estimates the cointegration matrix $\beta \in \mathbb{R}^{N \times r}$, as part of the VECM representation, where N is the number of time series (in the case at hand, $N = 4$: Brent, Shanghai, Dubai, WTI), and r is the cointegration rank determined by the trace test, $r = 3$. Each column of β represents a stationary linear combination of the original non-stationary log-price series as tested in Section 3.2.1.

To construct the spread, we extract the first column of β , which corresponds to the leading eigenvector associated with the largest eigenvalue of the Π matrix. This vector captures the dominant long-term equilibrium relationship among the four series.

Π Matrix. Once cointegration among non-stationary time series has been established, we use a Vector Error Correction Model, which captures both short-term dynamics and long-term equilibrium relationships. The VECM is a restricted form of the Vector Autoregressive (VAR) model designed for use with cointegrated series, specifically because deviations from the long-run equilibrium affect the short-run evolution of the system because of the error correction term. The VECM can be expressed as follows:

$$\Delta Y_t = \mu + \Pi Y_{t-1} + \sum_{i=1}^{p-1} \Gamma_i Y_{t-i} + \epsilon_t \quad (3.2)$$

here, $Y_t \in \mathbb{R}^N$ is the vector of non-stationary (integrated of order one, $I(1)$) log-price series, μ is a drift term, and ϵ_t is a vector of white noise residuals. The matrix $\Pi \in \mathbb{R}^{N \times N}$ captures the long-term equilibrium relationships among the variables.

If the series in Y_t are effectively cointegrated, then Π is of reduced rank $r < N$ and can be decomposed as:

$$\Pi = \alpha \beta^\top \quad (3.3)$$

where:

- $\beta \in \mathbb{R}^{N \times r}$ is the **cointegration matrix**, whose columns are the **cointegrating vectors** (stationary linear combinations of Y_t), and
- $\alpha \in \mathbb{R}^{N \times r}$ is the **loading matrix**, whose entries reflect how each individual series adjusts toward the long-run equilibrium defined by $\beta^\top Y_t$, namely, the speed with which deviations from the cointegration relationship are corrected.

This factorization ensures that $\Pi Y_{t-1} = \alpha \beta^\top Y_{t-1}$ captures deviations from equilibrium

and corrects them, as specified above, by the adjustment speeds in α . (Johansen 1988)

Eigenstructure and Cointegration Vectors. In the Johansen procedure, the cointegration vectors are obtained by solving a generalized eigenvalue problem derived from the maximum likelihood estimation (MLE) of the VECM under the assumption of Gaussian errors specified by the term ϵ_t in the VECM. Specifically, this involves finding eigenvectors β that maximize the likelihood function subject to the rank restriction on $\Pi = \alpha\beta^\top$ given by the cointegration rank r . The eigenvalue problem can be expressed as:

- Eigenvalues $\lambda_1 > \lambda_2 > \dots > \lambda_r > 0$, which indicate the strength of each cointegrating relationship, and
- Eigenvectors, namely the columns of β , that define the stationary spreads.

The estimated eigenvalues for our system are:

$$\lambda = \begin{bmatrix} 0.3201 \\ 0.0955 \\ 0.0583 \\ 0.0113 \end{bmatrix} \quad (3.4)$$

with the associated cointegration matrix:

$$\beta = \begin{bmatrix} 2.63 & 13.30 & 28.06 & 2.08 \\ 3.90 & -8.62 & 18.40 & -5.27 \\ 28.50 & -0.29 & -10.13 & -0.45 \\ -35.24 & -4.57 & -37.70 & 0.12 \end{bmatrix} \quad (3.5)$$

It should be noted that the cointegration matrix β is not normalized, meaning that the cointegration vectors have not been scaled or standardized in any particular way to retain the original units of the log-prices. This is typical in cointegration analysis, as the focus is on the relationships between series rather than their absolute levels. In practice, β is identified only up to a normalization factor, because any scalar multiple of a cointegrating vector defines the same stationary linear combination.

Spread Construction. The first eigenvector $\beta_1 = \beta(:, 1)$ is the leading eigenvector of the matrix associated with the value $\lambda_1 = 0.3201$, which captures the dominant cointegration relationship among the log-price series. Hence the construction of the cointegration spread S_t is done as follows:

$$S_t = \beta_1^\top Y_t \quad (3.6)$$

where Y_t is the vector of log-prices at time t , whereas S_t is a synthetic spread process that is stationary by construction, as it fluctuates around a long-run mean due to the cointegrating relationship captured by β_1 . To eliminate any determinist drift or bias, S_t is mean adjusted by subtracting the sample, long-term mean $\bar{S} = \frac{1}{T} \sum_{t=1}^T S_t$, where T is the total number of observations in the sample. This ensures that the spread oscillates around zero, making it suitable for mean-reversion strategies, and allowing a clearer visualization of its dynamics. The resulting time series, shown in Figure 3.3, represents the core signal for subsequent development.

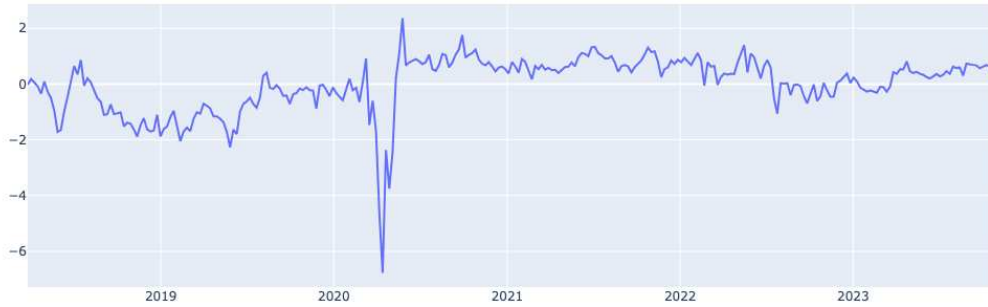


FIGURE 3.3. Cointegration spread process S_t .

While the spread process S_t is stationary, we refit a VECM on the original data under the chosen cointegration rank $r = 4$. This dynamic model extends beyond the static eigenvector solution by explicitly capturing both the long-run equilibrium relationship and the short-run adjustment dynamics through the error correction term α and the lagged differences ΔY_t . This refitting allows us to quantify how deviations from equilibrium correct over time, fully characterizing the spread's behavior within the VECM framework.

Visual Inspection of S_t . The spread S_t exhibits mean-reverting behavior, as expected from the cointegration relationship. Nonetheless, between 2020 and 2021, the series shows a sharp deviation from its usual oscillations. This period corresponds to the COVID-19 pandemic, which caused unprecedented disruptions in the global crude oil markets. These disruptions have been likely driven by a combination of demand collapse, extreme supply volatility, and inventory build-ups. Such shocks

affect the assumption of stationarity underpinning the cointegration relationship, and quantitatively, the error correction speed might slow down during these periods, making mean reversion weaker and riskier.

Diagnostics. To validate the mean-reverting nature of the constructed spread S_t , we analyze its autocorrelation function (ACF). The ACF is, as expected, decaying slowly, indicating persistence and consistency of the spread with an autoregressive structure. The PACF, on the other hand, shows rapid decay after lag 1. These findings support and reinforce the assumption of mean-reversion studied through the VECM.

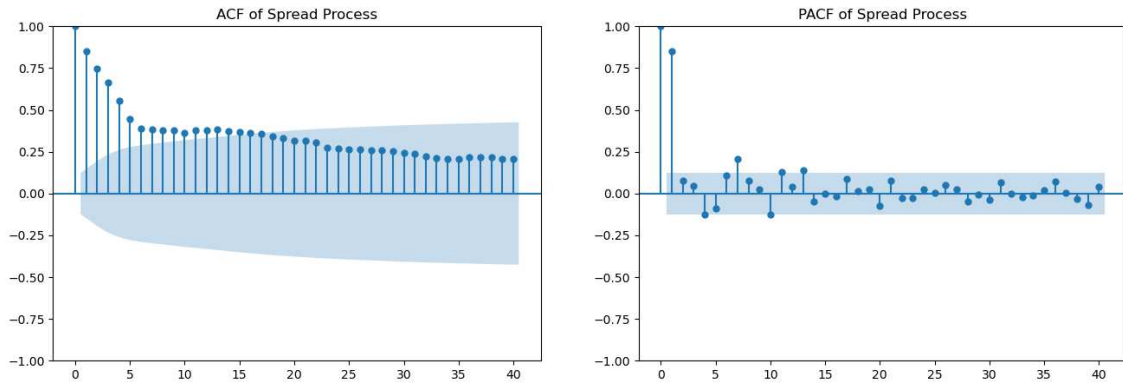


FIGURE 3.4. ACF and PACF of the spread process S_t .

Daily Cointegration Spread Construction. To enhance the temporal resolution and enable a more granular regime identification, we extend the cointegration framework to daily frequency. After estimating the cointegration vector β_1 on the weekly dataset, we construct the daily spread process using the same vector applied to the daily log-prices, this is done because the cointegration relationship is expected to hold at higher frequencies as well. The daily spread S_t^{daily} is defined as:

$$S_t^{\text{daily}} = \beta_1^\top Y_t^{\text{daily}} \quad (3.7)$$

ensuring that the spread remains consistent with the equilibrium relationship captured by the weekly johansen analysis. The resulting series is then standardized by removing its sample mean and dividing by its standard deviation to eliminate any possible deterministic drift and facilitate comparability across periods and strategies. This step preserves the mean-reverting structure of the original spread, while allowing the stochastic model and strategies to leverage higher-frequency signals with greater responsiveness and accuracy. The standardized spread serves as the primary input for the regime-switching models and strategy execution described in the following

sections. The code associated with the computation of the daily spread is available at Appendix A.5.

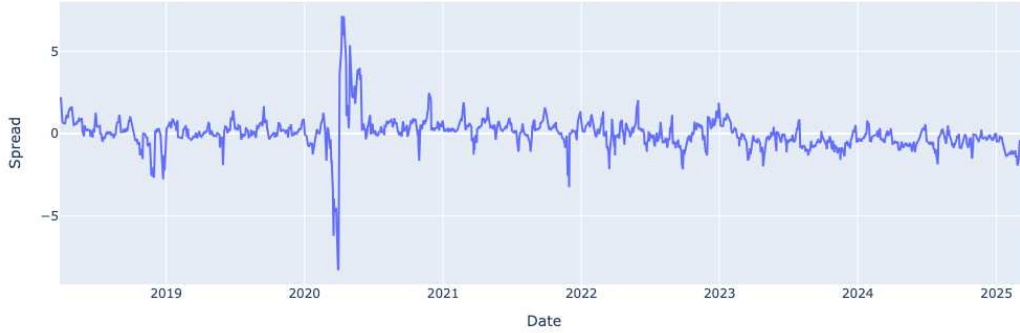


FIGURE 3.5. Cointegration spread process S_t obtained at a daily frequency.

3.3 OU-HMM Implementation

In this section, we provide a detailed explanation of the implemented “OUHMM” class, which operationalizes the theoretical model introduced in Section 2.2.4. The design follows the hybrid structure of the Ornstein-Uhlenbeck Hidden Markov Model (OU-HMM) as outlined in Fanelli, Fontana, and Rotondi (2023), incorporating both continuous-time mean-reverting dynamics and discrete regime-switching behavior. (Appendix A.6)

The key implementation challenge lies in initializing and estimating the regime-specific parameters $\theta_i, \mu_i, \sigma_i$ and the transition matrix $P = [p_{ij}]$ in a data-efficient and statistically consistent manner. The class employs a two-phase approach: a heuristic initialization via clustering, followed by a recursive filtering and parameter updating procedure in line with the classical EM framework in classical HMMs.

3.3.1 Model Initialization and Clustering

The estimation begins with the model comparison results presented in Table 3.4. Based on the previously introduced Schwarz Information Criterion (SIC) and log-likelihood, we select the optimal number of hidden states $N = 3$.

N states	Log-Likelihood	BIC
1	-1109.599479	2241.402651
2	-314.960579	696.532240
3	-181.331623	488.484178
4	-520.239422	1050.971970
5	-1000.349752	2000.342975

TABLE 3.4. Model comparison results for different numbers of hidden states in the HMM. The optimal N (number of hidden states) according to SIC/BIC: 3.

Practical Initialization via Clustering. Although the OU-HMM is formally defined in continuous time with latent, unobserved regimes, the initial estimation of its parameters in this work adopts a practical data-driven approach to enable efficient and robust parameter calibration. Specifically, a **K-Means Clustering** algorithm is applied to the lagged daily spread series S_t within a burn-in period to partition the data into K preliminary regimes.

This clustering is not meant to estimate the latent states Z_t directly, but rather to construct an empirically-informed *initial guess* for the regime-specific OU parameters $\theta_i, \mu_i, \sigma_i$. This hybrid strategy serves as a warm start for the Expectation-Maximization algorithm embedded in the model's recursive filtering. By using clustering to approximate similar mean-reversion behavior across the spread, the model avoids poor local minima during early iterations of filtering and parameter updates. The clustering is particularly useful when data exhibits strong regime patterns that are not easily captured through random initialization.

This implementation choice reflects a common practice in empirical finance, where data complexity often necessitates heuristic-based pre-estimation. (Appendix A.7)

Estimation of Initial OU Parameters. The model estimates the parameters governing each regime's OU process using method-of-moments estimators rooted in discrete-time AR (1) approximations (Hamilton 1989). The persistence of the process, encoded by the AR (1) coefficient $\hat{\alpha}_i$ is computed for each cluster by regressing S_t on S_{t-1} within the corresponding group. Mathematically, this is expressed as:

$$\hat{\alpha}_i = \frac{\sum_{t \in \mathbb{C}_i} S_t S_{t-1}}{\sum_{t \in \mathbb{C}_i} S_{t-1}^2} \quad (3.8)$$

This coefficient represents the conditional linear dependency of S_t on its immediate past and reflects the discrete analog of mean reversion. To transition to the

continuous-time domain, which is central in Fanelli, Fontana, and Rotondi (2023) framework, the mean-reversion speed coefficient θ is derived using:

$$\hat{\theta}_i = -\log(\hat{\alpha}_i) \quad (3.9)$$

which links discrete-time autoregression to the exponential decay property of the Ornstein-Uhlenbeck process.

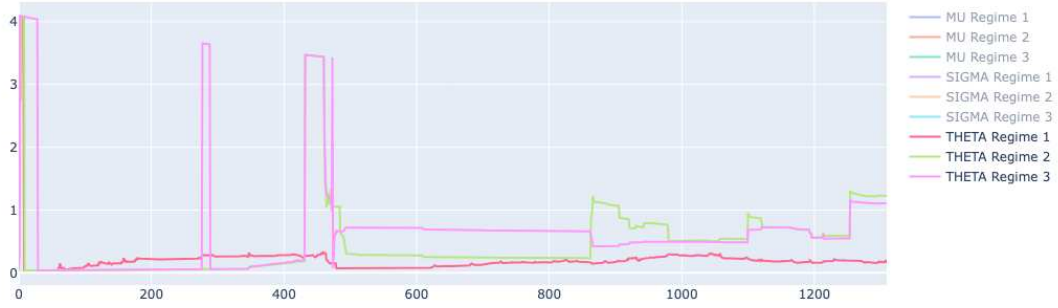


FIGURE 3.6. Recursive updates of the parameter θ_i throughout the fit loop.

Once $\hat{\alpha}_i$ and $\hat{\theta}_i$ are obtained, the model estimates the regime-specific long-run mean μ_i based on the stationarity condition:

$$\hat{\mu}_i = \frac{\gamma_i}{1 - \alpha_i}, \quad \text{where} \quad \gamma_i = \mathbb{E}[S_t] - \alpha_i \mathbb{E}[S_{t-1}] \quad (3.10)$$

This relationship is derived from the stationary solution of the AR (1) process and plays a fundamental role in our modeling framework. It guarantees that each latent state is characterized by its own unique long-term mean, thereby assigning a distinct equilibrium value to every state in the model. In essence, the relationship captures the average behavior of the process when it is in equilibrium, while also taking into account the effect of persistence, represented by the parameter α_i . This parameter quantifies how strongly the process tends to revert back toward its equilibrium level after experiencing a deviation, thus ensuring that the long-term mean reflects not only the inherent central tendency of the system but also the influence of how past deviations decay over time.

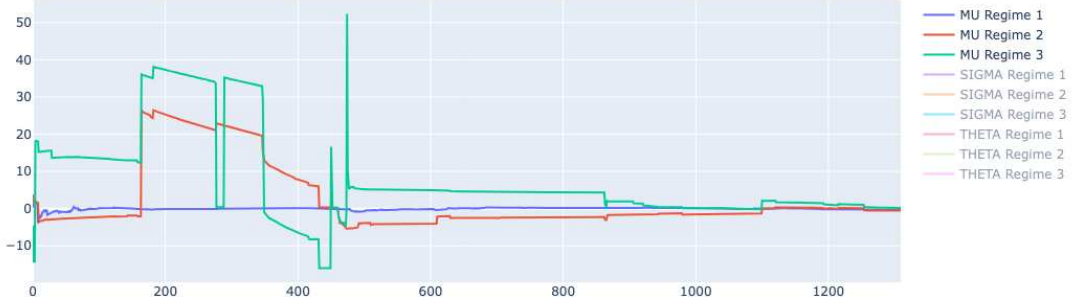


FIGURE 3.7. Recursive updates of the parameter μ_i throughout the fit loop.

The short-term volatility σ_i is estimated using the variance of the residuals from the regime-specific AR (1) regression, adapted into the continuous-time OU framework via:

$$\hat{\sigma}_i = \sqrt{\frac{2\theta_i\eta_i^2}{1 - \alpha_i^2}} \quad (3.11)$$

where η_i denotes the standard deviation of the AR (1) residuals in regime i .

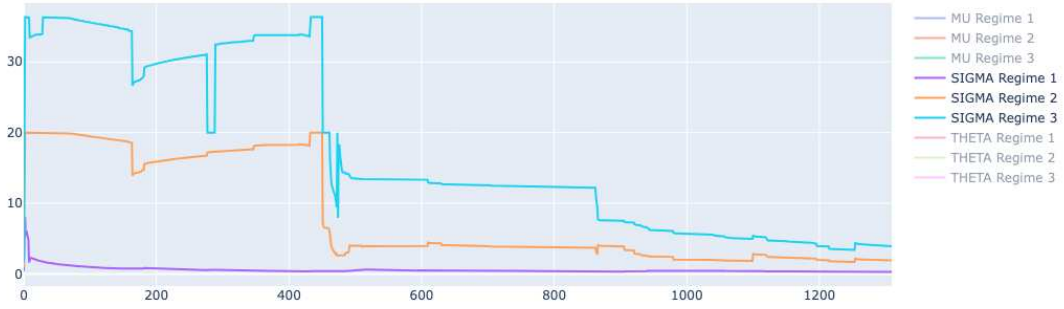


FIGURE 3.8. Recursive updates of the parameter σ_i throughout the fit loop.

This transformation respects the variance of the exact solution of the OU process and ensures consistency with the conditional Gaussian transition density later used in the emission probability step. Importantly, these parameter estimates are not maximum likelihood estimates (MLEs), but rather serve as empirical starting values for the iterative inference procedure. This method aligns with the strategy outlined

in Fanelli, Fontana, and Rotondi (2023), who similarly stress the importance of well-initialized and well-separated latent regimes in filtering applications. (Appendix A.8)

Transition Matrix and State Probabilities. The transition dynamics between regimes are inferred by analyzing observed transitions in the sequence of assigned labels. The transition matrix $P = [p_{ij}]$ is estimated using empirical frequency counts:

$$p_{ij} = \frac{\text{Number of transitions from regime } i \text{ to } j}{\text{Total transitions from regime } i} \quad (3.12)$$

Each row is normalized to ensure that $\sum_j p_{ij} = 1$, maintaining the stochasticity condition required of a proper Markov transition matrix. This empirical estimation captures the latent market-switching behavior and initializes the model's state dynamics before recursive inference.

The initial regime distribution π_0 is similarly estimated based on the frequency of initial cluster assignments in the burn-in period. To avoid issues arising from rare transitions or underrepresented states, Laplacian smoothing is applied to both π_0 and P , thereby preventing degeneracies and numerical instabilities in early iterations.

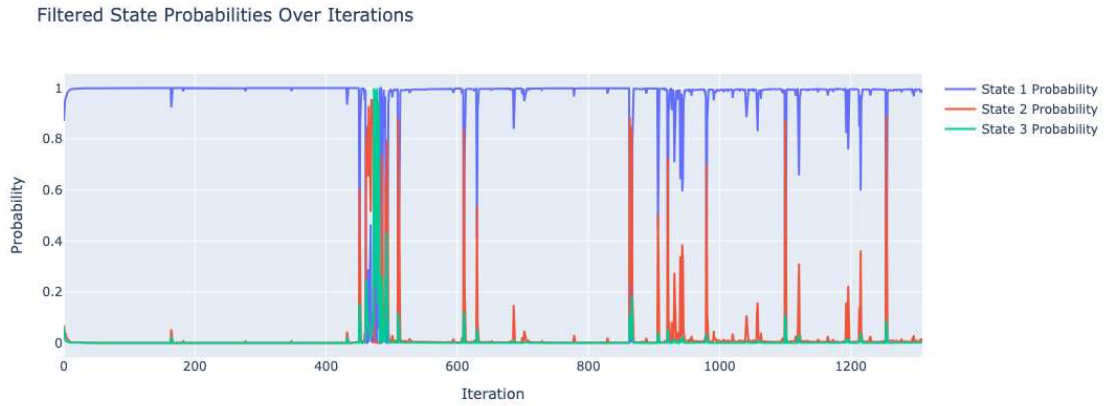


FIGURE 3.9. Filtered State Probabilities over Iterations. Visually, we can clearly see the perceived difference between the regimes, with Regime 1 being associated with a large probability at all times, except some rare occasions in which the spread process enters Regime 2 before entering Regime 3.

To enhance identifiability, a **Kullback-Leibler (KL)** (Appendix A.9) divergence constraint is enforced across all regime pairs. Specifically, if the KL divergence between any pair of Gaussian emissions is below a threshold (e.g., 0.2), the corresponding regime parameters $\mu_i, \theta_i, \sigma_i$ are perturbed to ensure that each regime is statistically distinguishable, preventing mode collapse during filtering, and supporting robust convergence. Such identifiability enforcement is conceptually aligned with Bayesian regularization strategies and has shown strong empirical value in

regime-switching models (Yau et al. 2011), including the framework proposed by Fanelli, Fontana, and Rotondi (2023). (Appendix A.10, Appendix A.11)

Recursive Filtering and Expectation-Maximization Updating. Following initialization, the model proceeds with a recursive filtering phase in which both the hidden state probabilities and regime parameters are updated over time. This structure represents an online variant of the Expectation-Maximization (EM) algorithm. The E-step is approximated via recursive forward filtering, which yields the latent regime probabilities α_t . Instead of a global maximization in the M-step, model parameters are updated incrementally using exponentially-weighted sufficient statistics J_{ij}^t , O_i^t , and $G_i^t(f)$. This streaming approach allows the model to adapt to structural changes in real time while maintaining statistical coherence as seen in Cappé and Moulines (2009).

At each time t , the conditional density of the current spread S_t given the previous observation S_{t-1} and latent state $Z_t = i$ is given by the OU transition density:

$$(S_t | S_{t-1}, Z_t = i) \sim \mathcal{N}\left(\mu_i + (S_{t-1} - \mu_i)e^{-\theta_i}, \frac{\sigma_i^2}{2\theta_i}(1 - e^{-2\theta_i})\right) \quad (3.13)$$

This emission distribution captures both the deterministic drift toward the long-run mean μ_i and the exponentially decaying conditional variance of the OU process. Using this likelihood, the forward probabilities $\alpha_t^i = \mathbb{P}(Z_t = i | S_{1:t})$ are updated recursively via Bayes' rule:

$$\alpha_t^i = \frac{\sum_{j=1}^K \alpha_{t-1}^j p_{ji} \cdot p(S_t | S_{t-1}, Z_t = i)}{\sum_{k=1}^K \sum_{j=1}^K \alpha_{t-1}^j p_{jk} \cdot p(S_t | S_{t-1}, Z_t = k)} \quad (3.14)$$

This filtering step maintains a full posterior distribution over regimes at each time point, enabling soft regime classification and probabilistic decision-making for strategy design. To support online parameter estimation, the algorithm maintains the vector of sufficient statistics for each regime $([J_{ij}^t, O_i^t, G_i^t(f)])$, tracking weighted expectations of S_t , S_{t-1} , cross-products, and squared residuals. These statistics are updated incrementally using a learning rate ρ_t that decays over time (e.g., $\rho_t = t^{-0.5}$), ensuring convergence while preserving adaptability. (Appendix A.11, Appendix A.14)

From the updated sufficient statistics, new values of α_i , γ_i , η_i are derived, which are in turn used to re-estimate the OU parameters $(\theta_i, \mu_i, \sigma_i)$ via the closed-form moment-based formulas. The transition matrix P is also re-estimated using expected state transitions implied by α_t , further adapting the model to recent structural

changes.

This recursive estimation framework enables the model to adapt dynamically to evolving market conditions, including shifts in volatility, mean levels, and persistence of the cointegration spread, the phenomena that a static OU process would fail to capture. As emphasized in Fanelli, Fontana, and Rotondi (2023), such adaptability is crucial for effective regime-aware trading and robust statistical arbitrage.

Numerical Stability and Constraints. Because of the recursive and non-linear nature of the OU-HMM filtering process, numerical stability is a core consideration throughout the implementation. Several practical safeguards are embedded in the model to ensure that the output remains statistically sound and financially interpretable.

First, all emission probabilities are clipped within a safe range to avoid underflow when evaluating Gaussian densities, especially in tails. Similarly, logarithmic transformations of probabilities are computed with floor values to prevent computational errors like $\log(0)$. Variance terms are strictly lower-bounded to avoid divisions by zero or ill-conditioned covariance matrices.

Parameter constraints are imposed to maintain economic plausibility and model interpretability. The mean-reversion speed θ_i is bounded within $[10^{-4}, 30]$, ensuring the process remains stationary without becoming excessively stiff. The volatility σ_i is constrained to lie within $[0.2, 20]$, avoiding degenerate zero-volatility regimes or extreme variance outliers. Although these bounds reflect common empirical ranges found in energy futures markets, they specifically come from an empirical analysis performed on the training set of S_t .

These techniques collectively prevent the model from diverging in edge cases, such as during market shocks (as seen during the period of the COVID-19 pandemic), regime-switching ambiguity, or sparse data periods. They are especially important in high-frequency environments where extreme observations and outliers are common. The model thus retains its ability to generalize without overfitting or collapsing, mimicking best practices in the applied regime-switching literature. (Ang and Timmermann 2012), (J. Bulla and I. Bulla 2006) (Appendix A.12, Appendix A.13)

Interpretation. The estimated OU-HMM serves not only as a predictive model for spread dynamics but also as a structural outlook through which latent market states can be inferred. Each hidden regime Z_t is associated with a distinct set of OU parameters $(\theta_i, \mu_i, \sigma_i)$ as highlighted in Section 2.2, offering a high-resolution description of the cointegration spread’s statistical properties at time t .

Throughout this work, three distinct regimes have been identified in the spread dynamics. Empirical evidence highlights meaningful differences across these regimes in terms of volatility, mean reversion speed, and the nature of observed shocks:

- **Regime 1: Low Volatility, Strong Mean Reversion.** This regime represents the most stable phase of the spread process. It is characterized by low variance (σ) and a relatively high speed of mean reversion (θ). It captures the “quiet” periods in the market, with dynamics centered around the long-term mean and minimal disturbances.

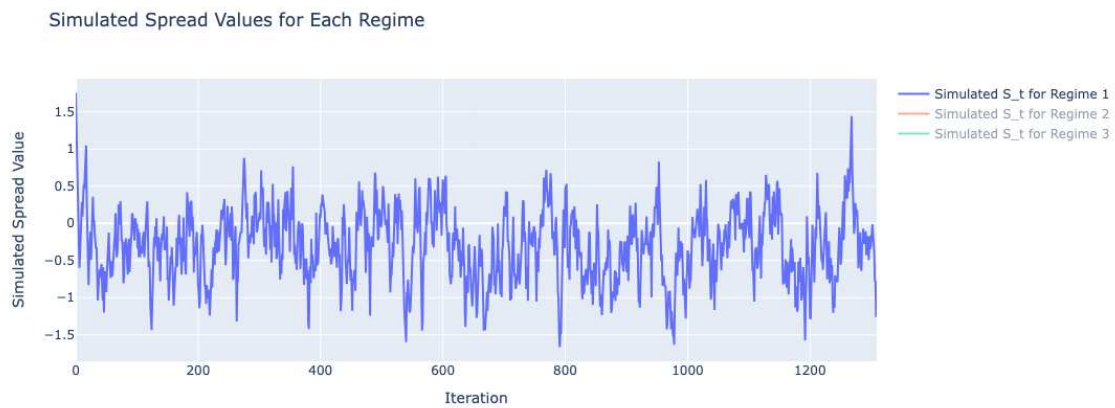


FIGURE 3.10. Simulation of **Regime 1** given estimated parameters through Ornstein-Uhlenbeck exact equation.

- **Regime 2: Moderate Volatility, Persistent Mean.** Regime 2 maintains a similar long-term mean to Regime 1 but exhibits significantly higher volatility. While mean-reverting behavior remains present, the spread oscillates with more amplitude, reflecting phases of increased market uncertainty without full breakdowns in structure.

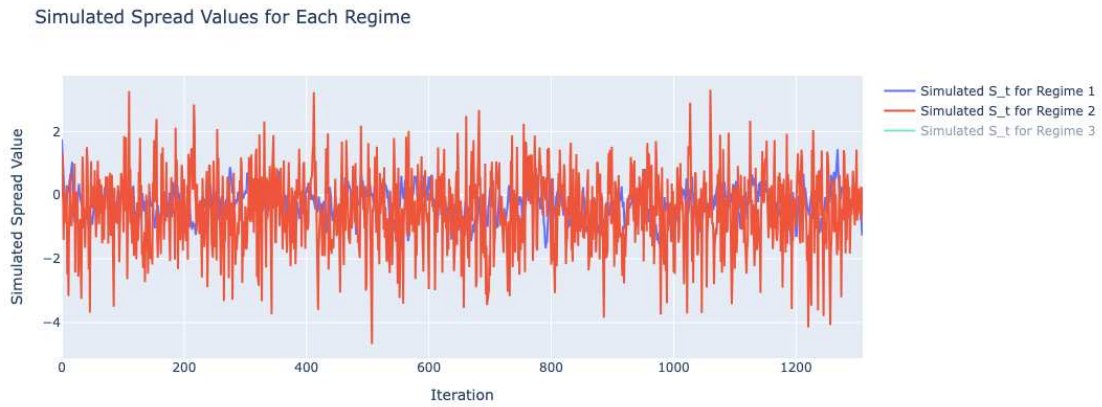


FIGURE 3.11. Simulation of **Regime 2** overlaying Regime 1 given estimated parameters through Ornstein-Uhlenbeck exact equation.

- **Regime 3: High Volatility, Extreme Deviations.** This regime captures highly turbulent periods, with extreme variance and wider deviations from the mean. Although mean reversion is still statistically significant, the regime is marked by the presence of outliers and abrupt shocks, pertaining to periods of structural breaks or black swan events, such as those seen during the COVID-19 pandemic.

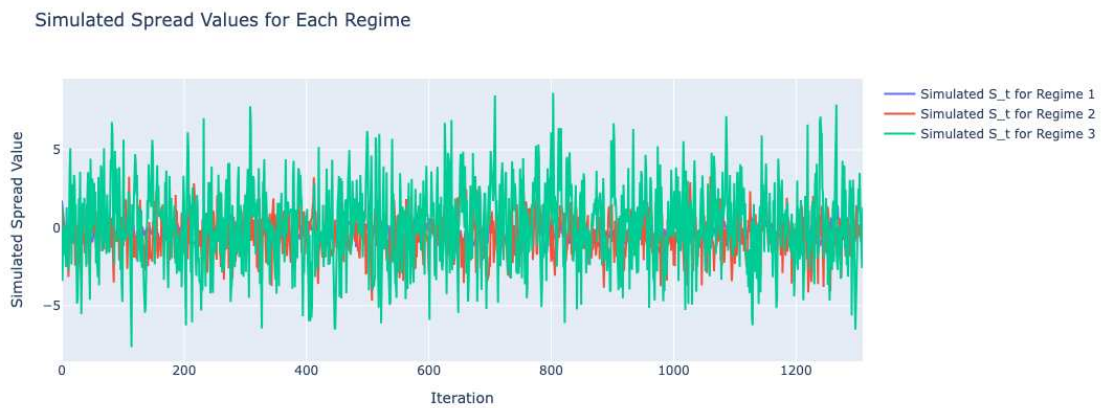


FIGURE 3.12. Simulation of **Regime 3** overlaying Regimes 1 and 2 given estimated parameters through Ornstein-Uhlenbeck exact equation.

The filtered posterior probabilities α_t^i offer a probabilistic view of the current market regime. Rather than making hard assignments, the model allows traders and researchers to make decisions weighted by uncertainty.

3.4 Trading Strategy Implementation

Following the cointegration analysis performed in Section 3.2, we implement the trading strategies outlined in Section 2.3. The strategies are applied to the cointegration spread S_t as expressed in Equation (3.6), and are evaluated based on their profitability, risk-adjusted returns, and robustness across different market conditions. Every strategy is backtested via a *walk-forward* environment, simulating at best real-life dynamics and information, over the period spanning between October 27th, 2023, through March 25th, 2025.

3.4.1 Benchmark Strategy: Buy and Hold

(Appendix A.15) The implementation of the **Buy and Hold** strategy serves as a foundational benchmark for evaluating the performance of the more sophisticated trading strategies that will follow. The logic behind this implementation is intentionally straightforward: it simulates the behavior of a passive investor who allocates a fixed amount of capital at the start of the trading period and holds the position until the end, without engaging in any rebalancing or active management.

In practice, the function takes as input a time series of closing prices and an initial capital amount. The number of shares purchased is determined by dividing the capital by the first closing price available. From that point onward, the portfolio value (i.e., the equity curve) evolves solely based on price changes of the underlying asset. The function computes this equity curve by multiplying the number of shares by the closing price at each time step.

Daily returns are then derived from the equity curve, allowing for further downstream analysis. While the code includes the calculation of key performance metrics, which are later used for evaluation, the primary purpose of this implementation is to establish a baseline strategy that assumes no predictive modeling, and no reactivity to market conditions.

This static investment approach provides a crucial reference point: it allows us to isolate and assess the added value generated by more complex, model-driven strategies by comparing their performance relative to a passive alternative.

3.4.2 Plain Vanilla Strategy Implementation

(Appendix A.16) This implementation of the **Plain Vanilla** strategy is designed to exploit the mean-reverting behavior of a cointegration spread composed of the multiple crude oil futures taken into account in this thesis. Unlike the passive buy-and-hold benchmark, this strategy is built on the assumption that deviations from

the spread's historical mean are temporary (thanks to the *cointegrating* relationship highlighted in Section 3.2) and can be traded upon reversion.

The trading logic follows a walk-forward approach: at each time step, the algorithm computes the historical mean of the spread up to that point and checks for a threshold-crossing event. Specifically, the strategy opens a position when the spread crosses its rolling mean, either from below (indicating an upward reversion) or from above (indicating a downward one). These crossover signals are interpreted as mean-reversion entry points.

Once a trade is initiated, the strategy monitors for a reversion of the spread toward its mean and closes the position when the spread crosses zero in the direction of the initial signal. Position sizing is determined using a beta-weighted allocation across the underlying assets, where capital is distributed proportionally to the magnitude of the beta coefficients. This ensures that the strategy remains neutral with respect to the cointegrated relationship.

Each trade's profit or loss is calculated using the price difference between entry and exit, scaled by the allocated capital and directional exposure. All trades are recorded along with their respective timestamps. The final equity curve is constructed by aggregating trade-level payouts across time, assuming an initial capital base.

This code structure is modular and efficient, capable of handling any number of cointegrated assets provided in the input. It also allows for realistic simulation of a sequential, non-anticipative strategy, making it suitable for out-of-sample backtesting on real market data.



FIGURE 3.13. Trades performed during the backtest of the Plain Vanilla strategy.

3.4.3 Probability Interval Strategy Implementation

(Appendix A.17) The **Probability Interval (ProbI)** strategy is a more statistically-informed enhancement of the plain vanilla approach. It introduces dynamic thresholds based on the rolling standard deviation of the spread to determine entry and exit signals, thereby accounting for local volatility regimes in real time.

This strategy applies a walk-forward framework, where for each point in time, the historical mean and standard deviation of the spread are estimated over a fixed-length rolling window (e.g., 20 days). Using these, upper and lower probabilistic bands are computed as:

$$\text{Upper Band}_t = \mu_t + z_\alpha \cdot \sigma_t, \quad \text{Lower Band}_t = \mu_t - z_\alpha \cdot \sigma_t \quad (3.15)$$

Here, z_α serves as a confidence threshold, typically set at 2.0 to approximate a 95% interval under normality assumptions. A trading signal is generated when the spread deviates significantly from its local mean, i.e., breaches either the upper or lower band. This signifies a statistically rare event under the current distribution, warranting a mean-reversion bet.

Once in a trade, the position is closed when the spread reverts back within the probabilistic corridor, specifically, when the absolute Z-score drops below the chosen threshold. The position sizing logic mirrors that of the Plain Vanilla strategy, leveraging the *beta* coefficients to allocate capital proportionally and ensuring exposure neutrality. Each completed trade is logged, and the cumulative capital is updated accordingly to construct an equity curve. The resulting performance series is then used to evaluate profitability over time.

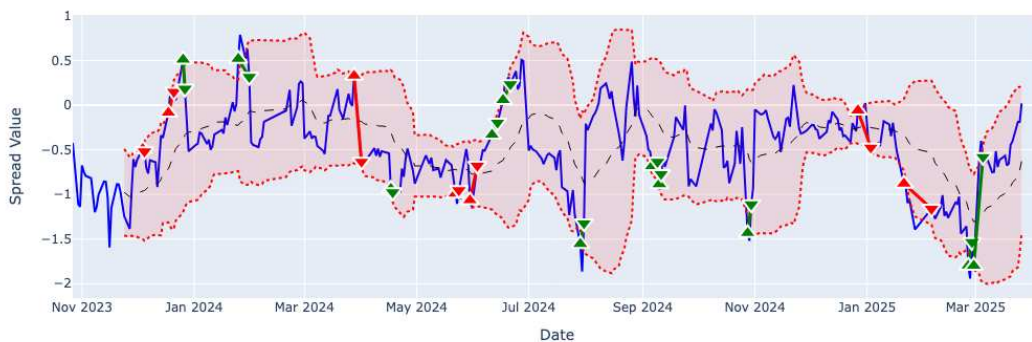


FIGURE 3.14. Trades performed during the backtest of the Probability Interval (ProbI) strategy, along with the probabilistic bounds.

3.4.4 Realized Increment Strategy Implementation

(Appendix A.18) The **Realized Increment (RI)** strategy represents a data-driven approach designed to detect sharp relative changes in the spread process. Unlike the previous methods, which operate on the absolute level of the spread or its normalized deviation from the mean, this strategy monitors the first-order difference of the spread in relative terms, referred to as the realized increment, to capture directional momentum and local shocks.

Mathematically, the realized increment at time t , denoted x_t , is computed as:

$$x_t = \frac{S_t}{S_{t-1}} - 1 \quad (3.16)$$

This transformation effectively measures the percentage change of the spread from one day to the next, emphasizing short-term dynamics and allowing for a non-stationary interpretation of momentum. The series of x_t is evaluated using a rolling window of fixed length k , from which the lower and upper empirical quantiles q_{down} and q_{up} are extracted (e.g., the 2.5th and 97.5th percentiles, respectively). When a new observation x_t falls outside of this interval, i.e. is unusually low or high relative to the recent empirical distribution, it is interpreted as a statistical anomaly. A position is opened in the direction opposite to the extreme increment, in expectation of a reversion. The trade is held until the realized increment reverts back within the inner quantile band, indicating a return to normal market behavior. This exit condition is designed to prevent prolonged exposure to trending anomalies and ensures that profits are captured as the spread stabilizes. As in previous implementations, the strategy simulates capital allocation using the cointegration vector β , with proportional capital weights and exposure-neutral sizing.

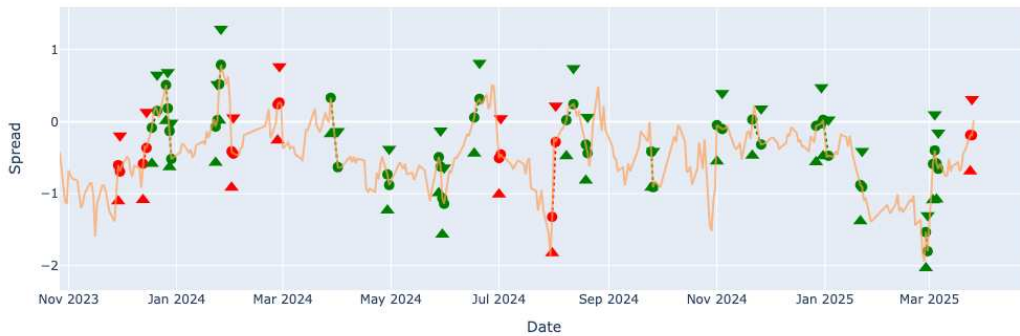


FIGURE 3.15. Trades performed during the backtest of the Realized Increment (RI) strategy.

3.4.5 Prediction Interval Strategy Implementation

(Appendix A.19) The **Prediction Interval (PredI)** strategy is a fully adaptive regime-switching trading system that dynamically updates its parameters in real time by exploiting the online filtering and estimation framework introduced in Section 2.2. Unlike the previous methods, which rely on empirical thresholds or stationary dynamics, this strategy generates model-based probabilistic forecasts for the spread. At each time t , the strategy uses the current HMM-filtered belief over hidden regimes, encoded in the vector α_t , to produce a prediction interval for the next spread value S_t , conditional on S_{t-1} . Concretely, the forecast is derived from the conditional expectation of the OU process under the most probable regime z_t . The expected spread $\mathbb{E}[S_t \mid z_t]$ is modeled as:

$$\mathbb{E}[S_t \mid S_{t-1}, z_t] = \hat{\gamma}^{(z_t)} + \hat{\alpha}^{(z_t)} S_{t-1} \quad (3.17)$$

where $\hat{\gamma}^{(z_t)}$ and $\hat{\alpha}^{(z_t)}$ represent the regime-specific OU parameters. The prediction interval is then constructed symmetrically around this forecast using a user-defined quantile factor q_α such as 1.96 for a 95% confidence level. The entry rule is triggered when the observed spread S_t violates this interval, signaling a statistically unexpected market deviation under the current model. This acts as a high-confidence indicator for mean reversion, prompting a contrarian position. Importantly, the entire model is updated online after each observation via the *SufficientStatistics* object, which accumulates the necessary data to refine the regime probabilities, OU parameters, and transition matrix in real time. The emission probabilities are recalculated at every step, ensuring the model remains calibrated to the evolving spread dynamics.

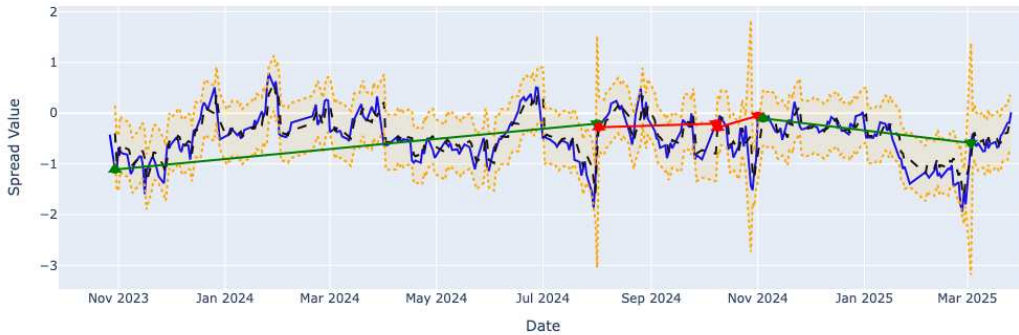


FIGURE 3.16. Trades performed during the backtest of the Prediction Interval (PredI) strategy, along with the probabilistic forecasted bounds.

3.4.6 Predicted Increment Strategy Implementation

The final trading rule explored in this work, detailed in Appendix A.20, is the **Predicted Increment (PI) Strategy**. This strategy leverages the forward-looking nature of the OU process, similarly to the *PredI* strategy, to forecast the next-period return of the spread and trades only on statistically rare deviations of this forecast.

At the core of the approach is the predicted increment $\hat{x}_{t+1}$, defined as the model-implied return:

$$\hat{x}_{t+1} = \frac{\mathbb{E}[S_{t+1} \mid S_t, z_t]}{S_t} - 1 \quad (3.18)$$

This is computed at each step using the regime-specific OU forecast:

$$\mathbb{E}[S_{t+1} \mid S_t, z_t] = \mu^{(z_t)} + (S_t - \mu^{(z_t)})e^{-\theta(z_t)} \quad (3.19)$$

The emission probabilities for each regime are computed using the conditional OU distribution, and the filtering step updates the current belief vector α_t , capturing the probability of being in each hidden state.

Once the predicted increment $\hat{x}_{t+1}$ is obtained, it is compared against empirical quantiles of its own historical distribution. The strategy opens a position only when $\hat{x}_{t+1}$ lies outside of the interval $[q_{\text{lower}}, q_{\text{upper}}]$, that is, when the model expects an unusually large deviation in the spread movement. In this setup, the forecast serves as a dynamic signal whose statistical extremeness determines entry timing. The use of the forecast's own history for thresholding makes the strategy non-parametric and self-adaptive, adjusting to different volatility regimes without the need for fixed thresholds. Exit occurs symmetrically when the predicted increment re-enters the quantile band, signaling a dissipation of the extreme signal and a likely mean-reverting completion of the cycle.



FIGURE 3.17. Trades performed during the backtest of the Predicted Increment (PI) strategy.

Chapter 4

Results

4.1 Trading Strategies Empirical Performance

This chapter presents the comprehensive empirical analysis of the trading strategies developed throughout this study. Following the theoretical framework established in previous chapters and the methodology outlined for trading strategies implementation, we now turn to the critical evaluation of real-world performance across different market conditions and asset classes. The empirical investigation encompasses three key dimensions of strategy assessment. First, we examine the fundamental performance metrics that quantify the profitability and efficiency of each trading approach, including returns, volatility, and risk-adjusted performance measures. Second, we conduct a thorough risk analysis to understand the potential downside exposures and tail risk characteristics inherent in each strategy. Finally, we provide a comparative analysis that positions our findings within the broader context of quantitative trading literature and benchmarks our results against established market indices and alternative investment strategies. The results presented here form the foundation for the strategic recommendations and conclusions that follow, offering both theoretical insights into market behavior and practical guidance for portfolio management applications.

4.2 Performance Measures

Strategy	Cumulative Return (%)	Max Drawdown (%)	Sharpe Ratio	Calmar Ratio
Buy and Hold	16.30	19.35	1.54	3.24
Plain Vanilla	-0.03	0.03	-1.12	-0.77
Probability Interval	1.40	0.84	0.90	1.28
Realized Increment	2.17	1.19	1.12	1.39
Prediction Interval	0.87	0.40	0.69	1.67
Predicted Increment	0.20	0.99	0.15	0.16

TABLE 4.1. Trading Strategies Performance Comparison

Table 4.1 reports the performance outcomes of the six trading strategies analyzed in this study, measured across four fundamental dimensions: cumulative return, maximum drawdown, Sharpe ratio, and Calmar ratio. These metrics collectively offer a comprehensive evaluation of each strategy's profitability, risk profile, and reward-to-risk efficiency.

Cumulative Return. This metric captures the overall return generated by each strategy over the evaluation period, expressed as a percentage of initial capital. It represents the most direct measure of profitability. The **Buy and Hold** benchmark yields the highest cumulative return at **16.30%**, reflecting the general upward drift of asset prices during the sample period. Among the cointegration-based strategies, the **Realized Increment (RI)** strategy delivers the strongest return at **2.17%**, followed by the **Probability Interval (ProBI)** at **1.40%**. The **Prediction Interval (PredI)** and **Predicted Increment (PI)** strategies yield modest yet positive returns of **0.87%** and **0.20%**, respectively. The **Plain Vanilla** strategy, in contrast, produces a slightly negative cumulative return of **-0.03%**, essentially breaking even.

Maximum Drawdown. Drawdown measures the largest peak-to-trough equity decline experienced during the trading period, providing insight into downside risk and capital vulnerability. The **Buy and Hold** strategy exhibits the highest drawdown at **19.35%**, consistent with its exposure to full market risk. All other strategies, being market-neutral and mean-reversion based, demonstrate substantially lower drawdowns, typically below **1.2%**. Notably, the **Prediction Interval** strategy achieves the lowest drawdown at just **0.40%**, suggesting strong capital preservation

even during volatile periods. The **Plain Vanilla** strategy also records a nearly negligible drawdown (**0.03%**), but this is primarily due to its lack of directional exposure rather than superior risk control.

Sharpe Ratio. The Sharpe ratio measures risk-adjusted returns by normalizing the strategy's average excess return by its volatility. This metric is crucial for quantifying how much return is being earned per unit of risk. The benchmark **Buy and Hold** again dominates, with a robust Sharpe ratio of **1.54**, reflecting its strong performance during a generally favorable market regime. Among the model-driven strategies, **RI** and **ProbI** exhibit relatively attractive risk-adjusted returns, with Sharpe ratios of **1.12** and **0.90**, respectively. These values indicate that the strategies are not only profitable, but also stable in terms of volatility. The **Plain Vanilla** strategy shows a negative Sharpe ratio (**-1.12**), highlighting both poor returns and relatively high volatility compared to its gains. The **PI** strategy returns a Sharpe of only **0.15**, suggesting low signal strength in its forecasts, at least over this specific evaluation window.

Calmar Ratio. The Calmar ratio complements the Sharpe by measuring the return relative to the worst-case drawdown, offering an alternative risk-adjusted view that is more sensitive to tail risk. Interestingly, while **Buy and Hold** maintains strong performance with a Calmar ratio of **3.24**, the **Prediction Interval** strategy achieves a higher Calmar ratio of **1.67** than both **RI** (**1.39**) and **ProbI** (**1.28**), despite having a lower raw return. This illustrates the strategy's remarkable ability to manage drawdowns and preserve capital under adverse conditions. Conversely, the **Plain Vanilla** and **PI** strategies yield very low Calmar ratios (**-0.77** and **0.16**, respectively), confirming their inefficiency in balancing risk and reward.

Interpretation and Implications. These results paint a clear picture of relative strategy performance. While the **Buy and Hold** approach remains the most profitable in absolute terms, it does so at the cost of substantial drawdowns and full market exposure. In contrast, the most effective market-neutral strategy, **Realized Increment**, offers a more favorable balance between risk and reward, achieving a Sharpe ratio above 1 with drawdowns capped at just over 1%. The **ProbI** strategy performs similarly well, demonstrating that simpler, statistically-driven signals (e.g., rolling Z-scores) still hold considerable alpha potential when used on cointegration spreads. The more complex, regime-switching approaches like **PredI** and **PI** perform respectably, especially in risk management, though their returns could potentially improve with further parameter tuning or refined filtering methods.

From a quantitative trading perspective, this analysis confirms that model-driven mean-reversion strategies can compete with passive benchmarks in terms of risk-adjusted returns, even if not in outright profitability. For institutional applications where capital preservation and tail-risk control are paramount, these strategies provide meaningful value and merit consideration as portfolio diversifiers or low-volatility alpha generators.

4.3 Risk Analysis

Metric	Buy & Hold	Plain Vanilla	Prob. Interval	Real. Increment	Pred. Interval	Pred. Increment
<i>Monte Carlo-based Kernel Density Estimation (KDE)</i>						
Mean Sim. Return	86.25%	-2.14%	-2.14%	1.66%	0.66%	0.15%
Std Sim. Return	78.20%	1.86%	1.86%	1.50%	0.97%	1.06%
Skewness	1.430	0.031	0.031	0.018	0.002	0.005
Kurtosis	4.948	0.075	0.075	0.072	0.070	0.071
<i>Value at Risk (VaR)</i>						
VaR 95%	12.75%	5.23%	5.23%	0.83%	0.96%	1.62%
VaR 97.5%	23.58%	5.81%	5.81%	1.30%	1.27%	1.96%
VaR 99%	33.49%	6.41%	6.41%	1.79%	1.59%	2.31%
<i>Conditional Value at Risk (CVaR)</i>						
CVaR 95%	25.59%	5.98%	5.98%	1.43%	1.36%	2.06%
CVaR 97.5%	33.53%	6.46%	6.46%	1.83%	1.62%	2.34%
CVaR 99%	41.52%	7.02%	7.02%	2.28%	1.91%	2.67%

TABLE 4.2. Comparative Analysis of Hedging Strategy Metrics

Monte Carlo-based Kernel Density Estimation (KDE). The distributional properties reveal a clear division between the benchmark and the market-neutral strategies. The **Buy and Hold** strategy is characterized by an exceptionally high mean simulated return of 86.25%, but this comes at the cost of extreme volatility, with a standard deviation of 78.20%. Its strong positive skewness of **1.430** indicates a tendency for large positive returns, but the high kurtosis of **4.948** points to “fat tails”, signifying a greater-than-normal probability of extreme outcomes. On the other hand, the five market-neutral strategies exhibit very low volatility (all below 2.0%) and returns centered around zero. Their near-zero skewness and very low kurtosis suggest highly stable, symmetric, and predictable return distributions with thin tails, implying a much lower risk of surprise events.

Value at Risk (VaR). VaR quantifies the maximum potential loss at specific confidence levels, offering a clear measure of downside risk. The **Buy and Hold** strategy demonstrates significant risk, with a 99% VaR of **33.49%**, meaning there is

a 1% chance of losing at least this amount over the simulated period. The market-neutral strategies showcase a decisively superior capital preservation. The **Realized Increment** and **Prediction Interval** strategies are the standout performers, with exceptionally low 99% VaR values of **1.79%** and **1.59%**, respectively. This highlights their effectiveness in containing losses. The **Plain Vanilla** and **Probability Interval** strategies, while far safer than the benchmark, are the riskiest among the active approaches, with a 99% VaR of **6.41%**.

Conditional Value at Risk (CVaR). CVaR provides a more conservative risk measure by calculating the average loss beyond the VaR threshold. This metric further emphasizes the risk disparity between strategies. The **Buy and Hold** strategy's 99% CVaR is a staggering **41.52%**, indicating that in the worst 1% of scenarios, the average loss is over **40%** of the portfolio's value. The risk-management strength of the active strategies is again evident here. The **Prediction Interval** strategy posts the lowest 99% CVaR at just **1.91%**, followed closely by the **Realized Increment** strategy at **2.28%**. This confirms their robustness against tail risk. Conversely, the **Plain Vanilla** and **Probability Interval** strategies show a much higher 99% CVaR of **7.02%**, reinforcing that they offer less protection against severe market downturns compared to the more advanced mean-reversion models.

Interpretation. This risk analysis paints a vivid picture of the trade-offs inherent in each strategy. The **Buy and Hold** approach, while generating the highest average returns in the simulation, presents the potential for catastrophic losses.

Among the market-neutral options, the **Realized Increment** and **Prediction Interval** strategies emerge as the most compelling from a risk-management perspective. They successfully neutralize market risk, keeping volatility and potential drawdowns (as measured by VaR and CVaR) at remarkably low levels. Their return profiles are modest but achieved with exceptional stability. The **Plain Vanilla** and **Probability Interval** strategies, while still low-risk compared to the benchmark, are demonstrably less efficient at managing tail risk than their more sophisticated counterparts. Ultimately, the choice depends on the investor's objective: maximizing returns with high risk (Buy and Hold) versus prioritizing capital preservation and stable, although lower, returns (Realized Increment and Prediction Interval).

4.4 Results Comparison

A comparative analysis against the existing literature, particularly the work of Fanelli, Fontana, and Rotondi (2023), is essential for contextualizing the outcomes of this thesis. Despite the strong methodological alignment in the use of similar mean-reversion strategies and risk assessment techniques like VaR and KDE, the empirical results of the two studies diverge. This divergence is not a contradiction but rather a profound illustration of how the efficacy and nature of statistical arbitrage are deeply correlated with the prevailing market regime and the specific strategic objectives pursued. The first point of contrast emerges when comparing the raw and risk-adjusted performance of the active strategies against their passive benchmarks. This thesis found that the Buy and Hold benchmark was the absolute performance leader, delivering a cumulative return of 16.30% and a Sharpe Ratio of 1.54. The most successful active strategy, Realized Increment, while profitable, yielded a comparatively modest return of 2.17%. This outcome strongly suggests that the study's backtesting period was characterized by a persistent and broad-based bull market. In such an environment, strategies designed to be market-neutral by capturing relative value in price spreads will naturally struggle to match the raw performance of a simple long-only exposure that fully captures the market's upward beta. The primary value of the active strategies in this context shifts from outright profit generation to capital preservation and diversification.

Conversely, the study by Fanelli, Fontana, and Rotondi (2023) paints an entirely different picture. In their analysis of the crude oil futures market, the Prediction Interval (PredI) and Probability Interval (ProbI) strategies generated massive returns of 72.77% and 56.37%, respectively, far surpassing the S&P benchmark. The authors explicitly note that their test sample coincided with a “global decline of crude oil futures prices”, a market regime that is hostile to passive long investments. This environment proved to be the ideal ground for statistical arbitrage. When the overall market direction is downward or volatile, the ability to hedge out directional risk and profit solely from the mean reversion of a price spread becomes an invaluable source of alpha.

In this thesis, the market-neutral strategies were demonstrably deployed as risk-reduction tools. The empirical evidence is overwhelming: the computed Value at Risk (VaR) and Conditional Value at Risk (CVaR) figures were consistently an order of magnitude lower than those of the Buy and Hold benchmark. A 99% VaR of 1.59% for the Prediction Interval strategy, when the benchmark's was 33.49%, underscores a clear evidence for capital preservation potential. The KDE analysis complements this view, showing stable, symmetric return distributions with low kurtosis.

In contrast, the strategies implemented by Fanelli, Fontana, and Rotondi (2023) appear to be higher-risk, alpha-generating endeavors. The authors themselves concede that the “attractive performances... are mitigated by a considerable level of risk, if compared to the simple S&P strategy”. The data supports this, with the 99% VaR for their active strategies (e.g., -10.86% for ProbI) being substantially higher than their equity benchmark’s VaR (-2.80%).

Chapter 5

Conclusion

This thesis presents a substantial contribution to the domain of statistical arbitrage in international crude oil futures markets. Building on the analytical framework developed by Fanelli, Fontana, and Rotondi (2023), the study introduces an advanced Ornstein-Uhlenbeck Hidden Markov Model (OU-HMM) to enhance traditional cointegration-based strategies. This model addresses a core limitation of static frameworks, namely their inability to capture evolving market conditions, by allowing the statistical properties of the cointegration spread, specifically its long-term mean, mean-reversion speed, and volatility, to change dynamically across latent market regimes.

The empirical setting focused on four major benchmark crude oil futures: Brent, West Texas Intermediate (WTI), Dubai, and Shanghai. A comprehensive pipeline was implemented, from rigorous data preprocessing and aggregation to robust model calibration and backtesting. Shanghai futures prices were converted into USD to ensure cross-market comparability, and weekly sampling was employed to preserve relevant market dynamics while managing computational load. The final cointegration spread was constructed at daily frequency to enable precise regime detection and responsive strategy execution.

Initial econometric diagnostics confirmed that all log-price series were integrated of order one. Subsequent Johansen cointegration testing, based on a VAR (2) specification chosen via the Schwarz Information Criterion (SIC), revealed three statistically significant cointegrating vectors. A Vector Error Correction Model (VECM) was employed to account for both long- and short-term dependencies, and the primary cointegration vector was used to define the arbitrage spread.

To model regime dynamics, a K-Means-based initialization was applied to estimate the OU-HMM parameters, followed by a filter-based Expectation-Maximization (EM) algorithm for recursive parameter updates. The model successfully identified

three distinct market regimes:

- **Regime 1:** Low volatility and strong mean-reversion, typical of calm market phases.
- **Regime 2:** Moderate volatility and persistent deviations, reflecting market uncertainty.
- **Regime 3:** High volatility and significant shocks, including structural breaks such as the COVID-19 pandemic.

Numerical stability was ensured using KL-divergence regularization and parameter clipping to maintain identifiability.

Five trading strategies were tested over the period from October 2023 to March 2025, a persistently bullish phase in global energy markets. These included:

- A passive Buy-and-Hold benchmark,
- A naive mean-reversion approach (Plain Vanilla),
- A statistically driven Probability Interval (ProbI) strategy,
- Three regime-aware strategies, Predicted Increment (PI), Realized Increment (RI), and Prediction Interval (PredI).

The Buy-and-Hold strategy achieved the highest cumulative return (16.30%) and Sharpe Ratio (1.54), but incurred the highest drawdown (19.35%) and risk exposure, including a 99% Value-at-Risk (VaR) of 33.49% and Conditional VaR (CVaR) of 41.52%. In contrast, the active, market-neutral strategies generated lower absolute returns but delivered superior risk-adjusted profiles.

Specifically, the Prediction Interval strategy achieved the lowest drawdown (0.40%) and exhibited strong risk control, with a 99% VaR of just 1.59%. The Realized Increment strategy also performed well, with a 99% VaR of 1.79% and 99% CVaR of 2.28%. These results validate their utility as volatility dampening tools and robust hedging instruments in risk-conscious portfolios.

Importantly, this thesis confirms a key insight found in Fanelli, Fontana, and Rotondi (2023): the profitability and utility of statistical arbitrage strategies are deeply conditioned by prevailing market regimes. While their study, conducted during bear market phases highlighted alpha generation, this thesis, set in a bull market environment, finds the primary benefit of active strategies to lie in risk mitigation and capital preservation. This underscores the necessity and importance of regime-aware models that can adapt both their expectations and risk thresholds dynamically.

Appendix A

Code Listings

A.1 Data Preprocessing

LISTING A.1. Weekly Aggregation of Daily Series

```
1  def aggregate_weekly(df: pl.DataFrame) -> pl.DataFrame:
    # Map aggregation functions to columns
    agg_map = {
        "Close": (pl.col("Close").last(), "Close"),
        "Open": (pl.col("Open").first(), "Open"),
        "Low": (pl.col("Low").min(), "Low"),
        "High": (pl.col("High").max(), "High"),
        "Return": (pl.col("Return").sum(), "Return"),
        "LogPrice": (pl.col("LogPrice").last(), "LogPrice"),
        "Volume": (pl.col("Volume").sum(), "Volume"),
        "LogReturn": (pl.col("LogReturn").sum(), "LogReturn"),
    }
    cols = set(df.columns)
    aggs = [expr.alias(alias) for col, (expr, alias) in
agg_map.items() if col in cols]

16  return (
    df.sort("Date")
        .group_by_dynamic("Date", every="1w")
        .agg(aggs)
        .sort("Date")
21  )

# Split the data into training and testing sets
def split_train_test(dfs, test_size=0.2, shuffle=False):
    trains, tests = [], []
26  for df in dfs:
```

```

        train, test = train_test_split(df.to_pandas(),
        test_size=test_size, shuffle=shuffle)
        trains.append(train)
        tests.append(test)
        return trains, tests
31
wti_weekly, shanghai_weekly, dubai_weekly, brent_weekly, sp_weekly =
    [aggregate_weekly(df) for df in [wti, shanghai_converted, dubai,
    brent, sp]]

# Create a common calendar from Shanghai data (lowest n. of rows)
common_calendar = shanghai_weekly.select("Date").sort("Date")
36

# Align all weekly dataframes to the common calendar
for df_name in ["wti_weekly", "shanghai_weekly", "dubai_weekly",
    "brent_weekly", "sp_weekly"]:
    globals()[df_name] = align_on_calendar(common_calendar,
    globals()[df_name])

41
all_weekly = [wti_weekly, shanghai_weekly, dubai_weekly, brent_weekly,
    sp_weekly]
weekly_train, weekly_test = split_train_test(all_weekly)

```

LISTING A.2. VAR Model and SIC Computation

```

def compute_var_sic(data: np.ndarray, max_lags: int = 10) -> dict:
    """
3    Compute Schwarz Information Criterion (SIC) for VAR models with
    different lag orders.

    Parameters:
    -----
    data : np.ndarray
8        Time series data matrix where each column represents a
    different variable
    max_lags : int, default=10
        Maximum number of lags to test

    Returns:
    -----
13    dict : Dictionary mapping lag order to SIC value
    """
    model = VAR(data)
    bic_results = {}
18    for p in range(1, max_lags + 1):

```

```

        result = model.fit(p)
        bic_results[p] = result.bic
    return bic_results

23 def plot_sic(bic_results: dict) -> None:
    """
    Plot the SIC values for different lag orders to visualize model
    selection.

    Parameters:
    28 -----
    bic_results : dict
        Dictionary containing lag orders as keys and SIC values as
        values
    """
    fig = go.Figure()
    33 fig.add_trace(go.Scatter(x=list(bic_results.keys()),
                                y=list(bic_results.values()),
                                mode='lines+markers',
                                name='SIC'))
    fig.update_layout(title='SIC for VAR(p) Model',
    38                      xaxis_title='Lag Order (p)',
                      yaxis_title='SIC')
    fig.show()

    # Stack log-price arrays from training data into a matrix
    43 # Each column represents a different crude oil futures contract
    logprice_arrays = [weekly_df["LogPrice"].to_numpy() for weekly_df in
        weekly_train]
    matrix = np.column_stack(logprice_arrays)
    print(matrix.shape)

    48 # Compute SIC for different lag orders to find optimal VAR
        specification
    sic_results = compute_var_sic(matrix, max_lags=10)
    plot_sic(sic_results)

    # Fit VAR model with optimal lag order determined by minimum SIC
    53 model = VAR(matrix)
    optimal_lag = min(sic_results, key=sic_results.get)
    results = model.fit(optimal_lag)

    print(results.summary())

```

LISTING A.3. VAR Model Estimation Results Summary

```

Summary of Regression Results
=====
3  Model:                                                    VAR
   Method:                                                    OLS
   Date:                                                    Wed, 16, Jul, 2025
   Time:                                                    12:16:49
   -----
8  No. of Equations:      4.00000      BIC:                  -26.3257
   Nobs:                  284.000      HQIC:                 -26.6028
   Log likelihood:        2228.02      FPE:                 2.32290e-12
   AIC:                   -26.7883      Det (Omega_mle):      2.05038e-12
   -----
13 Results for equation y1
   =====
      coefficient      std. error      t-stat      prob
   -----
18  const          0.096539      0.062338      1.549      0.121
   L1.y1          1.414614      0.156702      9.027      0.000
   L1.y2         -0.026666      0.104088     -0.256      0.798
   L1.y3          0.376759      0.111638      3.375      0.001
   L1.y4         -0.493084      0.187010     -2.637      0.008
   L2.y1         -0.437699      0.158164     -2.767      0.006
23  L2.y2          0.041147      0.102612      0.401      0.688
   L2.y3         -0.291930      0.100455     -2.906      0.004
   L2.y4          0.394211      0.187843      2.099      0.036
   =====
28 Results for equation y2
   =====
      coefficient      std. error      t-stat      prob
   -----
33  const          0.078009      0.053947      1.446      0.148
   L1.y1          0.235942      0.135608      1.740      0.082
   L1.y2          0.624076      0.090076      6.928      0.000
   L1.y3          0.129711      0.096610      1.343      0.179
   L1.y4          0.108947      0.161836      0.673      0.501
   L2.y1         -0.199265      0.136873     -1.456      0.145
38  L2.y2          0.240257      0.088799      2.706      0.007
   L2.y3          0.022791      0.086932      0.262      0.793
   L2.y4         -0.179728      0.162557     -1.106      0.269
   =====
43 Results for equation y3
   =====

```

	coefficient	std. error	t-stat	prob

const	0.096885	0.048128	2.013	0.044
L1.y1	0.228888	0.120981	1.892	0.058
L1.y2	0.005147	0.080360	0.064	0.949
L1.y3	0.756089	0.086189	8.772	0.000
L1.y4	0.048788	0.144379	0.338	0.735
L2.y1	-0.034481	0.122109	-0.282	0.778
L2.y2	-0.004209	0.079220	-0.053	0.958
L2.y3	-0.209908	0.077555	-2.707	0.007
L2.y4	0.188268	0.145023	1.298	0.194
=====				
Results for equation y4				
=====				
	coefficient	std. error	t-stat	prob

const	0.136739	0.058055	2.355	0.019
L1.y1	0.689127	0.145934	4.722	0.000
L1.y2	0.087901	0.096935	0.907	0.365
L1.y3	0.206040	0.103966	1.982	0.048
L1.y4	0.179906	0.174159	1.033	0.302
L2.y1	-0.524529	0.147295	-3.561	0.000
L2.y2	-0.019773	0.095560	-0.207	0.836
L2.y3	-0.050222	0.093551	-0.537	0.591
L2.y4	0.403509	0.174935	2.307	0.021
=====				
Correlation matrix of residuals				
	y1	y2	y3	y4
y1	1.000000	0.706992	0.720095	0.931158
y2	0.706992	1.000000	0.607908	0.770014
y3	0.720095	0.607908	1.000000	0.732083
y4	0.931158	0.770014	0.732083	1.000000

LISTING A.4. Johansen Test for Cointegration

```

def johansen_trace_test(data, det_order=0, alpha=0.05, k_ar_diff=2):
2  result = coint_johansen(data, det_order, k_ar_diff=k_ar_diff)
    crit_idx = {0.10: 0, 0.05: 1, 0.01: 2}[alpha]

    rows = []
    for i, stat in enumerate(result.lrl1):
7         crit = result.cvt[i, crit_idx]
        h = int(stat > crit)

```

```

    pval = f"< {alpha:.2f}" if h else f"> {alpha:.2f}"
    rows.append([i, h, round(stat, 3), round(crit, 3), pval])

12 result_table = pd.DataFrame(rows, columns=["r", "h", "Trace test
    stat.", "Crit. val.", "pValue"])
    suggested_rank = result_table[result_table["h"] == 1]["r"].max() + 1

    return result, result_table, suggested_rank

```

LISTING A.5. Daily Cointegration Spread Construction

```

daily_matrix = np.column_stack([df["LogPrice"].to_numpy() for df in
    dataframes])

daily_final_spread = (daily_matrix @ beta_vec)
daily_final_spread = (daily_final_spread -
    np.mean(daily_final_spread)) / np.std(daily_final_spread)

```

A.2 OUHMM Model

LISTING A.6. OUHMM Class: Initialization and Basic Setup

```

1 class OUHMM:
    """
    Ornstein-Uhlenbeck Hidden Markov Model with regime switching.

    This class implements a Hidden Markov Model where each regime
    follows an
6    Ornstein-Uhlenbeck process with different parameters.
    """

    def __init__(self, data, n_regimes=3, random_state=42,
    burn_in_ratio=0.7):
        """
11    Initialize the OUHMM model.

    Parameters:
    -----
    data : array-like
16    Time series data to model
    n_regimes : int, default=3
        Number of hidden regimes
    random_state : int, default=42
        Random seed for reproducibility

```

```

21         burn_in_ratio : float, default=0.7
           Fraction of data to use for burn-in period
           """
           self.data = np.array(data)
           self.K = n_regimes
26         self.random_state = random_state
           self.burn_in_ratio = burn_in_ratio
           self.epsilon = 1e-6
           self.delta_t = 1
           self.S_train = None
31         self.S_test = None

           # Initialize all parameters to None
           self._reset_parameters()

36         # Prepare data splits
           self._prepare_data()

def _reset_parameters(self):
    """Reset all model parameters to None."""
41     # OU Parameters
           self.theta = None
           self.mu = None
           self.sigma = None
           self.alpha_hat = None
46         self.gamma_hat = None
           self.eta_hat = None

           # HMM Parameters
           self.transition_matrix = None
51         self.initial_probabilities = None

           # Filtering results
           self.alpha_history = None
           self.log_likelihoods = None
56         self.param_history = None

           # Clustering results
           self.labels = None
           self.per_regime_residual_variances = None

61         # Sufficient statistics
           self.ss = None

           # Additional tracking

```

```

66         self.entropy_history = None
        self.residuals = None
        self.is_fitted = False

    def _prepare_data(self):
71         """Prepare training data and burn-in period."""
        self.S_train, self.S_test = train_test_split(self.data,
        test_size=0.2, random_state=self.random_state)
        self.burn_in_period = self.S_train[:int(len(self.S_train) *
        self.burn_in_ratio)]

```

LISTING A.7. OUHMM Class: Clustering and Residual Variance

```

def _initial_clustering(self):
2     """Perform initial K-means clustering on lagged data."""
        lags = np.column_stack([self.burn_in_period[1:],
        self.burn_in_period[:-1]])
        kmeans = KMeans(n_clusters=self.K,
        random_state=self.random_state, n_init='auto')
        self.labels = kmeans.fit_predict(lags)

7     # Calculate residual variances per regime
        self.per_regime_residual_variances = []
        for k in range(self.K):
            indices = np.where(self.labels == k)[0]
            if len(indices) > 1:
12                y_t = self.burn_in_period[indices + 1]
                y_t_minus_1 = self.burn_in_period[indices]
                residuals = y_t - y_t_minus_1

        self.per_regime_residual_variances.append(np.var(residuals))
        else:
17            self.per_regime_residual_variances.append(0)

    def _initialize_parameters(self):
        """Initialize all model parameters."""
        # Perform initial clustering
22        self._initial_clustering()

        # Estimate initial parameters
        self.alpha_hat = self._estimate_alpha_hat(self.burn_in_period,
        self.labels)
        theta_hat = self._estimate_theta_hat(self.alpha_hat)
27        self.gamma_hat = self._estimate_gamma_hat(self.burn_in_period,
        self.labels, self.alpha_hat)

```

```

        mu_hat = self._estimate_mu_hat(self.gamma_hat, self.alpha_hat)
        self.eta_hat = self._estimate_eta_hat(self.burn_in_period,
self.labels, self.alpha_hat, mu_hat)
        sigma_hat = self._estimate_sigma_hat(theta_hat, self.eta_hat,
self.alpha_hat)

32     # Set OU parameters
        self.theta = theta_hat.copy()
        self.mu = mu_hat.copy()
        self.sigma = sigma_hat.copy()

37     # Estimate HMM parameters
        self.transition_matrix =
self._estimate_transition_matrix(self.labels)
        self.initial_probabilities =
self._estimate_initial_probabilities(self.labels)

        # Enforce KL separation
42     self.mu, self.sigma = self._enforce_KL_separation(self.mu,
self.sigma)

        # Initialize sufficient statistics
        self.ss = SufficientStatistics(self.K,
self.per_regime_residual_variances)

47     # Initialize tracking variables
        self.alpha_history = [self.initial_probabilities.copy()]
        self.log_likelihoods = [0, 0]
        self.residuals = []
        self.entropy_history = []

52     # Initialize parameter history
        self.param_history = {
            "theta": [self.theta.copy()],
            "mu": [self.mu.copy()],
57     "sigma": [self.sigma.copy()],
            "alpha_hat": [self.alpha_hat.copy()],
            "gamma_hat": [self.gamma_hat.copy()],
            "eta_hat": [self.eta_hat.copy()]
        }

```

LISTING A.8. OUHMM Class: Moment-Based Estimators

```

def _estimate_alpha_hat(self, S, labels):
    """Estimate alpha hat for initialization."""

```

```

alpha_hat = np.zeros(self.K)

4
    for k in range(self.K):
        indices = np.where(labels == k)[0]
        if len(indices) > 1:
            y_t = S[indices + 1]
            y_t_minus_1 = S[indices]
9
            alpha_hat[k] = np.sum(y_t * y_t_minus_1) /
np.sum(y_t_minus_1**2)
        else:
            alpha_hat[k] = 0

14
    return alpha_hat

def _estimate_theta_hat(self, alpha_hat):
    """Estimate theta hat from alpha hat."""
    return -np.log(np.clip(alpha_hat, self.epsilon, 1 -
self.epsilon)) / self.delta_t

19
def _estimate_gamma_hat(self, S, labels, alpha_hat):
    """Estimate gamma hat for initialization."""
    gamma_hat = np.zeros(self.K)

24
    for k in range(self.K):
        indices = np.where(labels == k)[0]
        if len(indices) > 1:
            y_t = S[indices + 1]
            y_t_minus_1 = S[indices]
29
            gamma_hat[k] = np.mean(y_t) - alpha_hat[k] *
np.mean(y_t_minus_1)
        else:
            gamma_hat[k] = 0

34
    return gamma_hat

def _estimate_mu_hat(self, gamma_hat, alpha_hat):
    """Estimate mu hat from gamma hat and alpha hat."""
    return gamma_hat / (1 - alpha_hat + self.epsilon)

39
def _estimate_eta_hat(self, S, labels, alpha_hat, mu_hat):
    """Estimate eta hat for initialization."""
    eta_hat = np.zeros(self.K)

44
    for k in range(self.K):
        indices = np.where(labels == k)[0]

```

```

        if len(indices) > 1:
            y_t = S[indices + 1]
            y_t_minus_1 = S[indices]
            residuals = y_t - (mu_hat[k] + alpha_hat[k] *
y_t_minus_1)
49         eta_hat[k] = np.sqrt(np.var(residuals))
        else:
            eta_hat[k] = 0

    return eta_hat

54
def _estimate_sigma_hat(self, theta_hat, eta_hat, alpha_hat):
    """Estimate sigma hat from other parameters."""
    sigma_hat = np.zeros(self.K)

    for i in range(self.K):
59         denominator = 1 - alpha_hat[i]**2
        if denominator > 0:
            sigma_hat[i] = np.sqrt(2 * theta_hat[i] *
eta_hat[i]**2 / denominator)
        else:
64         sigma_hat[i] = 0

    return sigma_hat

def _estimate_transition_matrix(self, labels):
69     """Estimate transition matrix from labels."""
    transition_matrix = np.zeros((self.K, self.K))

    for i in range(len(labels) - 1):
        current_state = labels[i]
74         next_state = labels[i + 1]
        transition_matrix[current_state, next_state] += 1

    # Normalize rows
    for i in range(self.K):
79         row_sum = np.sum(transition_matrix[i])
        if row_sum > 0:
            transition_matrix[i] /= row_sum
        else:
            transition_matrix[i] = 1.0 / self.K
84

    return transition_matrix

def _estimate_initial_probabilities(self, labels):

```

```

89     """Estimate initial state probabilities."""
    initial_probabilities = np.zeros(self.K)

    for k in range(self.K):
        initial_probabilities[k] = np.sum(labels == k) /
len(labels)

94     # Add smoothing
    initial_probabilities = 0.9 * initial_probabilities + 0.1 *
np.ones(self.K) / self.K
    initial_probabilities /= np.sum(initial_probabilities)

    return initial_probabilities

```

LISTING A.9. OU-HMM Class: KL Divergence Enforcement

```

def _compute_KL(self, mu1, sigma1, mu2, sigma2):
2     """Compute KL divergence between two normal distributions."""
    return 0.5 * (2 * np.log(sigma2 / sigma1) + (sigma1**2 + (mu1 -
mu2)**2) / sigma2**2 - 1)

def _enforce_KL_separation(self, mu, sigma, min_KL=0.2,
max_iter=10000):
    """Enforce minimum KL divergence between regimes."""
7     mu = mu.copy()
    sigma = sigma.copy()

    for _ in range(max_iter):
        KL12 = self._compute_KL(mu[0], sigma[0], mu[1], sigma[1])
        KL13 = self._compute_KL(mu[0], sigma[0], mu[2], sigma[2])
        KL23 = self._compute_KL(mu[1], sigma[1], mu[2], sigma[2])

        if min(KL12, KL13, KL23) >= min_KL:
            break

17     if KL12 < min_KL:
        mu[1] += np.sign(mu[1] - mu[0]) * 0.05
        sigma[1] += 0.05
    if KL13 < min_KL:
        mu[2] += np.sign(mu[2] - mu[0]) * 0.05
        sigma[2] += 0.05
22     if KL23 < min_KL:
        mu[2] += np.sign(mu[2] - mu[1]) * 0.05
        sigma[2] += 0.05
27

```

```

        return mu, sigma

def _compute_all_KL_divergences(self):
    """Compute all pairwise KL divergences between regimes."""
32    KL_divergences = []
    for i in range(self.K):
        for j in range(i + 1, self.K):
            kl = self._compute_KL(self.mu[i], self.sigma[i],
self.mu[j], self.sigma[j])
            KL_divergences.append(kl)
37    return KL_divergences

```

LISTING A.10. OUHMM Class: Emission Probability and Filtering

```

def _compute_emission_probabilities(self, S_t, S_t_minus_1, mu, theta,
sigma):
    """Compute emission probabilities for OU process."""
3    expected_mean = mu + (S_t_minus_1 - mu) * np.exp(-theta *
self.delta_t)
    variance = (np.maximum(sigma, self.epsilon)**2 / (2 * theta))
    * (1 - np.exp(-2 * theta * self.delta_t))
    std = np.sqrt(np.maximum(variance, self.epsilon))

    if np.isnan(expected_mean) or np.isnan(variance) or
np.isnan(std):
8        return self.epsilon

    return norm.pdf(S_t, loc=expected_mean, scale=std)

def _forward_filtering_step(self, alpha_t_minus_1,
transition_matrix, emission_probabilities):
13    """Perform forward filtering step."""
    numerator = np.dot(alpha_t_minus_1, transition_matrix) *
emission_probabilities

    # Handle NaN values
    if np.isnan(numerator).any():
18        numerator = np.nan_to_num(numerator, nan=self.epsilon)

    normalization = np.sum(numerator) + self.epsilon

    if np.isnan(normalization):
23        normalization = self.epsilon

    alpha_t = numerator / normalization

```

```

        alpha_t = np.nan_to_num(alpha_t, nan=self.epsilon,
posinf=self.epsilon, neginf=self.epsilon)

28     alpha_t_sum = np.sum(alpha_t)
        if alpha_t_sum < self.epsilon:
            alpha_t = np.full(self.K, 1.0 / self.K)
        else:
            alpha_t /= alpha_t_sum

33     return alpha_t, normalization

```

LISTING A.11. OUHMM Class: Parameter Updates and Transition Matrix

```

1  def _update_ou_parameters(self, ss, t):
    """Update OU parameters from sufficient statistics."""
    alpha_hat = np.zeros(self.K)
    gamma_hat = np.zeros(self.K)
    eta_hat = np.zeros(self.K)

6     for i in range(self.K):
        # Update alpha_hat
        numerator = ss.G[i, 2] - (ss.G[i, 0] * ss.G[i, 1]) /
(ss.O[i] + self.epsilon)
        denominator = ss.G[i, 3] - (ss.G[i, 1] ** 2) / (ss.O[i] +
self.epsilon)

11        if np.isnan(denominator) or denominator <= 0:
            alpha_hat[i] = 0
        else:
            alpha_hat[i] = numerator / (denominator + self.epsilon)
16            alpha_hat[i] = np.clip(alpha_hat[i], 0.05, 0.995)

        # Update gamma_hat
        denominator_gamma = ss.O[i] + self.epsilon
        if not np.isnan(denominator_gamma) and denominator_gamma >
0:
21            gamma_hat[i] = (ss.G[i, 0] - alpha_hat[i] * ss.G[i,
1]) / denominator_gamma
        else:
            gamma_hat[i] = 0

        # Update eta_hat
26        if not np.isnan(ss.O[i] + self.epsilon) and (ss.O[i] +
self.epsilon > 0):
            variance = ss.G[i, 4] / (ss.O[i] + self.epsilon)

```

```

        if np.isnan(variance) or variance < 0:
            eta_hat[i] = 0
        else:
31         eta_hat[i] = np.sqrt(variance)
    else:
        eta_hat[i] = 0

    # Apply smoothing
36    smoothing_rate = max(0.9 - 0.0001 * t, 0.90)
    alpha_hat = smoothing_rate * alpha_hat + (1 - smoothing_rate)
    * np.ones(self.K) / self.K

    return alpha_hat, gamma_hat, eta_hat

41 def _invert_parameters(self, alpha_hat, gamma_hat, eta_hat):
    """Invert parameters to get theta, mu, sigma."""
    # Clip alpha_hat to avoid numerical issues
    alpha_hat = np.clip(alpha_hat, self.epsilon, 1 - self.epsilon)

46    theta_hat = -np.log(alpha_hat) / self.delta_t
    mu_hat = gamma_hat / (1 - alpha_hat + self.epsilon)
    sigma_hat = np.zeros(self.K)

    for i in range(self.K):
51         denominator = 1 - alpha_hat[i]**2
        if not np.isnan(denominator) and denominator >
self.epsilon:
            val = 2 * theta_hat[i] * eta_hat[i]**2 / denominator
            if np.isnan(val) or val < 0:
                sigma_hat[i] = self.epsilon
56         else:
            sigma_hat[i] = np.sqrt(val)
        else:
            sigma_hat[i] = self.epsilon

61    theta_hat = np.clip(theta_hat, 1e-4, 10)

    return theta_hat, mu_hat, sigma_hat

66 def _update_transition_matrix(self, ss, smoothing=0.01):
    """Update transition matrix from sufficient statistics."""
    transition_matrix = np.zeros((self.K, self.K))

    for i in range(self.K):
        row_sum = np.sum(ss.J[i, :]) + self.epsilon

```

```

71         if row_sum > 0:
            raw_probs = (ss.J[i, :] + smoothing) / (row_sum +
smoothing * self.K)
            transition_matrix[i, :] = raw_probs / np.sum(raw_probs)
        else:
            transition_matrix[i, :] = 1.0 / self.K
76
    return transition_matrix

```

LISTING A.12. OUHMM Class: Fitting Loop

```

def fit(self, verbose=False):
    """
3       Fit the OUHMM model to the data.

    Parameters:
    -----
    verbose : bool, default=False
8       Whether to print progress information

    Returns:
    -----
    dict : Dictionary containing fitting results
13    """
    # Initialize parameters
    self._initialize_parameters()

    # Main fitting loop
18    for t in range(1, len(self.S_train)):
        S_t = self.S_train[t]
        S_t_minus_1 = self.S_train[t - 1]

        # Compute emission probabilities
23        emission_probabilities = np.array([
            self._compute_emission_probabilities(S_t, S_t_minus_1,
self.mu[i], self.theta[i], self.sigma[i])
            for i in range(self.K)
        ])

        emission_probabilities = np.clip(emission_probabilities,
28        1e-20, 1e2)

        # Forward filtering step
        alpha_t, normalization = self._forward_filtering_step(
            self.alpha_history[-1], self.transition_matrix,

```

```

33     emission_probabilities
        )

        # Update sufficient statistics
        self.ss.update(
            self.alpha_history[-1], self.transition_matrix,
38     emission_probabilities,
            alpha_t, S_t, S_t_minus_1, self.log_likelihoods, t,
            self.mu, self.alpha_hat
        )

        # Update OU parameters
        self.alpha_hat, self.gamma_hat, self.eta_hat =
self._update_ou_parameters(self.ss, t)
43     self.theta, self.mu, self.sigma = self._invert_parameters(
            self.alpha_hat, self.gamma_hat, self.eta_hat
        )

        # Apply parameter constraints
48     self.mu = np.clip(self.mu, -10, 100)
        self.sigma = np.clip(self.sigma, 0.2, 20)
        self.theta = np.clip(self.theta, -10, 30)

        # Update transition matrix
53     self.transition_matrix =
self._update_transition_matrix(self.ss, smoothing=0.01)

        # Enforce KL separation
        min_KL = 0.3
        KL_penalties = self._compute_all_KL_divergences()
58     if min(KL_penalties) < min_KL:
            self.mu, self.sigma =
self._enforce_KL_separation(self.mu, self.sigma, min_KL=min_KL)

        # Calculate log likelihood
        log_likelihood = (np.sum(np.log(emission_probabilities +
63     self.epsilon) * alpha_t) +
            np.log(normalization + self.epsilon))

        # Store results
        self.alpha_history.append(alpha_t)
        self.log_likelihoods.append(log_likelihood)

68     # Update parameter history
        self.param_history["theta"].append(self.theta.copy())

```

```

        self.param_history["mu"].append(self.mu.copy())
        self.param_history["sigma"].append(self.sigma.copy())
73
    self.param_history["alpha_hat"].append(self.alpha_hat.copy())

    self.param_history["gamma_hat"].append(self.gamma_hat.copy())
    self.param_history["eta_hat"].append(self.eta_hat.copy())

    # Calculate entropy
78
    H = -np.sum(alpha_t * np.log(alpha_t + self.epsilon))
    self.entropy_history.append(H)

    if verbose and t % 100 == 0:
        print(f"Iteration {t}/{len(self.S_train)-1}, Log
Likelihood: {log_likelihood:.4f}")
83

    self.is_fitted = True

    return {
        'log_likelihood': self.log_likelihoods[-1],
88
        'theta': self.theta,
        'mu': self.mu,
        'sigma': self.sigma,
        'transition_matrix': self.transition_matrix,
        'final_probabilities': self.alpha_history[-1]
93
    }

```

LISTING A.13. OUHMM Class: Prediction and Summary Utilities

```

def predict_regime(self, n_steps=1):
2
    """Predict the most likely regime for the next n steps."""
    if not self.is_fitted:
        raise ValueError("Model must be fitted before making
predictions")

    current_probs = self.alpha_history[-1]
7
    predictions = []

    for _ in range(n_steps):
        next_probs = np.dot(current_probs, self.transition_matrix)
        predicted_regime = np.argmax(next_probs)
12
        predictions.append(predicted_regime)
        current_probs = next_probs

    return predictions

```

```

17     def get_regime_summary(self):
        """Get a summary of the fitted regimes."""
        if not self.is_fitted:
            raise ValueError("Model must be fitted before getting
regime summary")

22         summary = {}
        for i in range(self.K):
            summary[f'Regime_{i+1}'] = {
                'theta': self.theta[i],
                'mu': self.mu[i],
27                 'sigma': self.sigma[i],
                'alpha_hat': self.alpha_hat[i],
                'gamma_hat': self.gamma_hat[i],
                'eta_hat': self.eta_hat[i]
            }

32         return summary

```

LISTING A.14. SufficientStatistics Class

```

class SufficientStatistics:
2     """
        Class to track sufficient statistics for the OUHMM model.
        """

    def __init__(self, K, per_regime_residual_variances):
7         """
            Initialize sufficient statistics.

            Parameters:
            -----
12         K : int
            Number of regimes
            per_regime_residual_variances : array-like
            Initial residual variances per regime
            """

17         self.J = np.zeros((K, K)) # J_t^ij
            self.O = np.zeros(K)     # O_t^i
            self.G = np.zeros((K, 5)) # G_t^i for y_t, y_t-1, y_t*y_t-1,
y_t^2, residuals^2
            self.G[:, 4] = per_regime_residual_variances
            self.epsilon = 1e-6

22

```

```

def update(self, alpha_t_minus_1, transition_matrix,
emission_probabilities,
    gamma_t, S_t, S_t_minus_1, ll_series, t, mu, alpha_hat):
    """
    Update sufficient statistics.

    Parameters:
    -----
    alpha_t_minus_1 : array
        Previous time step probabilities
    transition_matrix : array
        Transition matrix
    emission_probabilities : array
        Emission probabilities
    gamma_t : array
        Current filtered probabilities
    S_t : float
        Current observation
    S_t_minus_1 : float
        Previous observation
    ll_series : list
        Log likelihood series
    t : int
        Current time step
    mu : array
        Current mu parameters
    alpha_hat : array
        Current alpha hat parameters
    """
    # Adaptive learning rate
    rho = min(0.01, 1 / (10 + np.log(t)))

    # Squeeze observations to avoid shape issues
    S_t = np.squeeze(S_t)
    S_t_minus_1 = np.squeeze(S_t_minus_1)

    # Update O with safeguard to avoid overflow
    self.O = (1 - rho) * self.O + rho * gamma_t

    # Update J: add epsilon in denominator to avoid division by
    zero/overflow
    numerator = alpha_t_minus_1[:, None] * transition_matrix *
emission_probabilities
    normalization_factor = np.sum(numerator) + self.epsilon
    if normalization_factor > self.epsilon:

```

```

        self.J = (1 - rho) * self.J + rho * (numerator /
normalization_factor)

67         # Update the fourth column of G using clipped residuals
        for i in range(len(gamma_t)):
            residual = S_t - (mu[i] + alpha_hat[i] * S_t_minus_1)
            if np.abs(residual) > 1e3:
                residual = np.sign(residual) * 1e3
72         self.G[i, 4] = (1 - rho) * self.G[i, 4] + rho * gamma_t[i]
        * residual**2

        # Ensure G has at least 5 columns to avoid shape issues
        if self.G.shape[1] < 5:
            self.G = np.hstack((self.G, np.zeros((self.G.shape[0], 5 -
self.G.shape[1]))))

77         # Update the remaining sufficient statistics for G
        for i in range(len(gamma_t)):
            self.G[i, 0] = (1 - rho) * self.G[i, 0] + rho * gamma_t[i]
        * S_t
            self.G[i, 1] = (1 - rho) * self.G[i, 1] + rho * gamma_t[i]
        * S_t_minus_1
82         self.G[i, 2] = (1 - rho) * self.G[i, 2] + rho * gamma_t[i]
        * S_t * S_t_minus_1
            self.G[i, 3] = (1 - rho) * self.G[i, 3] + rho * gamma_t[i]
        * S_t**2

        # Update log likelihood series
87         ll = np.sum(np.log(emission_probabilities + self.epsilon) *
gamma_t) + np.log(np.sum(numerator) + self.epsilon)
        ll_series.append(ll)
        self.residuals = S_t - (mu + alpha_hat * S_t_minus_1)
        self.residuals = np.nan_to_num(self.residuals, nan=0.0,
posinf=0.0, neginf=0.0)
        self.residuals = np.clip(self.residuals, -1e3, 1e3)
92         self.residuals = self.residuals.tolist()

```

A.3 Trading Strategies

LISTING A.15. Benchmark Strategy: Buy and Hold

```
def buy_and_hold(df: pd.DataFrame, initial_capital: float = 10000):
```

```

entry_price = df['Close'].iloc[0]
3  shares = initial_capital / entry_price
equity_curve = shares * df['Close']
returns = equity_curve.pct_change().fillna(0)
cumulative_return = equity_curve.iloc[-1] / equity_curve.iloc[0] -
1
max_drawdown = (equity_curve.cummax() - equity_curve).max() /
equity_curve.cummax().max()
8  sharpe_ratio = returns.mean() / returns.std() * np.sqrt(252)
calmar_ratio = returns.mean() * 252 / max_drawdown

metrics = {
    'Cumulative Return': (cumulative_return - 1) * 100,
13    'Max Drawdown': max_drawdown * 100,
    'Sharpe Ratio': sharpe_ratio,
    'Calmar Ratio': calmar_ratio
}

18  return equity_curve, metrics

equity_curve, metrics = buy_and_hold(sp_weekly.to_pandas())

```

LISTING A.16. Plain Vanilla Strategy

```

def walk_forward_plain_vanilla_strategy(spread: np.ndarray,
                                       dates: np.ndarray,
                                       assets_test:
15  list[pl.DataFrame],
                                       beta: np.ndarray,
                                       initial_capital: float =
5  10000):
    prices_np = np.stack([df["Close"].to_numpy() for df in
assets_test], axis=1)
    dates_np = dates.to_numpy()

    portfolio = []
10    position_open = False
    entry_idx = None
    direction = None

    for i in range(1, len(spread)):
15        spread_now = spread[i]
        spread_prev = spread[i - 1]
        mean_so_far = spread[:i].mean()

```

```

    if not position_open and spread_prev < mean_so_far and
spread_now > mean_so_far:
20         entry_idx = i - 1
            position_open = True
            direction = -1

    elif not position_open and spread_prev > mean_so_far and
spread_now < mean_so_far:
25         entry_idx = i - 1
            position_open = True
            direction = 1

    elif position_open and (spread_now * direction >= 0):
        exit_idx = i

30         p_entry = prices_np[entry_idx]
        p_exit = prices_np[exit_idx]

        pos = direction * np.sign(beta)
        weights = np.abs(beta) / np.sum(np.abs(beta))
35         capital_alloc = initial_capital * weights
        units = capital_alloc / p_entry
        payout = np.sum((p_exit - p_entry) * units * pos)

        portfolio.append({
40             "entry_date": dates_np[entry_idx],
            "exit_date": dates_np[exit_idx],
            "payout": payout
        })

45         position_open = False

equity_curve = pd.Series(index=pd.to_datetime(dates_np), data=0.0)
capital = initial_capital
for trade in portfolio:
50     capital += trade["payout"]
    equity_curve.loc[pd.to_datetime(trade["exit_date"])] = capital

equity_curve = equity_curve.replace(0,
np.nan).ffill().fillna(initial_capital)
returns = equity_curve.pct_change().fillna(0)

55     cum_return = equity_curve.iloc[-1] / equity_curve.iloc[0] - 1
    max_dd = (equity_curve.cummax() - equity_curve).max() /
equity_curve.cummax().max()
    sharpe = returns.mean() / returns.std() * np.sqrt(252)
    calmar = returns.mean() * 252 / max_dd

```

```

60     metrics = {
        "Cumulative Return": cum_return,
        "Max Drawdown": max_dd,
        "Sharpe Ratio": sharpe,
65     "Calmar Ratio": calmar
    }

    return equity_curve, portfolio, metrics

70 equity_curve, portfolio, metrics =
    walk_forward_plain_vanilla_strategy(spread_daily, dates,
    [wti_test, shanghai_converted_test, dubai_test,
    brent_test], beta=beta_vec)

```

LISTING A.17. Probi Strategy

```

def walk_forward_probi_strategy(spread: np.ndarray,
                               dates: np.ndarray,
                               assets_test: list[pl.DataFrame],
                               beta: np.ndarray,
5                               z_alpha: float = 2.0,
                               window: int = 20,
                               initial_capital: float = 10000):
    prices_np = np.stack([df["Close"].to_numpy() for df in
10 assets_test], axis=1)
    dates_np = dates.to_numpy()

    portfolio = []
    position_open = False
    entry_idx = None
    direction = None

15     for i in range(window, len(spread)):
        rolling_mean = spread[i - window:i].mean()
        rolling_std = spread[i - window:i].std()

        upper_band = rolling_mean + z_alpha * rolling_std
        lower_band = rolling_mean - z_alpha * rolling_std
        spread_now = spread[i]

        if not position_open:
25             if spread_now > upper_band:
                entry_idx = i
                position_open = True

```

```

        direction = -1
    elif spread_now < lower_band:
        entry_idx = i
        position_open = True
        direction = 1
    elif position_open:
        z_score = (spread_now - rolling_mean) / rolling_std
        if abs(z_score) < z_alpha:
            exit_idx = i

        p_entry = prices_np[entry_idx]
        p_exit = prices_np[exit_idx]

        pos = direction * np.sign(beta)
        weights = np.abs(beta) / np.sum(np.abs(beta))
        capital_alloc = initial_capital * weights
        units = capital_alloc / p_entry
        payout = np.sum((p_exit - p_entry) * units * pos)

        portfolio.append({
            "entry_date": dates_np[entry_idx],
            "exit_date": dates_np[exit_idx],
            "payout": payout
        })

        position_open = False

equity_curve = pd.Series(index=pd.to_datetime(dates_np), data=0.0)
capital = initial_capital
for trade in portfolio:
    capital += trade["payout"]
    equity_curve.loc[pd.to_datetime(trade["exit_date"])] = capital

equity_curve = equity_curve.replace(0,
np.nan).ffill().fillna(initial_capital)
returns = equity_curve.pct_change().fillna(0)

cum_return = equity_curve.iloc[-1] / equity_curve.iloc[0] - 1
max_dd = (equity_curve.cummax() - equity_curve).max() /
equity_curve.cummax().max()
sharpe = returns.mean() / returns.std() * np.sqrt(252)
calmar = returns.mean() * 252 / max_dd

metrics = {
    "Cumulative Return": cum_return,

```

```

        "Max Drawdown": max_dd,
        "Sharpe Ratio": sharpe,
        "Calmar Ratio": calmar
    }

75     return equity_curve, portfolio, metrics

```

LISTING A.18. RI Strategy

```

def walk_forward_ri_strategy(spread: np.ndarray,
                             dates: np.ndarray,
                             assets_test: list[pl.DataFrame],
4                             beta: np.ndarray,
                             alpha_l: float = 0.025,
                             alpha_u: float = 0.975,
                             window: int = 20,
                             initial_capital: float = 10000):
9     prices_np = np.stack([df["Close"].to_numpy() for df in
                             assets_test], axis=1)
    dates_np = dates.to_numpy()

    x_t = (spread[1:] / spread[:-1]) - 1 # Realized increment
    x_t = np.insert(x_t, 0, 0) # Align shape with spread
14

    portfolio = []
    position_open = False
    entry_idx = None
    direction = None
19

    for i in range(window, len(spread)):
        roll_x = x_t[i - window:i]
        q_low = np.quantile(roll_x, alpha_l)
        q_up = np.quantile(roll_x, alpha_u)
24        x_now = x_t[i]

        if not position_open:
            if x_now < q_low:
                entry_idx = i
                position_open = True
                direction = 1
            elif x_now > q_up:
                entry_idx = i
                position_open = True
                direction = -1
34        elif position_open:

```

```

        if q_low <= x_now <= q_up:
            exit_idx = i

39         p_entry = prices_np[entry_idx]
            p_exit = prices_np[exit_idx]

            pos = direction * np.sign(beta)
            weights = np.abs(beta) / np.sum(np.abs(beta))
44         capital_alloc = initial_capital * weights
            units = capital_alloc / p_entry
            payout = np.sum((p_exit - p_entry) * units * pos)

            portfolio.append({
49                 "entry_date": dates_np[entry_idx],
                 "exit_date": dates_np[exit_idx],
                 "payout": payout
            })

54         position_open = False

equity_curve = pd.Series(index=pd.to_datetime(dates_np), data=0.0)
capital = initial_capital
for trade in portfolio:
59     capital += trade["payout"]
    equity_curve.loc[pd.to_datetime(trade["exit_date"])] = capital

equity_curve = equity_curve.replace(0,
np.nan).ffill().fillna(initial_capital)
returns = equity_curve.pct_change().fillna(0)

64     cum_return = equity_curve.iloc[-1] / equity_curve.iloc[0] - 1
    max_dd = (equity_curve.cummax() - equity_curve).max() /
equity_curve.cummax().max()
    sharpe = returns.mean() / returns.std() * np.sqrt(252)
    calmar = returns.mean() * 252 / max_dd

69     metrics = {
        "Cumulative Return": cum_return,
        "Max Drawdown": max_dd,
        "Sharpe Ratio": sharpe,
74         "Calmar Ratio": calmar
    }

    return equity_curve, portfolio, metrics

```

LISTING A.19. PredI

```

def walk_forward_prediction_interval_strategy_online(
    spread: np.ndarray,
3    dates: np.ndarray,
    assets_test: list[pl.DataFrame],
    beta: np.ndarray,
    alpha_0: np.ndarray,
    transition_matrix_init: np.ndarray,
8    mu_init: np.ndarray,
    theta_init: np.ndarray,
    sigma_init: np.ndarray,
    initial_ss: SufficientStatistics,
    q_alpha: float = 1.96,
13    initial_capital: float = 10000,
    epsilon: float = 1e-6
):
    prices_np = np.stack([df["Close"].to_numpy() for df in
    assets_test], axis=1)
    dates_np = dates.to_numpy()

18
    K = len(alpha_0)
    alpha_t = alpha_0.copy()
    mu = mu_init.copy()
    theta = theta_init.copy()
23    sigma = sigma_init.copy()
    transition_matrix = transition_matrix_init.copy()
    ss = initial_ss

    portfolio = []
28    position_open = False
    entry_idx = None
    direction = None

    for t in range(1, len(spread)):
33        S_t = spread[t]
        S_t_minus_1 = spread[t - 1]

        # === Forward step ===
        emission_probs = np.array([
38            compute_emission_probabilities(S_t, S_t_minus_1, mu[i],
            theta[i], sigma[i])
            for i in range(K)
        ])
        emission_probs = np.clip(emission_probs, 1e-20, 1e2)

```

```

43     alpha_t, _ = forward_filtering_step(alpha_history[-1],
transition_matrix, emission_probs)
    alpha_history.append(alpha_t)
    z_hat = np.argmax(alpha_t)

    # === Conditional forecast ===
48     alpha_hat = param_history['alpha_hat'][-1] # optional if you
want smoothed init
    gamma_hat = param_history['gamma_hat'][-1]
    eta_hat = param_history['eta_hat'][-1]

    alpha_reg = alpha_hat[z_hat]
53     gamma_reg = gamma_hat[z_hat]
    eta_reg = eta_hat[z_hat]

    E_st = gamma_reg + alpha_reg * S_t_minus_1
    std = eta_reg
58     lower_bound = E_st - q_alpha * std
    upper_bound = E_st + q_alpha * std

    # === Entry and exit logic ===
    if not position_open:
63         if S_t < lower_bound:
            entry_idx = t
            position_open = True
            direction = 1
        elif S_t > upper_bound:
68         entry_idx = t
            position_open = True
            direction = -1
    elif position_open:
        if lower_bound <= S_t <= upper_bound:
73         exit_idx = t
            p_entry = prices_np[entry_idx]
            p_exit = prices_np[exit_idx]

            pos = direction * np.sign(beta)
78         weights = np.abs(beta) / np.sum(np.abs(beta))
            capital_alloc = initial_capital * weights
            units = capital_alloc / p_entry
            payout = np.sum((p_exit - p_entry) * units * pos)

83     portfolio.append({
        "entry_date": dates_np[entry_idx],

```

```

        "exit_date": dates_np[exit_idx],
        "payout": payout
    })

    position_open = False

    # === Online model update ===
    ss.update(alpha_t, transition_matrix, emission_probs, alpha_t,
S_t, S_t_minus_1, [0]) # ll_series dummy
    alpha_hat, gamma_hat, eta_hat = update_ou_parameters(ss)
    theta, mu, sigma = invert_parameters(alpha_hat, gamma_hat,
eta_hat)

    mu = np.clip(mu, -10, 100)
    sigma = np.clip(sigma, 0.2, 20)
    theta = np.clip(theta, 1e-4, 30)

    transition_matrix = update_transition_matrix(ss,
smoothing=0.001)

    # Save history if needed (optional)
    param_history['alpha_hat'].append(alpha_hat.copy())
    param_history['gamma_hat'].append(gamma_hat.copy())
    param_history['eta_hat'].append(eta_hat.copy())

    # === Equity curve ===
    equity_curve = pd.Series(index=pd.to_datetime(dates_np), data=0.0)
    capital = initial_capital
    for trade in portfolio:
        capital += trade["payout"]
        equity_curve.loc[pd.to_datetime(trade["exit_date"])] = capital

    equity_curve = equity_curve.replace(0,
np.nan).ffill().fillna(initial_capital)
    returns = equity_curve.pct_change().fillna(0)

    cum_return = equity_curve.iloc[-1] / equity_curve.iloc[0] - 1
    max_dd = (equity_curve.cummax() - equity_curve).max() /
equity_curve.cummax().max()
    sharpe = returns.mean() / returns.std() * np.sqrt(252)
    calmar = returns.mean() * 252 / max_dd

    metrics = {
        "Cumulative Return": cum_return,
        "Max Drawdown": max_dd,

```

```

        "Sharpe Ratio": sharpe,
        "Calmar Ratio": calmar
    }

128     return equity_curve, portfolio, metrics

filtered_regimes = None

133 def get_filtered_regimes(S, alpha_0, P, mu, theta, sigma):
    filtered_regimes = []
    alpha_t = alpha_0.copy()
    for t in range(1, len(S)):
        emission_probs = np.array([
138             compute_emission_probabilities(S[t], S[t-1], mu[i],
            theta[i], sigma[i])
            for i in range(len(alpha_t))
        ])
        emission_probs = np.clip(emission_probs, 1e-20, 1e2)
        alpha_t, _ = forward_filtering_step(alpha_t, P, emission_probs)
143         regime = np.argmax(alpha_t)
        filtered_regimes.append(regime)
    return np.array(filtered_regimes)

alpha_0 = alpha_history[-1]
148 alphas = param_history['alpha_hat'][-1]
gammas = param_history['gamma_hat'][-1]
etas = param_history['eta_hat'][-1]
P = transition_matrix
mu_test, theta_test, sigma_test = mu, theta, sigma

153 filtered_regimes = get_filtered_regimes(spread_daily, alpha_0, P,
    mu_test, theta_test, sigma_test)

```

LISTING A.20. PI Strategy

```

1 def walk_forward_predicted_increment_strategy_online(
    spread: np.ndarray,
    dates: np.ndarray,
    assets_test: list[pl.DataFrame],
    beta: np.ndarray,
6     alpha_0: np.ndarray,
    mu_init: np.ndarray,
    theta_init: np.ndarray,
    sigma_init: np.ndarray,
    P_init: np.ndarray,

```

```

11     q_lower: float = 0.025,
        q_upper: float = 0.975,
        initial_capital: float = 10000
    ):
        K = len(alpha_0)
16     prices_np = np.stack([df["Close"].to_numpy() for df in
assets_test], axis=1)
        dates_np = dates.to_numpy()

        portfolio = []
        position_open = False
21     entry_idx = None
        direction = None

        # Historical predicted increments for quantile thresholding
        predicted_increments = []

26
        def compute_emission_probabilities(S_t, S_t_prev, mu, theta,
sigma, delta_t=1, epsilon=1e-6):
            expected_mean = mu + (S_t_prev - mu) * np.exp(-theta * delta_t)
            variance = (np.maximum(sigma, epsilon)**2 / (2 * theta)) * (1
- np.exp(-2 * theta * delta_t))
            std = np.sqrt(np.maximum(variance, epsilon))
31         return norm.pdf(S_t, loc=expected_mean, scale=std)

        def forward_filtering_step(alpha_t_minus_1, transition_matrix,
emission_probabilities, epsilon=1e-6):
            numerator = np.dot(alpha_t_minus_1, transition_matrix) *
emission_probabilities
            normalization = np.sum(numerator) + epsilon
36         alpha_t = numerator / normalization
            return alpha_t, normalization

        alpha_t = alpha_0.copy()
        mu, theta, sigma = mu_init.copy(), theta_init.copy(),
sigma_init.copy()
41     P = P_init.copy()

        for t in range(1, len(spread) - 1):
            S_t, S_t_prev = spread[t], spread[t - 1]

46
            emission_probs = np.array([
                compute_emission_probabilities(S_t, S_t_prev, mu[i],
theta[i], sigma[i])
                for i in range(K)

```

```

    ])
    emission_probs = np.clip(emission_probs, 1e-20, 1e2)
51    alpha_t, _ = forward_filtering_step(alpha_t, P, emission_probs)
    z_hat = np.argmax(alpha_t)

    # Compute predicted increment
    E_S_tp1 = mu[z_hat] + (S_t - mu[z_hat]) * np.exp(-theta[z_hat])
56    x_hat = E_S_tp1 / S_t - 1
    predicted_increments.append(x_hat)

    if t < 30:
        continue # avoid trading on insufficient history

61

    lower_q = np.quantile(predicted_increments[:-1], q_lower)
    upper_q = np.quantile(predicted_increments[:-1], q_upper)

    if not position_open and (x_hat < lower_q or x_hat > upper_q):
66        entry_idx = t
        position_open = True
        direction = 1 if x_hat > upper_q else -1

    elif position_open and (lower_q <= x_hat <= upper_q):
71        exit_idx = t
        p_entry = prices_np[entry_idx]
        p_exit = prices_np[exit_idx]

        pos = direction * np.sign(beta)
76        weights = np.abs(beta) / np.sum(np.abs(beta))
        capital_alloc = initial_capital * weights
        units = capital_alloc / p_entry
        payout = np.sum((p_exit - p_entry) * units * pos)

81        portfolio.append({
            "entry_date": dates_np[entry_idx],
            "exit_date": dates_np[exit_idx],
            "payout": payout
        })

86

    position_open = False

equity_curve = pd.Series(index=pd.to_datetime(dates_np), data=0.0)
capital = initial_capital
91 for trade in portfolio:
    capital += trade["payout"]
    equity_curve.loc[pd.to_datetime(trade["exit_date"])] = capital

```

```
equity_curve = equity_curve.replace(0,  
np.nan).ffill().fillna(initial_capital)  
96 returns = equity_curve.pct_change().fillna(0)  
  
cum_return = equity_curve.iloc[-1] / equity_curve.iloc[0] - 1  
max_dd = (equity_curve.cummax() - equity_curve).max() /  
equity_curve.cummax().max()  
101 sharpe = returns.mean() / returns.std() * np.sqrt(252)  
calmar = returns.mean() * 252 / max_dd  
  
metrics = {  
    "Cumulative Return": cum_return,  
    "Max Drawdown": max_dd,  
    "Sharpe Ratio": sharpe,  
    "Calmar Ratio": calmar  
106 }  
  
return equity_curve, portfolio, metrics, predicted_increments
```

Bibliography

- Ang, Andrew and Allan Timmermann (2012). “Regime Changes in Financial Markets”. In: *Handbook of Financial Econometrics and Statistics*. Springer, pp. 1067–1104. DOI: [10.1007/978-1-4419-9865-7_19](https://doi.org/10.1007/978-1-4419-9865-7_19).
- Artzner, Philippe et al. (1999). “Coherent measures of risk”. In: *Mathematical Finance* 9.3, pp. 203–228. DOI: [10.1111/1467-9965.00068](https://doi.org/10.1111/1467-9965.00068).
- Baillie, Richard T and Tim Bollerslev (1996). “Analysis of long-run exchange rate behavior: Re-examination of the purchasing power parity hypothesis”. In: *Journal of International Money and Finance* 15.6, pp. 681–708.
- Barron, A. R. and T. J. Barron (1994). *Time-Varying Parameters in Econometrics*. Cambridge Univ. Press.
- Baum, Leonard E and Ted Petrie (1966). “Statistical inference for probabilistic functions of finite state Markov chains”. In: *The Annals of Mathematical Statistics* 37.6, pp. 1554–1563. DOI: [10.1214/aoms/1177699147](https://doi.org/10.1214/aoms/1177699147).
- Behme, A., P. Di Tella, and A. Sideris (2024). “Volatility Modeling in a Markovian Environment”. In: *Stochastic Processes and Their Applications*.
- Bibby, Bo and Michael Sørensen (2004). “A diffusion-type model for the short-term interest rate”. In: *Finance and Stochastics* 8.4, pp. 503–528. DOI: [10.1007/s00780-004-0134-x](https://doi.org/10.1007/s00780-004-0134-x).
- Brandt, Michael W. (2010). *Applied Financial Econometrics*. Wiley.
- Bulla, Jan and Ines Bulla (2006). “Stylized facts of financial time series and hidden semi-Markov models”. In: *Computational Statistics & Data Analysis* 51.4, pp. 2192–2209. DOI: [10.1016/j.csda.2006.02.009](https://doi.org/10.1016/j.csda.2006.02.009).
- Burgess, Andrew N. (1999). “Profitability of Pairs Trading Strategy in the UK Equity Market”. In: *European Journal of Finance* 5.1, pp. 35–46.
- Cappé, Olivier and Éric Moulines (2009). “On-line expectation–maximization algorithm for latent data models”. In: *Journal of the Royal Statistical Society: Series B (Statistical Methodology)* 71.3, pp. 593–613. DOI: [10.1111/j.1467-9868.2008.00694.x](https://doi.org/10.1111/j.1467-9868.2008.00694.x).

- Chen, K. (2019). “Time-Consistent Mean-Variance Pairs Trading under Regime Switching”. In: *SIAM Journal on Financial Mathematics*.
- Chen, P. (2023). “Optimal Multi-Asset OU Pairs Trading via HJB Control”. In: *Nonlinear Analysis: Control, Optimization*.
- Chevallier, Julien (2011). *Econometric Analysis of Carbon Markets: The European Union Emissions Trading Scheme and the Clean Development Mechanism*. Springer.
- Cicmilović, D. (2025). “Hidden Markov Model for Correlated Ornstein-Uhlenbeck Processes”. In: *Applied Stochastic Modelling*.
- Dickey, David A and Wayne A Fuller (1979). “Distribution of the estimators for autoregressive time series with a unit root”. In: *Journal of the American statistical association* 74.366a, pp. 427–431.
- Dunis, Christian L., Jason Laws, and Ben Evans (2006). “Trading futures spreads: An application of correlation and threshold filters”. In: *Applied Financial Economics* 16.12, pp. 903–914. DOI: [10.1080/09603100500426432](https://doi.org/10.1080/09603100500426432).
- Elliott, Graham, Thomas J Rothenberg, and James H Stock (1996). “Efficient tests for an autoregressive unit root”. In: *Econometrica* 64.4, pp. 813–836.
- Elliott, Robert J, Lakhdar Aggoun, and John B Moore (1995). *Hidden Markov models: estimation and control*. Springer Science & Business Media.
- Elliott, Robert J. et al. (2005). “Tracking and trading a dynamic portfolio with a hidden Markov model”. In: *Journal of Economic Dynamics and Control* 29.12, pp. 2259–2279. DOI: [10.1016/j.jedc.2004.10.007](https://doi.org/10.1016/j.jedc.2004.10.007).
- Endres, S. and J. Stübinger (2018). *A Flexible Regime-Switching Model with Pairs Trading*. Tech. rep. FAU Erlangen-Nürnberg.
- Engle, Robert F and Clive WJ Granger (1987). “Co-integration and error correction: representation, estimation, and testing”. In: *Econometrica* 55.2, pp. 251–276.
- Erlwein, Christine and Rogemar S. Mamon (2006). “Regime switching in mean-reverting processes: modelling and estimation”. In: *Quantitative Finance* 6.5, pp. 423–431.
- Fama, Eugene F. (1970). “Efficient Capital Markets: A Review of Theory and Empirical Work”. In: *The Journal of Finance* 25.2, pp. 383–417. DOI: [10.2307/2325486](https://doi.org/10.2307/2325486).
- Fanelli, Viviana, Claudio Fontana, and Francesco Rotondi (2023). “A Hidden Markov Model for Statistical Arbitrage in International Crude Oil Futures Markets”. In: *arXiv preprint arXiv:2309.00875*. Revised September 2024. URL: <https://arxiv.org/abs/2309.00875>.
- Gardiner, C. W. (1985). *Handbook of Stochastic Methods*. Springer.

- Gatev, Evan, William N Goetzmann, and K Geert Rouwenhorst (2006). “Pairs trading: Performance of a relative-value arbitrage rule”. In: *Review of Financial Studies* 19.3, pp. 797–827.
- Gregory, Allan W and Bruce E Hansen (1996). “Testing for cointegration using the Engle-Granger method: some Monte Carlo results”. In: *Oxford Bulletin of Economics and Statistics* 58.3, pp. 555–560.
- Hamilton, James D (1989). “A new approach to the economic analysis of nonstationary time series and the business cycle”. In: *Econometrica* 57.2, pp. 357–384. DOI: [10.2307/1912559](https://doi.org/10.2307/1912559).
- Hull, John C. (2017). *Options, Futures, and Other Derivatives*. 10th. Boston: Pearson.
- Johansen, Søren (1988). “Statistical analysis of cointegration vectors”. In: *Journal of Economic Dynamics and Control* 12.2-3, pp. 231–254.
- (1991). “Estimation and hypothesis testing of cointegration vectors in Gaussian vector autoregressive models”. In: *Econometrica* 59.6, pp. 1551–1580.
- Kwiatkowski, Denis et al. (1992). “Testing the null hypothesis of stationarity against the alternative of a unit root”. In: *Journal of econometrics* 54.1-3, pp. 159–178.
- Leeds, Mark (2012). “Bollinger Bands Thirty Years Later”. In: *arXiv preprint arXiv:1212.4890*. URL: <https://arxiv.org/abs/1212.4890>.
- Lewis, Alan L. (2000). *Option Valuation under Stochastic Volatility: With Mathematica Code*. Newport Beach, CA: Finance Press.
- Lo, Andrew W (2002). “The statistics of Sharpe ratios”. In: *Financial Analysts Journal* 58.4, pp. 36–52.
- Ngo, M. M. and H. Pham (2014). “Optimal Switching for Pairs Trading Rules”. In: *arXiv preprint arXiv:1412.7649*.
- Phillips, Peter CB and Pierre Perron (1988). “Testing for a unit root in time series regression”. In: *Biometrika* 75.2, pp. 335–346.
- Rabiner, Lawrence R (1989). “A tutorial on hidden Markov models and selected applications in speech recognition”. In: *Proceedings of the IEEE* 77.2, pp. 257–286.
- Reuters (2018). “Shanghai Exchange Plans China’s First Crude Oil Futures”. In: *Reuters*. Accessed: 2025-05-25. URL: <https://www.reuters.com/article/markets/commodities/shanghai-exchange-plans-chinas-first-crude-oil-futures-idUSL4N1PZ3ST>.
- Robert, Christian P. and George Casella (2004). *Monte Carlo Statistical Methods*. 2nd. Springer Texts in Statistics. Springer. DOI: [10.1007/978-1-4757-4145-2](https://doi.org/10.1007/978-1-4757-4145-2).
- Schwarz, Gideon (1978). “Estimating the dimension of a model”. In: *The Annals of Statistics* 6.2, pp. 461–464.

- Suchato, J. et al. (2024). “An Application of the Ornstein-Uhlenbeck Process to Pairs Trading”. In: *arXiv preprint arXiv:2412.12458*.
- Uhlenbeck, George E and Leonard S Ornstein (1930). “On the theory of the Brownian motion”. In: *Physical Review* 36.5, p. 823.
- Vasicek, Oldrich (1977). “An equilibrium characterization of the term structure”. In: *Journal of Financial Economics* 5.2, pp. 177–188.
- Vidyamurthy, Ganapathy (2004). *Pairs Trading: Quantitative Methods and Analysis*. John Wiley & Sons.
- Yau, Christopher Y et al. (2011). “Bayesian inference for hidden Markov models via sequential Monte Carlo methods”. In: *Statistical Science* 26.1, pp. 116–129. DOI: [10.1214/10-STS326](https://doi.org/10.1214/10-STS326).
- Young, Terry W (1991). “Calmar ratio: A smoother tool”. In: *Futures* 20, pp. 40–41.